

Where everything is

A map of Analyst to Architect: every chapter, what it teaches, how long it takes, and what you practise it with. Generated from the manuscript, so it cannot drift from the book.

How to use this

Three ways in, depending on what you are looking for.

- **"Which book do I open?"** — the table in *The four books* below.
- **"Where is topic X taught?"** — *The topic index* at the end. It is alphabetical, built from every chapter's own Key terms list, and points at the chapter.
- **"What do I actually do in this chapter?"** — the part-by-part tables. Each chapter names the practice files that ship with it.

The book is **87 chapters**, about **1,349,957 words**. **49** chapters have a worked notebook and **14** ship runnable SQL in both PostgreSQL and MySQL.

The four books

Book	Parts	Chapters	What it is for
Book 1 • Theory	0, 1	9	The ideas, with no software to install. Read it first.
Book 2 • Practical	2, 3	25	The analyst's tools, hands on: spreadsheets, SQL, cleaning, charts, BI, Python, statistics, then the advanced layer.
Book 3 • Implementation	4, 5, 6, 7, closing	34	Building real systems: machine learning, data engineering, production ML and generative AI, architecture and leadership.
Book 4 • Be Interview Ready	8	19	The question banks, the extra-points method, take-homes and mock interviews.

Part 0 — First Principles: Data from Zero

Book 1 • Theory · 6 chapters · roughly 21 hours in total

Ch	Chapter	Time	What you practise with
1	What Is Data?	3–4 hours	<i>reading only</i>
2	How Computers Store, Move and Protect Data	3–4 hours	Excel workbook, Python script, data (json), dataset (csv), dataset (parquet)
3	How a Business Runs on Data	3–4 hours	<i>reading only</i>
4	Numbers Without Fear	4–5 hours	Excel workbook, Python script
5	Thinking Like an Analyst	3–4 hours	<i>reading only</i>
6	Planning Your Learning	2–3 hours	Python script

Chapter 1. What Is Data?

You will learn to: explain what data is, and how it differs from information, knowledge, and insight · turn everyday records, like a shop receipt, into rows and columns · name the type of any value (number, text, date, true/false) and spot numbers that aren't really numbers · tell qualitative from quantitative data, and discrete from continuous · use the four levels of measurement to decide which calculations make sense · recognize structured, semi-structured, and unstructured data · read and write metadata · say where data comes from · judge whether a small dataset is fit to use.

Before you start: nothing. This chapter assumes no technical knowledge at all.

Time needed: 3–4 hours, including the exercises and the project.

Sections: 1.1 What data actually is · 1.2 From data to information, knowledge, and insight · 1.3 Records, fields, and datasets · 1.4 Kinds of values: data types · 1.5 Qualitative and quantitative data · 1.6 Levels of measurement: which math is allowed · 1.7 Structured, semi-structured, and unstructured data · 1.8 Metadata: data about data · 1.9 Where data comes from · 1.10 Data quality in one page

Chapter 2. How Computers Store, Move and Protect Data

You will learn to: explain how a computer stores letters and numbers as bits and bytes · read file sizes from bytes to terabytes, and work out how long a download takes · tell memory from storage · work with files, folders, paths, and extensions without surprises · choose between CSV, Excel, JSON, XML, PDF, and Parquet for a job, and avoid each format's traps · explain what a database, a server, the internet, and the cloud are · describe what an API does and read its replies · protect data with good passwords, encryption, access rules, and backups.

Before you start: Chapter 1 (what data is: rows, columns, types, and quality).

Time needed: 3–4 hours, including the exercises and the project.

Sections: 2.1 Bits and bytes: how computers store everything · 2.2 How big is big? From kilobytes to terabytes · 2.3 Memory and storage: the desk and the shelves · 2.4 Files, folders, paths, and extensions · 2.5 Data file formats: the same data, packed five ways · 2.6 Databases: data that many people can use at once · 2.7 Servers, the internet, and the cloud · 2.8 APIs: how systems talk to each other · 2.9 Keeping data safe

In companion/ch02/ — *Python script:* `api_demo.py` · *Excel workbook:* `orders_feb_2026.xlsx` · *dataset (csv):* `orders_feb_2026.csv` · *dataset (parquet):* `orders_feb_2026.parquet`

Chapter 3. How a Business Runs on Data

You will learn to: name a company's departments and the data each creates · follow one order from enquiry to cash, and say which system records each step · explain what ERP, CRM, HRMS, POS, e-commerce, and support systems do · tell a transaction from a report, and bookings from billings from collections · define a KPI so two people calculate the same number · describe how dashboards, meetings, and decision rights turn data into decisions · find where people copy, re-type, and email data by hand, and estimate the cost.

Before you start: Chapter 1 (rows, columns, grain, data quality) and Chapter 2 (files, databases, and APIs).

Time needed: 3–4 hours, including the exercises and the project.

Sections: 3.1 The departments of a company and the data they create · 3.2 Following one order from enquiry to cash · 3.3 Business systems: ERP, CRM, HRMS, POS, and e-commerce · 3.4 Transactions and reports: booked, billed, and collected · 3.5 What a KPI is · 3.6 Dashboards, meetings, and who decides what · 3.7 Where manual work hides

Chapter 4. Numbers Without Fear

You will learn to: calculate percentages forward and backward, and see why a rise and an equal fall don't cancel · tell percentage points from percent change · choose the right denominator for a ratio or rate · measure growth month by month and year by year, and calculate compound growth and CAGR · choose between mean, median, and mode, and use weighted averages · round without changing the story · read tables and charts without being fooled · think about probability as "how often, out of how many" · estimate quickly and sanity-check any number · spot the number tricks in business news.

Before you start: Chapter 1 (especially section 1.6, levels of measurement) and Chapter 3 (bookings, billings, KPIs).

Time needed: 4–5 hours, including the exercises and the project.

Sections: 4.1 Percentages: of, back, and change · 4.2 Percentage points and percent change · 4.3 Ratios and rates: always ask about the denominator · 4.4 Growth, compounding, and CAGR · 4.5 Averages: mean, median, mode, and weighted · 4.6 Rounding and significant figures · 4.7 Reading tables and charts correctly · 4.8 Probability: how often, out of how many · 4.9 Orders of magnitude and quick estimation · 4.10 Number tricks in business news

In companion/ch04/ — *Python script:* `make_ch04_workbook.py` · *Excel workbook:* `numbers_practice.xlsx`

Chapter 5. Thinking Like an Analyst

You will learn to: ask the questions that turn a request into useful analysis · rewrite a vague request as a precise question tied to a decision · write hypotheses that data can prove wrong · break a problem into an issue tree whose branches are MECE (no overlaps, no gaps) · tell facts from opinions and assumptions · check a claim or chart before believing it, including correlation that isn't causation · recognize the biases that bend how people read data, including your own · structure a decision so that data can inform it.

Before you start: Chapter 1 (the data-to-insight ladder), Chapter 3 (booked, billed, and collected), and Chapter 4 (percentages, averages, and small numbers).

Time needed: 3–4 hours, including the exercises and the project.

Sections: 5.1 Curiosity and asking good questions · 5.2 From a vague request to a precise question · 5.3 Hypotheses: possible answers you can test · 5.4 Breaking problems down: issue trees and MECE · 5.5 Fact, opinion, and assumption · 5.6 Checking claims and charts · 5.7 Bias in how we see data · 5.8 Deciding with data

Chapter 6. Planning Your Learning

You will learn to: estimate honestly how many hours this book takes, and turn them into weeks at your real pace · build a weekly rhythm you can keep, and recover from a missed week · check whether your computer is ready, and know which chapter brings each tool · read an official page to settle a question for yourself · use AI assistants to learn faster without letting them do your thinking · plan your route and your first 90 days.

Before you start: Chapters 1–5, and "How to Use This Book" at the front of the book.

Time needed: 2–3 hours, including the exercises and the project.

Sections: 6.1 How long it really takes · 6.2 A weekly rhythm you can keep · 6.3 What you'll need, and when · 6.4 Reading documentation · 6.5 Learning with AI assistants without letting them think for you

In companion/ch06/ — *Python script:* `check_setup.py`

Part 1 — The Map

Book 1 • Theory · 3 chapters · roughly 8 hours in total

Ch	Chapter	Time	What you practise with
7	The Data Landscape	2–3 hours	<i>reading only</i>
8	The Career Tree: How Skills Unlock Roles	3–4 hours	Excel workbook
9	How Expertise Actually Forms	2–3 hours	Excel workbook

Chapter 7. The Data Landscape

You will learn to: name the four questions every data job answers, and place any job title under one of them · describe what each of the ten main data roles does, what it produces, and which tools it uses · explain why the field grows like a tree, with a shared trunk and branches that rejoin at the top · recognize the automation and integration track, and the automation hidden inside every other role · compare centralized, embedded, and hub-and-spoke data teams, and say when each fits · follow one business request through every role, from the first question to a running platform · judge what AI assistants change in each role, and what stays your responsibility.

Before you start: Chapter 1 (what data is) and Chapter 2 (files, databases, servers, and APIs). Chapter 3 (how a business runs on data) helps but isn't required.

Time needed: 2–3 hours, including the exercises and the project.

Sections: 7.1 The four questions every data job answers · 7.2 The tracks and the ten roles · 7.3 The shape of the field: a trunk that branches · 7.4 The automation and integration track · 7.5 How data teams are organized · 7.6 One request, every role · 7.7 How AI assistants are changing each role · 7.8 Where you are right now

Chapter 8. The Career Tree: How Skills Unlock Roles

You will learn to: use the "keys and doors" idea to decide what to learn next · name the seven tiers of the career tree and the roles each one unlocks · read a skills matrix to see what each role really needs · picture a normal working day in each of the ten roles · decode a job description line by line and score your own fit · read salary figures carefully, and check them yourself · choose a reading pathway through this book for your goal · plan an entry route as a fresher, a career switcher, or through an internal move.

Before you start: Chapter 7 (the four questions, the ten roles, and how teams are organized).

Time needed: 3–4 hours, including the exercises. Allow another 2–3 hours for the project, which uses real job postings.

Sections: 8.1 Skills are keys; roles are doors · 8.2 The tiers, and what each unlocks · 8.3 The skills matrix: what each role really needs · 8.4 A day in the life of each role · 8.5 Decoding a job description · 8.6 What the roles pay, and how to read salary figures · 8.7 Reader pathways: your route through this book · 8.8 Entry routes: fresher, career switcher, internal move · 8.9 Using the tree: pick your door, then learn backward

In companion/ch08/ — *Excel workbook:* `skills_matrix.xlsx`

Chapter 9. How Expertise Actually Forms

You will learn to: estimate how long a stage of your learning will take, from hours and your real weekly schedule · explain why a long timeline is an advantage, not a punishment · combine the three ingredients of expertise: study, projects, and feedback over time · design deliberate practice sessions instead of hours of passive study · build a portfolio from the work you do while learning · find feedback, peers, and mentors, and ask for help in a way people say yes to · recognize a plateau in your own practice log and change what you do about it.

Before you start: Chapter 8 (the career tree, and choosing your next door).

Time needed: 2–3 hours, including the exercises. The project runs alongside your learning for 12 weeks.

Sections: 9.1 The honest timeline · 9.2 Why the long timeline is good news · 9.3 The three ingredients of expertise · 9.4 Deliberate practice · 9.5 Building a portfolio as you learn · 9.6 Finding feedback and mentors · 9.7 Handling plateaus · 9.8 What this means for how you use this book

In `companion/ch09/` — *Excel workbook:* `practice_log_template.xlsx`

Part 2 — The Analyst

Book 2 • Practical · 18 chapters · roughly 383 hours in total

Ch	Chapter	Time	What you practise with
10	Spreadsheet Fundamentals: Excel & Google Sheets	18–22 hours	3 Excel workbook, Python script, dataset (csv)
11	The Spreadsheet, Mastered: Excel & Google Sheets	35–45 hours	6 Excel workbook, Python script, 13 dataset (csv)
12	Databases & SQL Foundations	22–26 hours	2 SQL
13	SQL for Real Analysis	15–20 hours	2 Python script, 4 SQL
14	Data Cleaning & Preparation	20–25 hours	4 Python script, 11 SQL, data (json), 10 dataset (csv)
15	Data Visualization Principles	12–15 hours	Excel workbook, Python script, 2 SQL, 14 dataset (csv), notes
16	Business Intelligence with Power BI	22–26 hours	4 SQL, 2 dataset (csv), 2 notes
17	Python from Zero	28–32 hours	3 Python script, 2 data (json), 15 dataset (csv), notebook, 2 text
18	Python for Analysts: pandas & Automation	47–53 hours	2 Excel workbook, 6 Python script, data (json), notebook, notes
19	Spreadsheet Automation: Macros, VBA, Office Scripts & Apps Script	25–30 hours	13 Excel workbook, Python script, dataset (csv), notes, text
20	Automating Reports & Delivering Insights	20–25 hours	Python script, notebook
21	Descriptive Statistics & Probability	20–23 hours	Python script, dataset (csv), notebook, notes
22	Statistics Without Fooling Yourself	20–23 hours	Excel workbook, 2 Python script, 2 dataset (csv), notebook
23	Business Acumen, KPIs & Metrics	17–20 hours	Python script, 3 dataset (csv), notebook, notes
24	Requirements, Storytelling & Stakeholders	10–12 hours	Python script, 4 notes
25	The Business Analyst Track	6–8 hours	<i>reading only</i>
26	The Professional Toolkit: Git, Agile, Documentation & AI Assistants	9–11 hours	7 SQL, config, 2 text
27	Capstone: Your Analyst Portfolio	2–3 hours	Python script, 2 SQL, notebook

Chapter 10. Spreadsheet Fundamentals: Excel & Google Sheets

You will learn to: get a spreadsheet app working and check it · find your way around Excel and Google Sheets, create and save workbooks, and manage rows, columns, and sheets · copy, paste values, find and replace · tell what a cell really contains, not only what it shows · import a CSV file without losing leading zeros or scrambling dates · format numbers, dates, and currency · write formulas with relative, absolute, and mixed references · use the essential functions (SUM, AVERAGE, COUNT, IF, COUNTIFS, SUMIFS, text and date functions) · fetch a value from another sheet with a first lookup · sort, filter, and turn a range into a table · guard your data with validation and highlight it with conditional formatting · read error values and name ranges · split and fill data with Flash Fill and Smart Fill · remove duplicates · build a clear chart · print and save as PDF · share, protect, and track versions of a workbook · build a monthly sales tracker from a raw export.

Before you start: Chapter 1 (data types, levels of measurement, data quality), Chapter 2 (files and formats, especially CSV) and Chapter 4 (percentages and averages). Chapter 3 (how a business runs on data, including cancelled orders and targets) helps.

Time needed: 18–22 hours of reading and practice, spread over two to three weeks.

Sections: 10.0 Getting a spreadsheet and checking it works · 10.1 Excel and Google Sheets: what they are · 10.2 Workbooks, sheets, cells, and ranges · 10.3 What a cell really contains · 10.4 Importing a CSV file without damage · 10.5 Entering and formatting data · 10.6 Formulas and cell references · 10.7 Totals, counts, and rounding · 10.8 Decisions and conditions · 10.9 Text and date functions · 10.10 A first lookup: fetching values from another sheet · 10.11 Sorting, filtering, and tables · 10.12 Data validation and conditional formatting · 10.13 Charts · 10.14 Working together: sharing, protection, and version history · 10.15 Moving between apps, printing, and saving as PDF · 10.16 Keyboard shortcuts for both apps

In companion/ch10/ — *Python script:* `build_ch10_files.py` · *Excel workbook:* `ch10_practice.xlsx`, `ch10_tracker_check.xlsx`, `ch10_tracker_solution.xlsx` · *dataset (csv):* `riverstone_sales_export_2025.csv`

Chapter 11. The Spreadsheet, Mastered: Excel & Google Sheets

You will learn to: answer business questions with the conditional functions (SUMIF(S), COUNTIF(S), AVERAGEIF(S), MAXIFS, MINIFS) and SUMPRODUCT, including OR logic, date ranges, wildcards, weighted averages, and distinct counts · use the everyday toolkit of logical, text, date, and number functions · look up values in every way analysts need (exact, approximate, last match, several columns, two-way) with XLOOKUP and INDEX/MATCH · summarize thousands of rows in seconds with pivot tables, slicers, and pivot charts, and read results out of them with GETPIVOTDATA · write formulas that return whole lists with dynamic arrays (FILTER, UNIQUE, SORT, SEQUENCE, LET, TAKE, VSTACK, LAMBDA, SCAN) · combine a folder of monthly files and clean, group, merge, and reshape them with Power Query (including custom columns in M and handling broken files), so next month is one refresh · build a data model with relationships and write your first DAX measures in Power Pivot · answer "what would it take?" with Goal Seek and data tables · use Google Sheets' power features: QUERY, ARRAYFORMULA, IMPORTRANGE, and Connected Sheets · know what each app

can't do and what to use instead · audit a workbook and recognize when a job has outgrown the spreadsheet · and, as an optional challenge, solve a championship-style Excel case against the clock.

Before you start: Chapter 10.

Time needed: 35–45 hours of reading and practice, over five to six weeks: about 12 sittings of 3–4 hours (two for §11.2, one each for §11.1 and §11.3–11.5, one for the core of §11.6 with §11.10–11.11, one for §11.7, one for Goal Seek, §11.12–11.13 and the story, one for the project, one for the exercises, and two for the second pass).

Sections: 11.1 From a tracker to a system · 11.2 Conditional aggregation in depth: the IFS family and SUMPRODUCT · 11.3 The everyday function toolkit · 11.4 Lookups in depth · 11.5 Pivot tables · 11.6 Dynamic arrays: formulas that return whole tables · 11.7 Power Query: record the cleanup once, refresh forever · 11.8 Power Pivot and a first taste of DAX · 11.9 What-if analysis · 11.10 Google Sheets power features · 11.11 What Excel does that Sheets can't (and the other way round) · 11.12 Spreadsheet hygiene and auditing · 11.13 When to leave the spreadsheet

In companion/ch11/ — *Python script:* `build_ch11_files.py` · *Excel workbook:* `ch11_practice.xlsx`, `riverstone_rewards_case.xlsx`, `riverstone_rewards_solution.xlsx`, `riverstone_stockroom_case.xlsx`, and 2 more · *dataset (csv):* `sales_2025_01.csv`, `sales_2025_02.csv`, `sales_2025_03.csv`, `sales_2025_04.csv`, and 9 more

Chapter 12. Databases & SQL Foundations

You will learn to: explain what a database is and why businesses use one · read a table's structure (columns, data types, keys) and an entity-relationship diagram · write queries that choose, filter, sort, calculate, summarize, and combine data · avoid the classic traps (NULLs, AND/OR, duplicate rows from joins) · answer real business questions (inactive customers, discount leakage, overdue payments, sales performance) step by step · create, fill, correct, restructure, and remove your own databases and tables in both PostgreSQL and MySQL (CREATE, INSERT, UPDATE, DELETE, ALTER, DROP) · rebuild a real monthly report in SQL.

Before you start: Chapter 1 (what data is), Chapter 3 (how a business runs on data), and Chapters 10–11 (spreadsheets). You do not need any programming experience.

Time needed: 22–26 hours of reading and practice for the PostgreSQL core (installing, sections 12.1 to 12.16, and the exercises), spread over four to five weeks. Add 6–10 hours if you also follow the MySQL track and do the project.

Sections: 12.1 What a database actually is · 12.2 Meet Riverstone Supplies · 12.3 Setting up your SQL laboratory · 12.4 Your first query: SELECT and FROM · 12.5 ORDER BY and LIMIT: sorting, top-N, and unique values · 12.6 WHERE: keeping only the rows you want · 12.7 NULL: the value that isn't there · 12.8 Transforming values: CASE, dates, and text · 12.9 Summarizing: aggregate functions, GROUP BY, and HAVING · 12.10 JOIN: combining tables · 12.11 How the database reads your query · 12.12 Queries inside queries: subqueries and set operations · 12.13 Building and changing a database: CREATE, INSERT, UPDATE, DELETE, ALTER, and DROP · 12.14 Writing SQL that humans can read · 12.15 Putting it all together: Tuesday morning with the sales head · 12.16 The same SQL in MySQL

In companion/ch12/ — *SQL:* `ch12_queries_mysql.sql`, `ch12_queries_postgresql.sql`

Chapter 13. SQL for Real Analysis

You will learn to: break a hard question into named steps with common table expressions (CTEs) · save a business definition once as a view · use window functions to calculate shares, rankings, previous values, running totals, and moving averages without losing any rows · apply a library of patterns analysts use every week: top-N per group, deduplication, month-over-month growth, filling missing months, Pareto (ABC) analysis, funnels, cohort retention, streaks, pivots, and data-quality checks.

Before you start: Chapter 12. You should be comfortable with joins, GROUP BY, subqueries, and the fan-out trap.

Time needed: 15–20 hours of reading and practice, spread over three weeks. Week 1: sections 13.1 to 13.5. Week 2: sections 13.6 and 13.7. Week 3: sections 13.8 and 13.9, and the project.

Sections: 13.1 Two practice databases · 13.2 Common table expressions: queries in named steps · 13.3 Window functions: calculations that keep every row · 13.4 Ranking: ROW_NUMBER, RANK, and DENSE_RANK · 13.5 LAG and LEAD: comparing a row with its neighbors · 13.6 Time windows: growth, targets, and moving averages · 13.7 Patterns for ranking, shares, and clean-up · 13.8 Patterns over time, and a final check · 13.9 Running this chapter in MySQL

In companion/ch13/ — *SQL:* `calendar_tables_mysql.sql`, `calendar_tables_postgresql.sql`, `ch13_queries_mysql.sql`, `ch13_queries_postgresql.sql` · *Python script:* `make_calendar_tables.py`, `make_mysql_queries.py`

Chapter 14. Data Cleaning & Preparation

You will learn to: follow a repeatable cleaning workflow (load, profile, fix, validate, reconcile, document) · profile an unfamiliar dataset in minutes with counts, patterns, and ranges · decide what to do with missing values: standardize, repair, quarantine, or keep and label · find exact and fuzzy duplicates without merging the wrong records · standardize categories and typos with mapping tables · judge whether an outlier is an error or reality · parse mixed date formats, reject impossible dates, convert time zones, and fix units and currency text · repair join keys and measure match rates before joining messy sources · write validation rules that must return zero · reconcile a cleaned table to its source to the rupee · keep a cleaning log that someone else can audit · do all of it in SQL (PostgreSQL and MySQL), and in Excel and Power Query.

Before you start: Chapter 1 (section 1.10, data quality), Chapter 10 (importing CSV files, text and date functions), Chapter 11 (Power Query), and Chapters 12–13 (SQL, CTEs, window functions).

Time needed: 20–25 hours of reading and practice, spread over three weeks. A plan that works: week 1, sections 14.1–14.4; week 2, sections 14.5–14.9; week 3, sections 14.10–14.13, the project, and the timed challenge. If time is short, leave Exercises 22 and 23 for later.

Sections: 14.1 A cleaning workflow you can repeat · 14.2 Profiling a new dataset · 14.3 Missing values · 14.4 Duplicates, exact and fuzzy · 14.5 Inconsistent categories and typos · 14.6 Outliers: error or reality? · 14.7 Dates, time zones, units, and currency · 14.8 Joining messy sources · 14.9 The whole pipeline in SQL · 14.10 Validation rules: checks that must return zero · 14.11 Reconciling and documenting every decision · 14.12 The whole pipeline in Excel and Power Query · 14.13 Running this chapter in MySQL

In companion/ch14/ — *SQL*: ch14_queries_mysql.sql, ch14_queries_postgresql.sql, ch14_clean_mysql.sql, ch14_clean_postgresql.sql, and 7 more · *Python script*: build_ch14_files.py, build_ch14_q3_export.py, build_ch14_sql.py, clean_orders_pandas.py · *dataset (csv)*: branch_map.csv, city_map.csv, clean_order_lines_pandas.csv, clean_truth_orders_q3_2025.csv, and 6 more

Chapter 15. Data Visualization Principles

You will learn to: explain how people read charts, and why position and length beat angle, area, and color · start from the question and pick the chart that answers it · build clear bar, line, histogram, box, scatter, stacked, waterfall, heatmap, and map charts, and know when each fails · use color with meaning: one highlight, sequential and diverging palettes, color-blind-safe choices · write titles that state the finding and annotations that explain it · remove clutter without removing information · recognize misleading charts (truncated axes, dual axes, 3D, cherry-picked ranges) and avoid making them · show missing and uncertain data openly · make charts accessible · build the charts in Excel and Google Sheets · redesign a poor management pack.

Before you start: Chapter 1 (levels of measurement), Chapter 4 (averages (mean vs median), percentages, and the chart checks in section 4.7), Chapter 10 and 11 (spreadsheets and pivot tables), and Chapter 13 (SQL aggregation). Chapter 14 helps: charts of dirty data are wrong however well they're drawn.

Time needed: 12–15 hours of reading and practice, spread over two weeks.

Sections: 15.1 How people read charts · 15.2 Start from the question · 15.3 Bar and column charts · 15.4 Line charts · 15.5 Distributions: histograms and box plots · 15.6 Relationships: scatter plots · 15.7 Parts of a whole: stacked bars, pies, and waterfalls · 15.8 Heatmaps and tables · 15.9 Maps · 15.10 Color with meaning · 15.11 Titles, labels, annotations, and clutter · 15.12 Misleading charts, and how not to make them · 15.13 Missing data, uncertainty, and accessibility · 15.14 Building charts in Excel and Google Sheets · 15.15 Running this chapter's SQL in MySQL

In companion/ch15/ — *SQL*: ch15_queries_mysql.sql, ch15_queries_postgresql.sql · *Python script*: build_ch15_data.py · *Excel workbook*: ch15_chart_data.xlsx · *dataset (csv)*: anscombe.csv, bridge_2024_2025.csv, category_month_2025.csv, city_2025.csv, and 10 more

Chapter 16. Business Intelligence with Power BI

You will learn to: install Power BI Desktop and build a first report · explain what a BI tool does that a spreadsheet can't, and which Power BI licence buys what · load Riverstone's full dataset with Power Query and choose between Import and DirectQuery · build a star schema with a proper date table · write DAX measures from zero, and know when a measure beats a calculated column · use CALCULATE to change filter context deliberately · build time intelligence (year to date, same period last year, year-over-year growth) that survives slicers · design a report page with Chapter 15's principles, using cards, slicers, tooltips, drill-through, and bookmarks · publish, schedule refresh, use a gateway, and share with an app · apply row-level security so each region sees only its own customers · keep a model fast and avoid the modelling mistakes that make numbers wrong · recognize the same ideas in Tableau and Looker Studio.

Before you start: Chapter 11 (Power Query, the Data Model, and DAX in Excel), Chapter 13 (SQL aggregation and the sales_lines view), Chapter 14 (cleaning: this chapter assumes clean data), and Chapter 15 (chart choice, color, titles: this chapter applies them rather than repeating them). You don't need any programming for this chapter. Python comes next, in Chapters 17 and 18.

Time needed: 22–26 hours, spread over three weeks.

Sections: 16.0 Install Power BI Desktop and make your first report · 16.1 What Power BI is, and what it costs · 16.2 Getting Riverstone's data in · 16.3 The model: a star schema and a date table · 16.4 DAX from zero · 16.5 CALCULATE and filter context · 16.6 Time intelligence · 16.7 Designing the report page · 16.8 A worked example: Riverstone's monthly pack · 16.9 Publishing, refresh, and sharing · 16.10 Row-level security · 16.11 Performance and the mistakes that make numbers wrong · 16.12 The same ideas in other tools · 16.13 Where BI tools are heading

In companion/ch16/ — *SQL:* ch16_checks_mysql.sql , ch16_checks_postgresql.sql , ch16_queries_mysql.sql , ch16_queries_postgresql.sql · *dataset (csv):* city_region.csv , user_region.csv

Chapter 17. Python from Zero

You will learn to: use a terminal for the handful of commands Python needs · install Python, VS Code, and Jupyter on Windows, macOS, or Linux, and run your first notebook · create a virtual environment and install packages into it · say what a program is and when Python beats a spreadsheet or SQL · work with variables, strings, numbers, and booleans · use lists, tuples, dictionaries, and sets, and know which to reach for · write conditions and loops, and read a comprehension · write functions with arguments, defaults, and return values · read and write CSV, JSON, and text files with pathlib and the standard library · read a traceback, handle errors on purpose, and debug · turn a notebook experiment into a script someone else can run · get unstuck without waiting for anyone.

Before you start: Chapters 1–16. Chapter 14 matters most (what cleaning involves), and this chapter keeps comparing Python with the spreadsheet formulas of Chapters 10–11 and the SQL of Chapters 12–13. No programming experience is assumed, and you don't need anything installed yet: section 17.0 installs everything, starting with the terminal.

Time needed: 28–32 hours over three to four weeks: about 2 hours to set up (section 17.0), then five sittings of five or six hours, each best split over two or three evenings. The "Stop here" notes mark the sittings. Programming is learned by typing, not by reading.

Sections: 17.0 Setting up Python, the terminal and Jupyter · 17.1 What a program is, and when to use one · 17.2 Your first Python · 17.3 Variables and types · 17.4 Lists and tuples · 17.5 Conditions: making decisions · 17.6 Loops · 17.7 Dictionaries and sets · 17.8 Functions · 17.9 Files and folders · 17.10 Errors, tracebacks, and debugging · 17.11 The standard library and packages · 17.12 From notebook to script · 17.13 Getting unstuck

In companion/ch17/ — *notebook:* ch17-notebook.ipynb · *Python script:* build_ch17_files.py , check_setup.py , summarize_exports.py · *dataset (csv):* broken_export.csv , month_summary.csv , riverstone_2025_01.csv , riverstone_2025_02.csv , and 11 more

Chapter 18. Python for Analysts: pandas & Automation

You will learn to: think in whole columns instead of loops, starting with NumPy arrays · load data from CSV, Excel, JSON, Parquet, a database, and an API · profile a DataFrame in five lines · select, filter, and create columns without falling into pandas' classic traps · aggregate with groupby, combine with merge, and reshape with pivot_table and melt · work with dates, resampling, and rolling windows · redo Chapter 14's cleaning in pandas and reconcile it to the rupee · draw Chapter 15's charts with matplotlib and seaborn through one reusable style function · write formatted, multi-sheet Excel files people are glad to receive · call an API, page through its results, and handle its status codes · turn the whole thing into one script with logging, checks, and a Markdown summary · know when to push work back to SQL and when to stop using pandas · organise a growing script into classes, data classes and composition, and judge honestly when a class is the wrong answer · use inheritance where several things really are a kind of one thing · give the work one entry point with an exit code, fire it from a .bat file and Task Scheduler, and prove the morning after that it ran.

Before you start: Chapter 17 (Python, the notebook, and the terminal from section 17.0), Chapters 12–13 (SQL and the sales_lines view), Chapter 14 (cleaning), Chapter 15 (chart choice and design), Chapter 16 (Power BI, whose measures pandas mirrors), and Chapters 10, 11, and 19 for the spreadsheet ideas pandas mirrors.

Time needed: 47–53 hours, spread over five weeks. Type every example. A plan that works: week 1, sections 18.1–18.5 (NumPy, reading, looking, filtering, new columns); week 2, sections 18.6–18.9 (grouping, joining, reshaping, dates); week 3, sections 18.10–18.12 (cleaning, charts, Excel); week 4, sections 18.13–18.16 (database, API, the whole script, performance); week 5, sections 18.17–18.18 (classes and scheduling), the project, and the timed challenge. Each week ends with a short checkpoint.

Sections: 18.1 DataFrames, Series, and vectorized thinking · 18.2 Reading data from anywhere · 18.3 Looking at a DataFrame · 18.4 Selecting and filtering · 18.5 Creating and changing columns · 18.6 groupby: split, apply, combine · 18.7 Combining tables: merge and concat · 18.8 Reshaping: pivot_table, melt, and tidy data · 18.9 Dates and time series · 18.10 Cleaning in pandas · 18.11 Charts with matplotlib and seaborn · 18.12 Writing Excel people are glad to receive · 18.13 Reading from a database · 18.14 Calling an API · 18.15 The whole thing as one script · 18.16 Performance, habits, and when to stop using pandas · 18.17 Organising the work: classes, data classes, and composition · 18.18 From script to a tool that runs itself

In companion/ch18/ — *notebook:* ch18-notebook.ipynb · *Python script:* api_demo.py, build_ch18_files.py, clean_orders_pandas.py, monthly_report.py, and 2 more · *Excel workbook:* riverstone-2025-12.xlsx, riverstone_2025.xlsx

Chapter 19. Spreadsheet Automation: Macros, VBA, Office Scripts & Apps Script

You will learn to: decide when automating inside a spreadsheet is the right answer · record a macro, save a macro-enabled workbook, and handle macro security without turning it off · read recorded code and rewrite it properly · program from zero in VBA: variables, If, Select Case, loops, arrays, collections, Sub and Function · work with Excel's object model instead of clicking · build the everyday toolkit: last row, loop the sheets, consolidate a folder of files, clean, format, pivot, PDF · send an Outlook email with

the report attached or in the body · write custom worksheet functions and a simple form · debug with breakpoints and the Immediate window, and handle errors on purpose · make a macro fast with arrays and screen updating · do the same work in Office Scripts (Excel on the web, TypeScript) and Google Apps Script (Sheets, JavaScript), including triggers, HTML email, and API calls · keep macros maintainable, and know when to move the job to Power Query, Python, or a pipeline.

Before you start: Chapters 10 and 11 (spreadsheets, formulas, lookups, pivot tables, Power Query). No programming experience is assumed: this is the first chapter in the book where you write code, and section 19.4 teaches the basics from zero in VBA.

Time needed: 25–30 hours, spread over three weeks: VBA (sections 19.1–19.10) 13–16 hours, Office Scripts and Apps Script (sections 19.11–19.14) 6–8 hours, and the project about 6 hours. Take sections 19.11–19.14 as a separate sitting.

Sections: 19.1 When automating inside a spreadsheet is the right answer · 19.2 Recording your first macro · 19.3 The VBA editor · 19.4 Programming from zero, in VBA · 19.5 Excel's object model · 19.6 The everyday toolkit · 19.7 Sending the report by Outlook · 19.8 Custom functions and a simple form · 19.9 Debugging and error handling · 19.10 Making macros fast · 19.11 Office Scripts: Excel on the web · 19.12 Google Sheets: macros and Apps Script · 19.13 Triggers, email, and APIs in Apps Script · 19.14 The same job in three languages · 19.15 Living with macros responsibly

In companion/ch19/ — *Python script:* `build_ch19_files.py` · *Excel workbook:*

`Riverstone_Bengaluru_2025-10.xlsx`, `Riverstone_Bengaluru_2025-11.xlsx`, `Riverstone_Bengaluru_2025-12.xlsx`, `Riverstone_Delhi_2025-10.xlsx`, and 9 more · *dataset (csv):* `enquiries_sample.csv`

Chapter 20. Automating Reports & Delivering Insights

You will learn to: place any report on the automation ladder and decide how far up it should go · map a report's flow and time its manual steps before automating anything · choose between VBA, Apps Script, Python, BI subscriptions, and low-code flows · produce the right output: formatted Excel, PDF, CSV, or the email body itself · build an HTML email with KPI tiles, a table, and a chart attached so that it shows in Outlook and Gmail · send mail safely from code with SMTP or a workspace API, with credentials in a `.env` file, and test it on a mail server on your own computer · schedule with Task Scheduler, cron, or a cloud scheduler, in the business's time zone · design exception reports and alerts that people don't learn to ignore · deliver to Teams, Slack, or WhatsApp · make an automation trustworthy: logging, checks, failure alerts, "no data today", retries, idempotency · manage recipients and confidentiality · document and hand over · measure what it saved.

Before you start: Chapter 19 (spreadsheet automation, and its box "HTML in ten minutes" in section 19.7), Chapter 13 (the SQL the report runs on), Chapter 14 (checks on data), Chapter 15 (chart design), Chapter 16 (BI subscriptions), Chapter 18 (pandas and scripts), and the terminal basics from Chapter 17, section 17.0. The scheduling section uses a few more terminal pieces, each explained where it appears; Chapters 26 and 34 teach the terminal properly.

Time needed: 20–25 hours, spread over two to three weeks. Allow two of those hours for setting up: the `.env` file, the local test mail server, and a scheduler.

Sections: 20.1 The automation ladder · 20.2 Map the flow before you automate · 20.3 Choosing the delivery tool · 20.4 Output formats: what to send · 20.5 The report as an email · 20.6 Sending mail from

code, safely · 20.7 Scheduling · 20.8 Alerts and exception reports · 20.9 Delivering to chat · 20.10 Low-code automation · 20.11 Making an automation trustworthy · 20.12 Recipients and confidentiality · 20.13 Documenting and handing over · 20.14 Measuring what it saved

In `companion/ch20/` — *notebook:* `ch20-notebook.ipynb` · *Python script:* `daily_flash.py`

Chapter 21. Descriptive Statistics & Probability

You will learn to: work out the mean, median, mode, variance, standard deviation, and quartiles by hand, in a spreadsheet, and in pandas · choose between mean, median, and mode, and say why they differ · measure spread with range, variance, standard deviation, and the interquartile range, and know which to quote · read percentiles and use them in service levels · describe the shape of data: skew, tails, and outliers · recognize the four distributions an analyst meets most (normal, binomial, Poisson, uniform) and what each implies · apply the rules of probability, including conditional probability and independence · use Bayes' rule on a real business question and explain the answer to a manager · understand sampling, sampling error, and the central limit theorem by simulating it · put all of it together to profile a dataset statistically.

Before you start: Chapter 4 (averages, percentages, and probability by counting), Chapter 15 (histograms, box plots, and quartiles by hand), and Chapters 17 and 18 (Python and pandas). No mathematics beyond Chapter 4 is assumed. Section 21.0 shows how to read the few symbols this chapter uses, and every formula is written once in symbols and then explained in words.

Time needed: 20–23 hours, spread over three weeks, in two halves: describing data (sections 21.0 to 21.4, about 9 hours) and probability and sampling (sections 21.5 to 21.9, about 12 hours). Each half ends with a short checkpoint.

Sections: 21.0 Seven deliveries, by hand · 21.1 The centre: mean, median, and mode · 21.2 Spread: range, variance, standard deviation, and IQR · 21.3 Percentiles and service levels · 21.4 Shape: skew, tails, and outliers · 21.5 Four distributions worth knowing · 21.6 Probability rules · 21.7 Bayes' rule · 21.8 Sampling and the central limit theorem · 21.9 Putting it together: profiling a measure

In `companion/ch21/` — *notebook:* `ch21-notebook.ipynb` · *Python script:* `build_ch21_files.py` · *dataset (csv):* `delivery_times_2025.csv`

Chapter 22. Statistics Without Fooling Yourself

You will learn to: turn sampling error into a confidence interval, for a mean and for a proportion, by hand, in a spreadsheet and in Python, and explain what it does and doesn't mean · state a hypothesis, run the right test, and read a p-value without overclaiming · design an A/B test: what to measure, how many people you need, when to stop · see why peeking at a running test manufactures winners · handle multiple comparisons and recognize p-hacking in your own work · separate correlation from causation, and spot confounders, Simpson's paradox, survivorship bias, and regression to the mean · weigh statistical significance against practical significance · write up a result plainly, including the ones that didn't work · fit and read a straight-line regression by hand, in a spreadsheet and in Python.

Before you start: Chapter 21 (distributions, sampling, standard error, Bayes), Chapter 18 (pandas and matplotlib), and Chapter 15 (scatter plots, trend lines, and showing uncertainty). Chapter 4's

percentages and Chapter 14's data quality both matter: no test survives bad data.

Time needed: 20–23 hours, spread over two and a half weeks, in two parts. Part A, "Uncertainty and tests" (sections 22.1–22.4), takes about 11 hours and ends with a checkpoint. Part B, "Traps and relationships" (sections 22.5–22.10), takes about 10 hours; the regression section 22.10 alone is about 4 of them. Allow 2 more for the project.

Sections: 22.1 Confidence intervals · 22.2 Hypothesis tests and p-values · 22.3 Designing an A/B test · 22.4 Peeking, p-hacking, and multiple comparisons · 22.5 Correlation, causation, and confounders · 22.6 Simpson's paradox · 22.7 Survivorship, selection, and regression to the mean · 22.8 Statistical significance versus practical significance · 22.9 Writing up a result · 22.10 Fitting a line: regression basics

In `companion/ch22/` — *notebook:* `ch22-notebook.ipynb` · *Python script:* `build_ch22_files.py`, `build_ch22_workbook.py` · *Excel workbook:* `ch22_by_hand.xlsx` · *dataset (csv):* `ab_test_2026.csv`, `transporters_q4_2025.csv`

Chapter 23. Business Acumen, KPIs & Metrics

You will learn to: trace how a business turns cash into stock into sales and back into cash · read a P&L, a balance sheet, and a cash flow statement well enough to hold a conversation with Finance · explain why a profitable company can still run out of money · use the core metrics of sales, marketing, finance, operations, and customer teams, and compute each one correctly · build a KPI tree that decomposes revenue into numbers a team can actually act on · diagnose a revenue change with a disciplined walk down that tree instead of a guess · tell a good metric from a vanity metric, and see how targets corrupt measures · write a metric definition precise enough that two teams get the same number.

Before you start: Chapter 3 (bookings, billings, collections, receivables, and what makes a KPI), Chapter 4 (percentages and the arithmetic of business), Chapter 13 (removing duplicate leads, Pattern 3), Chapter 18 (pandas), Chapter 21 (distributions and averages), and Chapter 22 (whether a change is real). Nothing here assumes an accounting or finance background: section 23.2 starts with the words Finance will use.

Time needed: 17–20 hours, spread over two weeks, in two parts. Part A, "Money" (sections 23.1–23.4), takes about 8 hours and ends with a checkpoint. Part B, "Metrics that run a business" (sections 23.5–23.13), takes about 10 hours. Allow 2 more for the project.

Sections: 23.1 How a business makes money: the cash cycle · 23.2 The profit & loss statement · 23.3 The balance sheet · 23.4 Cash flow: why profit isn't cash · 23.5 Sales metrics · 23.6 Marketing metrics · 23.7 Finance metrics · 23.8 Operations metrics · 23.9 Customer metrics · 23.10 KPI trees and the North Star · 23.11 Diagnosing a change · 23.12 Good metrics, vanity metrics, and gaming · 23.13 Defining a metric so two teams get the same number

In `companion/ch23/` — *notebook:* `ch23-notebook.ipynb` · *Python script:* `build_ch23_files.py` · *dataset (csv):* `marketing_2025.csv`, `monthly_revenue_2025.csv`, `working_capital_quarters_2025.csv`

Chapter 24. Requirements, Storytelling & Stakeholders

You will learn to: turn a vague ask into a specific, answerable question · write down a business rule so precisely that two people applying it get the same answer · map stakeholders by power and interest, and adjust how much you involve each one · lead with the answer instead of the journey (bottom-line-up-front, the pyramid principle) · design a slide around one message instead of a table of everything you found · write a one-page analysis memo that survives being forwarded without you in the room · present to executives: structure, pacing, and handling the interruption · respond to "can you just change the number?" without becoming difficult or becoming compliant.

Before you start: nothing new and technical — this chapter is about communication. It builds on three earlier chapters: Chapter 5, sections 5.2 and 5.4 (turning a vague request into a precise question, and issue trees), Chapter 15's chart-design rules (they apply directly to the slides in section 24.5), and Chapter 23, section 23.11 (walking a revenue change down the KPI tree), whose method gives this chapter's worked example its numbers.

Time needed: 10–12 hours, spread over a week. The reading is short; the exercises ask you to write, which takes longer than it looks.

Sections: 24.1 Turning an ask into a question · 24.2 Documenting business rules · 24.3 Mapping your stakeholders · 24.4 Bottom line up front: the pyramid principle · 24.5 One message per slide · 24.6 Writing a one-page analysis memo · 24.7 Presenting to executives · 24.8 Handling pushback: "can you just change the number?" · 24.9 Bringing it together

In `companion/ch24/` — *Python script:* `build_ch24_files.py`

Chapter 25. The Business Analyst Track

You will learn to: say what a business analyst does on a data team, and how the work divides between the BA, the data analyst, the data scientist, and the data engineer · place that work inside each phase of the software development life cycle · turn a vague ask into a written, testable requirement · map a process as a flowchart, as a swimlane, and in BPMN, and document it as-is before proposing a to-be · tell business, functional, and non-functional requirements apart, including the non-functional ones specific to data: freshness, grain, completeness, and volume · write the requirement for each of the four things a data team is asked to build: a report or dashboard, a pipeline, a model, and a metric definition · choose between a BRD, an FRD, and an SRS, and know who reads each · write a use case, and a user story with acceptance criteria that work for data, including reconciliation and edge cases · run a gap analysis that ends in a requirement · explain what user acceptance testing is for a data product, and why "the system works" is not "the number is right" · work with an IT team or an outside vendor · find the automation opportunities in a process, rank them, and specify the one you pick.

Before you start: Chapter 3 (how a business runs on data, especially section 3.2's order journey and section 3.7's manual work) and Chapter 24 (turning an ask into a question, documenting business rules, and stakeholders). Chapter 23 (KPIs, and section 23.13 on defining a metric so two teams agree) helps, as does Chapter 16 (dashboards) and Chapter 20 (delivering reports).

Time needed: 6–8 hours, including the exercises and the project. Two sittings work well: sections 25.1 to 25.5, then section 25.6 to the end.

Sections: 25.1 The business analyst on a data team · 25.2 The software development life cycle, and where you sit in it · 25.3 From a vague ask to a written requirement · 25.4 Mapping the process before you change it · 25.5 Business, functional, and non-functional requirements · 25.6 The four things a data team is asked to build · 25.7 The documents: BRD, FRD, and SRS · 25.8 Use cases and user stories · 25.9 Gap analysis · 25.10 User acceptance testing, and why "it works" is not "the number is right" · 25.11 Working with IT and vendors, and why domain knowledge decides everything · 25.12 Finding, ranking, and specifying an automation

Chapter 26. The Professional Toolkit: Git, Agile, Documentation & AI Assistants

You will learn to: use the terminal with confidence: move around, make and copy files, read any command, chain commands, and set environment variables · explain what version control solves, and why "final_v3 REALLY_FINAL.xlsx" is a symptom · create a repository and record work in it with git add, git commit, and git log · read git status and git diff and say exactly what each line means · keep secrets and generated files out of a repository with .gitignore and a .env file · undo the four things that go wrong, and know which of the four you are in before you type anything · use a branch to try something without breaking what works, and resolve a conflict · push to GitHub, open a pull request, and review someone else's · add one automated check that runs on every push · write a README that lets a stranger run your work · keep a SQL pattern library and documentation that people can still find in six months · work inside Scrum and Kanban, and read a Jira board without being told · use an AI assistant at work: what to delegate, how to check what it gives you, and what must never be pasted into one.

Before you start: Chapter 17, section 17.0 (you have opened a terminal, moved around in it, and run Python from it). Chapters 12 and 13 (the SQL you will be versioning), Chapters 17 and 18 (the Python scripts), Chapter 20 (the automated report), and Chapter 25 (user stories and business rules, which sections 26.9 and 26.10 refer to). This chapter installs Git. Nothing here needs a paid account.

Time needed: 9–11 hours, including the exercises and the project, in three sittings: sections 26.0 to 26.6, the terminal and Git on your own computer (3–4 hours, typing along); sections 26.7 to 26.9, GitHub, the automated check, and the README (2–3 hours); sections 26.10 and 26.11 and the project (3–4 hours).

Sections: 26.0 The terminal in 20 minutes · 26.1 The three places a file lives · 26.2 Your first repository · 26.3 Reading the history, and reading a change · 26.4 What must never go into a repository · 26.5 Undoing things: which of the four are you in? · 26.6 Branches, and why they are not an advanced topic · 26.7 GitHub, pushing, and the pull request · 26.8 One automated check on every push · 26.9 A repository a stranger can run · 26.10 How the work is actually planned · 26.11 Working with an AI assistant

In companion/ch26/ — *SQL:* monthly_revenue.sql, 1_limit_2025.sql, 3_category_split.sql, 4_branch_2026.sql, and 3 more

Chapter 27. Capstone: Your Analyst Portfolio

You will learn to: take one business question all the way from a database to a memo, using only what Part 2 taught · write down the cleaning decisions you made, and measure whether they changed the answer · find the check that turns a flattering result into an honest one, and report both · wrap the analysis in a function so it can be re-run and argued with · specify a one-page dashboard that serves a single decision · write the memo, including the part that says what you did not find · recognize selective reporting in your own portfolio, which is where it is most tempting · assemble three projects into a portfolio a hiring manager will actually open · tell the story of a project in two minutes and in ten · judge for yourself whether you are ready to apply.

Before you start: all of Part 2. This chapter adds no new tools and one new SQL function, NTILE (section 27.4). What is new is method: how to record and measure your own decisions, how to test a headline before believing it, and how to present work honestly. It uses Chapters 12 and 13 (SQL), 14 (cleaning), 17 and 18 (Python), 15 and 16 (visualization and Power BI), 20 (automation), 21 and 22 (statistics), 23 (metrics), 24 and 25 (requirements and stakeholders), and 26 (the repository this all lives in).

Time needed: reading and running the worked project, 2–3 hours. Your own portfolio, 20–30 hours over one to two weeks (the deep project alone, 8–12 hours).

Sections: 27.1 A question worth putting in a portfolio · 27.2 SQL: the headline (a recap, not a re-teach) · 27.3 Cleaning: the decisions, and whether they mattered · 27.4 The check that changes the answer · 27.5 Python: making the analysis arguable · 27.6 The dashboard: one page, one decision · 27.7 The memo · 27.8 Selective reporting, and why a portfolio is where it starts · 27.9 What a hiring manager does with your repository · 27.10 Telling the story, in two minutes and in ten · 27.11 What job-ready actually looks like

In companion/ch27/ — *notebook:* ch27-notebook.ipynb · *SQL:* ch27_queries_mysql.sql, ch27_queries_postgresql.sql · *Python script:* discount_analysis.py

Part 3 — Advanced Analytics & Analytics Engineering

Book 2 · Practical · 7 chapters · roughly 131 hours in total

Ch	Chapter	Time	What you practise with
28	Advanced SQL, Performance & Data Modeling	22–30 hours	3 Python script, 12 SQL, 5 dataset (csv), text
29	Python as Software, Not Scripts	20–24 hours	16 Python script, data (json), notebook, notes
30	Inference & Experiments	18–22 hours	3 Python script, 2 SQL, 3 dataset (csv), notebook
31	Causal Inference Without Experiments	15–18 hours	Python script, 4 dataset (csv), notebook
32	Analytics Engineering with dbt	18–22 hours	15 SQL, 7 config, notes, text
33	The Computer Science You Actually Need	14–18 hours	3 Python script, 2 dataset (csv), notebook
34	The Command Line, Linux & Networking Basics	9–12 hours	2 Python script, 64 dataset (csv)

Chapter 28. Advanced SQL, Performance & Data Modeling

You will learn to: walk org charts and bills of materials of any depth with recursive CTEs · protect a recursive query against loops · choose between ROWS, RANGE, and GROUPS frames, and use EXCLUDE and named windows · read a query plan with EXPLAIN and EXPLAIN ANALYZE · decide which indexes to create (single-column, composite, covering, partial) and which to avoid · recognize queries that can't use an index, and rewrite them · normalize a messy table to first, second, and third normal form · declare and test the grain of any table · design a star schema with facts, dimensions, and a date dimension · build slowly changing dimensions of types 1, 2, and 3 · decide when denormalizing is worth it · change a production database safely with versioned migration scripts.

Before you start: Chapter 12 (joins, keys, CREATE TABLE, ALTER TABLE, transactions, the fan-out trap) and Chapter 13 (CTEs, views, window functions, the sales_lines view). The chapter also leans on Chapter 4 (the median), Chapters 15 and 21 (percentiles in SQL, sections 15.5 and 21.3), Chapter 16 (section 16.3, the star schema you built in Power BI), Chapter 17 (running a Python script), and Chapter 26 (section 26.0, the terminal).

Time needed: 22–30 hours of reading and practice, including the project, spread over four to five weeks.

Sections: 28.1 Three practice databases · 28.2 Recursive CTEs: walking trees of any depth · 28.3 Advanced window frames · 28.4 How a database finds rows · 28.5 Reading query plans with EXPLAIN · 28.6 Indexes in practice · 28.7 Normalization, worked properly · 28.8 The grain discipline · 28.9 Dimensional modeling: facts, dimensions, and the star schema · 28.10 Slowly changing dimensions · 28.11 Denormalization trade-offs · 28.12 Schema migrations: changing a live database safely · 28.13 The same SQL in MySQL

In companion/ch28/ — *SQL:* ch28_2025_addons.sql, ch28_queries_mysql.sql, ch28_queries_postgresql.sql, ch28_star_schema.sql, and 8 more · *Python script:* ch28_perf.py,

`generate_riverstone_perf.py`, `migrate.py` · *dataset (csv)*: `customers.csv`, `employees.csv`, `order_items.csv`, `orders.csv`, and 1 more

Chapter 29. Python as Software, Not Scripts

You will learn to: recognize the signs that a script has outgrown being a script · split a script into small functions and modules with one job each · lay out a project with `src/`, `tests/`, and `pyproject.toml` · keep passwords and settings out of code · create reproducible environments with `venv`, pinned requirements, and a lockfile managed by `uv` · write small classes and data classes, and know when not to · add type hints and let `mypy` find bugs before users do · write fast, focused tests with `pytest`, fixtures, and parametrized cases, and run them on every push · raise meaningful exceptions and log instead of printing · build an API client that handles pagination, timeouts, retries, and rate limits · review code, and have your code reviewed, with a checklist.

Before you start: Chapter 17 (functions, files, errors, packages), Chapter 18 (pandas and the monthly report), Chapter 34 (the terminal, environment variables, exit codes), Chapter 12 (connecting to PostgreSQL), Chapter 2 (what an API is), and Chapter 26 (Git, pull requests, and the automated check of section 26.8).

Time needed: 20–24 hours of reading and practice, over three weeks, in four sittings marked "Stop here": sections 29.1–29.3 (about 5 hours), 29.4–29.6 (5 hours), 29.7–29.8 (6 hours), and 29.9–29.10 with the project (6–8 hours). Classes, type hints, a lockfile, and a test framework are all new here, so type the examples rather than reading them.

Sections: 29.1 The script that works until it doesn't · 29.2 Functions and modules: one job each · 29.3 Project structure and configuration · 29.4 Environments and lockfiles · 29.5 Classes, when they help · 29.6 Type hints · 29.7 Testing with `pytest` · 29.8 Errors and logging · 29.9 Robust API clients · 29.10 Code review

In `companion/ch29/` — *notebook*: `ch29-notebook.ipynb` · *Python script*: `mock_crm_api.py`, `__init__.py`, `cli.py`, `config.py`, and 12 more

Chapter 30. Inference & Experiments

You will learn to: compute a standard error and a confidence interval, by hand and in Python · compare two groups with a t-test and know when it doesn't apply · report an effect size, not only a p-value · test categorical results with a two-proportion test and chi-square · compare several groups with ANOVA without inflating your error rate · calculate the sample size a test needs before you run it · design an A/B test end to end, and write the plan down first · analyze one properly, with a sample-ratio check and guardrail metrics · recognize peeking, multiple testing, novelty effects, and outliers by watching them happen in simulations · read regression coefficients as statements about the business.

Before you start: Chapter 18 (pandas, and the NumPy basics in section 18.1), Chapter 21 (distributions, spread, sampling, and the central limit theorem, section 21.8), Chapter 22 (confidence intervals, p-values, chi-square, Type I and II errors, A/B test design, peeking, multiple comparisons, and regression basics in section 22.10), and Chapter 29 (`uv` projects, and Python as tested code). The

box "What's new since Chapter 22", after In plain English, says which parts of this chapter are a recap and which are new.

Time needed: 18–22 hours, spread over three weeks, in four sittings: (1) sections 30.1–30.4, standard errors, intervals, the t-test and effect size, about 5 hours; (2) sections 30.5–30.7, proportions, chi-square, ANOVA, power and sample size, about 5 hours, ending with a checkpoint; (3) sections 30.8–30.10, designing the test, analyzing it end to end, and the ways to fool yourself, about 5 hours; (4) sections 30.11–30.12, regression for inference and the write-up, about 4 hours. Allow 2 more for the project.

Sections: 30.1 The data, and the question · 30.2 Standard error and confidence intervals · 30.3 Comparing two groups: the t-test · 30.4 Effect size: how big, not just whether · 30.5 Categorical outcomes: proportions and chi-square · 30.6 More than two groups: ANOVA · 30.7 Power and sample size · 30.8 Designing an experiment · 30.9 Riverstone's website test, end to end · 30.10 Four ways to fool yourself, demonstrated · 30.11 Regression for inference · 30.12 Writing the result up

In companion/ch30/ — *notebook:* ch30-notebook.ipynb · *SQL:* load_mysql.sql, load_postgresql.sql · *Python script:* ab_summary.py, generate_riverstone_web.py, test_ab_summary.py · *dataset (csv):* ab_test_assignments.csv, web_events.csv, web_sessions.csv

Chapter 31. Causal Inference Without Experiments

You will learn to: read a difference of logarithms as a percentage change · say what a causal claim would mean when nobody randomized anything · spot why the two obvious comparisons (before-and-after, and treated-versus-untreated) usually mislead · estimate an effect with difference-in-differences, by hand and as a regression with fixed effects · test the parallel-trends assumption it rests on · build a synthetic control from untreated groups · build a comparison group with matching and propensity scores, and check the balance you achieved · use a threshold in a business rule as a natural experiment (regression discontinuity) · recognize a valid instrumental variable, and why you'll rarely have one · judge how much to trust each method, and write the claim up with its assumptions attached.

Before you start: Chapter 22 (correlation is not causation, section 22.5; fitting a line, section 22.10), Chapter 30 (confidence intervals, experiments, Cohen's d in section 30.4, and section 30.11's regression: one x , categories with `C()`, several variables, R^2 , the formula syntax, and logistic regression), Chapter 18 (pandas groupby, `pivot_table`, `pd.cut`, `lambda`, and the NumPy basics in section 18.1), Chapter 21 (standard deviation), and Chapter 11 (SUMPRODUCT). Section 31.0 below teaches the logarithms this chapter uses.

Time needed: 15–18 hours, spread over two and a half weeks, in two parts. Part A, comparisons over time (sections 31.0–31.5: logs, difference-in-differences, parallel trends, synthetic control), about 8 hours, ending with a checkpoint. Part B, comparisons across units (sections 31.6–31.9: matching, regression discontinuity, instruments, and how much to trust each), about 8 hours. Allow 2 more for the project.

Sections: 31.0 Logs in ten minutes · 31.1 The data, and the three true answers · 31.2 Why the two obvious comparisons mislead · 31.3 Difference-in-differences · 31.4 The assumption: parallel trends · 31.5 Synthetic control · 31.6 Matching and propensity scores · 31.7 Regression discontinuity: let the rule do the randomizing · 31.8 Instrumental variables: the idea · 31.9 How much to trust each answer

In companion/ch31/ — *notebook*: ch31-notebook.ipynb · *Python script*: generate_ch31_data.py · *dataset (csv)*: delivery_threshold.csv, qbr_program.csv, region_month.csv, pre_period_logs.csv

Chapter 32. Analytics Engineering with dbt

You will learn to: explain what analytics engineering is and where dbt fits · set up a dbt project against a real database, with credentials kept out of the code · turn SQL you already write into models, sources, and ref calls that dbt orders for you · lay a project out in staging, intermediate, and marts layers · choose materializations, and know what each one costs · write tests that fail loudly when a business rule breaks, including a reconciliation test · generate documentation and lineage nobody has to maintain by hand · keep type 2 history with snapshots · make a large model incremental and measure what it saves · write a macro so a business rule exists once · lint SQL and run the whole project on every pull request.

Before you start: Chapters 12 and 13 (SQL, the sales_lines view and its net-revenue rule), Chapter 28 (star schemas, grain, slowly changing dimensions, and its additions to riverstone_2025), Chapter 17 (virtual environments), Chapter 29 (uv in section 29.4, settings in environment variables, exit codes), Chapter 26 (Git, pull requests, the automated check and its YAML file), Chapter 34 (the command line, environment variables, running scripts, ports), and Chapter 16 (connecting Power BI to PostgreSQL, used in the project).

Time needed: 18–22 hours, spread over three weeks, in four sittings: sections 32.1–32.5 (setup, sources, staging, marts, the first build: 5–6 hours); sections 32.6–32.9 (materializations, tests, docs, snapshots: 4–5 hours); sections 32.10–32.13 (incremental models, macros, linting, CI: 4–5 hours); and the project (5–6 hours).

Sections: 32.1 What analytics engineering is · 32.2 Your first project · 32.3 Models, sources, and ref · 32.4 Layers: staging, intermediate, marts · 32.5 Building the star schema as models · 32.6 Materializations: view, table, incremental, ephemeral · 32.7 Tests: the part that earns the trust · 32.8 Documentation and lineage · 32.9 Snapshots: history dbt keeps for you · 32.10 Incremental models · 32.11 Macros: write a rule once · 32.12 Linting and continuous integration · 32.13 What dbt is not, and the semantic layer

In companion/ch32/ — *SQL*: ch32_queries_mysql.sql, ch32_queries_postgresql.sql, net_revenue.sql, dim_customer.sql, and 11 more

Chapter 33. The Computer Science You Actually Need

You will learn to: describe how long an algorithm takes in the language everyone uses (Big-O), and see the curves in measured times · choose between a list, a dictionary, a set, and a tuple for a reason · explain why a dictionary lookup is instant, and why the same trick powers a database hash join · use stacks and queues where they fit, including deque · write and read recursion without fear, and know when a loop is clearer · walk trees and graphs with depth-first and breadth-first search, and detect cycles · tell linear search from binary search, and know what sorted() costs · recognize memoization, dynamic programming, and greedy algorithms when you meet them · think about memory as well as speed · and talk through a coding problem the way interviewers expect.

Before you start: Chapter 17 (Python: lists, dictionaries, sets, loops, functions, CSV files), Chapter 18 (NumPy and pandas, which you'll compare with plain loops, and lambda), Chapter 28 (indexes, hash joins, and recursive CTEs, the database version of several ideas here), Chapter 29 (tests and generators), and Chapter 32 (a dbt project as a DAG).

Time needed: 14–18 hours, spread over three weeks, in four sittings: sections 33.1–33.4; recursion, trees and graphs (33.5–33.6); searching, sorting, memoization and dynamic programming (33.7–33.8); sections 33.9–33.10 and the project.

Sections: 33.1 Big-O: how work grows · 33.2 The four structures you'll use every day · 33.3 Hashing, and why dictionaries are instant · 33.4 Stacks and queues · 33.5 Recursion · 33.6 Trees and graphs · 33.7 Searching and sorting · 33.8 Memoization, dynamic programming, and greedy · 33.9 Memory, and the other costs · 33.10 What coding interviews actually test

In `companion/ch33/` — *notebook:* `ch33-notebook.ipynb` · *Python script:* `generate_ch33_data.py`, `match_fast.py`, `match_slow.py` · *dataset (csv):* `carrier_invoice_lines.csv`, `order_lines.csv`

Chapter 34. The Command Line, Linux & Networking Basics

You will learn to: combine small tools with pipes and redirection to answer questions in seconds · search text with `grep`, and count and summarize it with `wc`, `sort`, `uniq`, `cut`, and `awk` · find files by name, age, and size · read and set file permissions · use environment variables and understand `PATH` · write a shell script that takes arguments, fails safely, and returns a meaningful exit code · connect to a server with SSH keys and copy files to it · explain IP addresses, DNS, ports, and HTTP, and test a web server with `curl`.

Before you start: Chapter 26, section 26.0 (the terminal: moving around, making and copying files, reading a command, `&&`, environment variables, and `echo $?`) and Chapter 2 (files, formats, and what an API is). Chapter 20's scheduled jobs are useful background, and Chapter 14's regular expressions help with `grep -E`.

Time needed: 9–12 hours over two weeks, in three sittings (sections 34.0–34.4; 34.5–34.7; 34.8–34.9 and the project), most of it typing commands rather than reading.

Sections: 34.0 Where Chapter 26 left you · 34.1 Setting up for this chapter · 34.2 Pipes, redirection, and exit codes · 34.3 Looking inside files, and finding them · 34.4 Searching and summarizing text · 34.5 Permissions: who can read, write, and run · 34.6 Environment variables and `PATH` · 34.7 Shell scripts · 34.8 SSH: working on another machine · 34.9 Networking in plain English

In `companion/ch34/` — *Python script:* `daily_file_server.py`, `make_ch34_data.py` · *dataset (csv):* `orders_2025-11-01.csv`, `orders_2025-11-02.csv`, `orders_2025-11-03.csv`, `orders_2025-11-04.csv`, and 60 more

Part 4 — Machine Learning

Book 3 · Implementation · 10 chapters · roughly 144 hours in total

Ch	Chapter	Time	What you practise with
35	The Math Under the Models	14–17 hours	Python script, 3 dataset (csv), notebook
36	The Machine Learning Workflow & Feature Engineering	14–18 hours	3 dataset (csv), notebook
37	Supervised Learning Algorithms	20–24 hours	Python script, notebook
38	Unsupervised Learning	11–14 hours	Python script, notebook
39	Evaluation, Tuning, Interpretation & Honesty	16–20 hours	2 Python script, notebook
40	Time Series & Forecasting	16–20 hours	notebook, notes
41	NLP Foundations	12–15 hours	dataset (csv), notebook
42	Recommender Systems & Ranking	8–10 hours	Python script, notebook
43	A First Look at Deep Learning	12–15 hours	notebook, notes
44	Capstone: An End-to-End Data Science Project	5–7 hours	notebook, notes

Chapter 35. The Math Under the Models

You will learn to: treat each row of a dataset as a vector and measure distance and similarity between rows · multiply a matrix of features by a vector of weights to predict every row at once · read a derivative and a gradient as "which way is downhill, and how steep" · run gradient descent by hand for three steps, then in NumPy, and spot a bad learning rate or unscaled feature from the loss alone · describe data with the Bernoulli, binomial, Poisson, and normal distributions, and say when a distribution doesn't fit · find a maximum likelihood estimate and connect it to the loss a classifier minimizes · calculate entropy, cross-entropy, and log loss, and use them to judge a probability score · work through principal component analysis (PCA) on a small example by hand, then on all Riverstone customers.

Before you start: Chapter 4 (percentages, averages), Chapter 13 (the one-year database), Chapters 17 and 18 (Python, pandas, and the NumPy basics in section 18.1), Chapter 21 (mean, standard deviation, z-scores, probability, and the normal, binomial, and Poisson distributions with scipy.stats), Chapter 22, section 22.10 (fitting a straight line), and Chapter 31, section 31.0 (natural logarithms). No calculus or linear algebra is assumed, and section 35.8 adds the two facts about logarithms this chapter needs beyond section 31.0.

Time needed: 14–17 hours of reading and practice, spread over two to three weeks, in four sittings: A, sections 35.1–35.3 (vectors, dot products, matrices), about 3 hours; B, sections 35.4–35.6 (loss, derivatives, gradient descent), about 4 hours; C, sections 35.7–35.9 (distributions, likelihood,

logarithms, entropy, log loss), about 4 hours; D, sections 35.10–35.11 (PCA), about 3 hours; then the project. Sitzings B and C end with a ten-minute checkpoint. Do the hand calculations with a pen before you run the code.

Sections: 35.1 Vectors: every customer is a point · 35.2 The dot product: a number for "how aligned" · 35.3 Matrices: predicting every row at once · 35.4 Loss and the derivative: which way is downhill · 35.5 Gradients: the slope in every direction · 35.6 Gradient descent, by hand and in NumPy · 35.7 Probability distributions: describing what's likely · 35.8 Likelihood and maximum likelihood · 35.9 Entropy, cross-entropy, and log loss · 35.10 Principal component analysis (PCA) · 35.11 Which math sits under which model

In `companion/ch35/` — *notebook:* `ch35-notebook.ipynb` · *Python script:* `make_ch35_data.py` · *dataset (csv):* `customers_2025.csv`, `leads_2025.csv`, `orders_2025.csv`

Chapter 36. The Machine Learning Workflow & Feature Engineering

You will learn to: turn a business request into a machine learning problem with a clear target, prediction moment, and success measure · split data into training, validation, and test sets, and choose between a random and a time-based split · use scikit-learn's basic pattern (create, fit, then predict, `predict_proba` or `transform`) one piece at a time · use cross-validation and read the spread of its scores · engineer features from categories, numbers, dates, text, and activity logs · find data leakage with one question, and measure what each leak does to a score · handle missing values so the model learns from them instead of choking on them · build a scikit-learn pipeline that makes leakage from preprocessing impossible · set simple baselines and judge a model against them.

Before you start: Chapter 13 (deduplicating leads, section 13.7), Chapter 17 (Python basics), Chapter 18 (pandas, including `lambda` and dates), Chapter 21 (z-scores and samples), Chapter 22 (is a difference real?), Chapter 30 (logistic regression, met for inference in section 30.11), and Chapter 35 (loss, log loss, scaling, gradient descent, and the base-rate benchmark; it also installs scikit-learn).

Time needed: 14–18 hours over two weeks, in three sittings plus the project. Sitting A, sections 36.1–36.4 (framing, cleaning, splits, and your first scikit-learn cells): about 4 hours. Sitting B, sections 36.5–36.7 (cross-validation, feature engineering, leakage): about 5 hours. Sitting C, sections 36.8–36.10 (missing values, pipelines, baselines): about 3 hours. The project: 4–6 hours.

Sections: 36.1 Framing the problem · 36.2 One row per real enquiry · 36.3 Train, validation, and test sets · 36.4 scikit-learn, one piece at a time · 36.5 Cross-validation · 36.6 Feature engineering · 36.7 Data leakage · 36.8 Missing values in machine learning · 36.9 scikit-learn pipelines · 36.10 Baselines, and the final test

In `companion/ch36/` — *notebook:* `ch36-notebook.ipynb` · *dataset (csv):* `activities.csv`, `leads.csv`, `stage_history.csv`

Chapter 37. Supervised Learning Algorithms

You will learn to: explain how each major supervised learning algorithm makes a prediction, and calculate one small step of each by hand · fit and read linear regression, ridge, lasso, and elastic net · fit and read logistic regression, including odds ratios · use k-nearest neighbors and Naive Bayes, and know

their weak spots · build a decision tree split by hand with Gini and entropy, and see a tree overfit · explain why random forests and gradient boosting work, and use scikit-learn, XGBoost, LightGBM, and CatBoost · know when support vector machines are worth it · diagnose bias and variance with learning curves · tune hyperparameters with random search and Optuna without fooling yourself · pick an algorithm for a problem, and justify the choice.

Before you start: Chapter 21 §21.7 (Bayes' rule); Chapter 22 §22.10 (fitting a line, R^2 , residuals); Chapter 30 §30.11 (logistic regression, odds ratios, statsmodels); Chapter 35 (loss, gradients, likelihood, entropy); Chapter 36 (splits, cross-validation, leakage, pipelines, baselines). This chapter reuses Chapter 36's pipeline and lead-scoring data.

Time needed: 20–24 hours over three weeks, in two parts. Part A, sections 37.0 to 37.5 (the linear family, neighbors and Naive Bayes): about 9–10 hours. Part B, sections 37.6 to 37.12 and the project (trees, ensembles, tuning): about 11–13 hours. It's the longest chapter in Part 4; take it one algorithm at a time, and stop at the checkpoint between the parts.

Sections: 37.0 Setting up, and the data · 37.1 Linear regression · 37.2 Regularization: ridge, lasso, and elastic net · 37.3 Logistic regression · 37.4 k-nearest neighbors · 37.5 Naive Bayes · 37.6 Decision trees · 37.7 Random forests · 37.8 Gradient boosting · 37.9 Support vector machines · 37.10 Bias, variance, and learning curves · 37.11 Hyperparameter tuning · 37.12 Project result: lead scoring, baseline vs boosting

In [companion/ch37/](#) — *notebook:* [ch37-supervised-learning.ipynb](#) · *Python script:* [lead_data.py](#)

Chapter 38. Unsupervised Learning

You will learn to: run k-means by hand and in scikit-learn, and choose k with the elbow, the silhouette, stability, and business sense · profile and name clusters so a sales team can act on them · use hierarchical clustering and read a dendrogram · use DBSCAN, and know when density-based clustering suits a problem · reduce dimensions with PCA, t-SNE, and UMAP, and read their pictures without over-reading them · find anomalies with an isolation forest · run market basket analysis with support, confidence, and lift, by hand and with Apriori · judge unsupervised results, which have no test set to fall back on.

Before you start: Chapter 35 (distance, variance, and PCA, which you ran by hand in section 35.10), Chapter 36 (scaling, scikit-learn's fit, and pipelines), and Chapter 37 (the customer accounts dataset, and decision trees). No target variable is needed anywhere in this chapter.

Time needed: 11–14 hours over two weeks. There are two checkpoints: one after section 38.3 (the end of the first week) and one after section 38.8.

Sections: 38.0 The accounts, prepared · 38.1 k-means · 38.2 Choosing k, and whether the clusters are real · 38.3 Profiling and naming clusters · 38.4 Hierarchical clustering · 38.5 DBSCAN · 38.6 Seeing many dimensions: PCA, t-SNE, UMAP · 38.7 Anomaly detection · 38.8 Judging unsupervised results · 38.9 Market basket analysis

In [companion/ch38/](#) — *notebook:* [ch38-notebook.ipynb](#) · *Python script:* [account_features.py](#)

Chapter 39. Evaluation, Tuning, Interpretation & Honesty

You will learn to: build a confusion matrix and compute precision, recall, F1, and specificity from it by hand · draw and read ROC and precision–recall curves, and know which one to trust on imbalanced data · judge a regression model with MAE, RMSE, MAPE, and R^2 , and know what each hides · check whether predicted probabilities are honest (calibration) and fix them when they aren't · choose a decision threshold from business costs and team capacity, not from a default of 0.5 · handle imbalanced classes with class weights and resampling, and know what they do and don't change · tune a model's settings without fooling yourself, and keep tuning and threshold choice off the test set · explain a model globally and one prediction at a time with permutation importance, SHAP, and partial dependence · check whether a model treats groups of people fairly · write a model card that says what a model is for and where it fails.

Before you start: Chapter 35 (log loss and the base-rate benchmark, section 35.9), Chapter 36 (the lead-scoring pipeline and validation set, scikit-learn's pieces in section 36.4, cross-validation and AUC in section 36.5), Chapter 37 (logistic regression, Naive Bayes, the churn models, and tuning in section 37.11). This chapter evaluates the models you already built.

Time needed: 16–20 hours over two to three weeks, in four sittings: (1) sections 39.1–39.3, the metrics, each worked by hand before the code (4–5 hours); (2) sections 39.4–39.7, calibration, thresholds, imbalance, and tuning (4–5 hours); (3) sections 39.8–39.10, interpretation, fairness, and model cards (4–5 hours); (4) the exercises and the project (4–5 hours).

Sections: 39.1 The confusion matrix · 39.2 Curves: ROC and precision–recall · 39.3 Regression metrics · 39.4 Calibration: are the probabilities honest? · 39.5 Choosing the threshold by business cost · 39.6 Imbalanced data · 39.7 Tuning honestly · 39.8 Interpretation: permutation importance, SHAP, and partial dependence · 39.9 Fairness checks · 39.10 Model cards

In `companion/ch39/` — *notebook:* `ch39-notebook.ipynb` · *Python script:* `churn_data.py`, `lead_data.py`

Chapter 40. Time Series & Forecasting

You will learn to: break a time series into trend, seasonality, cycle, and noise, first by hand · measure autocorrelation, test for stationarity, and know why it matters · build the baselines every forecast must beat, by hand · run exponential smoothing and Holt–Winters, and see the smoothing steps on paper · read ACF and PACF plots and fit a SARIMA model · forecast with a machine learning model on lag features without leaking the future · backtest a forecast the way it will be used, and read the spread of errors · measure accuracy with MAPE, WAPE, and MASE, and know which to quote · turn a forecast into a production plan with safety stock · find anomalies in sensor data, including the kind that point-by-point checks miss.

Before you start: Chapter 18, section 18.9 (dates, resample, rolling, and shift in pandas), Chapter 21 (means, standard deviations, z-scores, and the normal distribution), Chapter 22 (confidence intervals, p-values, and correlation, sections 22.1, 22.2, and 22.5), Chapter 35, section 35.8 (likelihood), Chapter 36 (splits and leakage), Chapter 37, section 37.8 (gradient boosting), and Chapter 39, section 39.3 (MAE, MAPE, and WAPE).

Time needed: 16–20 hours over three weeks, in four sittings: sections 40.0–40.3 (the data, its parts, autocorrelation, stationarity, baselines); 40.4–40.5 (smoothing and SARIMA); 40.6–40.9 (machine learning, backtesting, metrics, the plan); and 40.10 with the project (sensor anomalies).

Sections: 40.0 Setting up · 40.1 What a time series is made of · 40.2 Autocorrelation and stationarity · 40.3 Baselines that are hard to beat · 40.4 Exponential smoothing · 40.5 ARIMA and SARIMA · 40.6 Machine learning for forecasting · 40.7 Backtesting · 40.8 Forecast accuracy: MAPE, WAPE, MASE · 40.9 Demand forecasting for a manufacturer · 40.10 Anomaly detection in sensor data

In `companion/ch40/` — *notebook:* `ch40-notebook.ipynb`

Chapter 41. NLP Foundations

You will learn to: clean and tokenize free text, and choose between stemming and lemmatization · build a bag-of-words and a TF-IDF representation by hand before using a library · measure similarity between documents · classify text with Naive Bayes and logistic regression, and read a confusion matrix for more than two classes · score sentiment with a word lexicon and know when it fails · train a sentiment classifier and compare it honestly with the lexicon · find topics in unlabelled text with LDA and NMF, and judge whether they mean anything · train word embeddings and see why they're the bridge to modern language models.

Before you start: Chapter 35 (dot product, vector length, cosine similarity, PCA), Chapter 36 (splits, pipelines, sparse matrices), Chapter 37 (Naive Bayes, logistic regression), Chapter 38 (judging clusters), Chapter 39 (precision, recall, F1). This chapter treats text as another kind of feature, built on those foundations.

Time needed: 12–15 hours over two weeks. Take it in two sittings: sections 41.0 to 41.4 (turning text into numbers, and classifying it), then sections 41.5 to 41.7 (sentiment, topics, and embeddings).

Sections: 41.0 Setting up · 41.1 Text as data · 41.2 Cleaning and tokenization · 41.3 Bag of words and TF-IDF · 41.4 Text classification · 41.5 Sentiment analysis · 41.6 Topic modeling · 41.7 Word embeddings: the bridge to LLMs

In `companion/ch41/` — *notebook:* `ch41-notebook.ipynb` · *dataset (csv):* `tickets.csv`

Chapter 42. Recommender Systems & Ranking

You will learn to: build a popularity baseline, and know why it's hard to beat with little data · build item-based collaborative filtering by hand and in scikit-learn · evaluate a ranked list with hit rate@k, precision@k, recall@k, NDCG@k and MRR, not accuracy · understand matrix factorization by hand, then fit it with scikit-learn's TruncatedSVD and the implicit library's ALS · build a content-based recommender from item descriptions, reusing Chapter 41's TF-IDF and cosine similarity · combine methods into a hybrid recommender · handle the cold-start problem for new products and new customers · apply all of this to B2B cross-selling, and test a segment-level baseline against every model.

Before you start: Chapter 35 (dot products, cosine similarity, PCA), Chapter 36 (baselines, the log transform, sparse matrices, train/test discipline), Chapter 37 (the accounts data, least squares, the ridge penalty, bias and variance), Chapter 38 (the basket data and market basket rules), Chapter 39

(precision, recall, average precision, the top-N table), Chapter 41 (TF-IDF, NMF). This chapter turns those pieces into personalized recommendations.

Time needed: 8–10 hours over one week. Take it in two sittings: sections 42.0 to 42.3 (the data, collaborative filtering and how to score a ranked list), then sections 42.4 to 42.7 (matrix factorization, content, cold start and the business case).

Sections: 42.0 Setting up · 42.1 The data, and a popularity baseline · 42.2 Item-based collaborative filtering · 42.3 Evaluating recommendations: hit rate, precision, NDCG · 42.4 Matrix factorization and implicit feedback · 42.5 Content-based filtering · 42.6 The cold-start problem · 42.7 B2B cross-selling and ranking by segment

In companion/ch42/ — *notebook:* ch42-notebook.ipynb · *Python script:* products_text.py

Chapter 43. A First Look at Deep Learning

You will learn to: install PyTorch and use its tensors · compute a single neuron by hand and recognize it as logistic regression in disguise · explain why one neuron can't solve every problem, and watch a network fail and then succeed on a real example · compute a full forward pass through a small network by hand and match it in PyTorch · work one backward pass by hand with the chain rule, and check it against PyTorch's automatic gradients and a hand-nudged gradient · train a small neural network on tabular data and compare it honestly with logistic regression and gradient boosting · understand what a convolution does, and train a small image classifier · walk through transfer learning end to end, and see when it helps and when it doesn't.

Before you start: Chapter 35 (vectors, matrix products, derivatives and gradients, gradient descent, log loss), Chapter 36 (splits, scikit-learn, pipelines), Chapter 37 (logistic regression and the sigmoid, gradient boosting, bias and variance), Chapter 29 (classes, section 29.5), Chapter 39 (AUC, and why accuracy misleads on imbalanced data), Chapter 41 (softmax, and word embeddings as a preview of learned representations).

Time needed: 12–15 hours over two weeks, in three sittings: sections 43.0–43.4 (PyTorch, neurons, and the backward pass by hand), about 5 hours; sections 43.5–43.6 (a network on tables, and one on images), about 4 hours; section 43.7 and the project, about 4 hours, plus the exercises. Everything runs on an ordinary laptop's CPU in a few minutes; no GPU is needed.

Sections: 43.0 Setting up · 43.1 A neuron is logistic regression · 43.2 Why one neuron isn't enough · 43.3 A full forward pass, by hand · 43.4 Backpropagation: the chain rule, automated · 43.5 A network on tabular data · 43.6 A first image classifier · 43.7 Transfer learning · 43.8 When to reach for deep learning

In companion/ch43/ — *notebook:* ch43-notebook.ipynb

Chapter 44. Capstone: An End-to-End Data Science Project

You will learn to: walk one business question through the whole Part 4 lifecycle — frame, prepare, model, evaluate, and communicate — without skipping a stage · turn a model's probability into rupees, and see why ranking by probability alone can miss most of the value · decide which accounts are worth a call at all, and which are worth it first · split an analysis into small reusable functions, so the model is

fitted once and the list can be rebuilt with one line · write a one-page, non-technical summary that a leader can act on, and know what belongs in it and what doesn't.

Before you start: this chapter assumes everything from Chapters 35–43, and Chapter 30's experiments. It does not re-teach any of it; it uses it. If a step feels unfamiliar, the chapter says which earlier chapter and section taught it, so you can go back rather than guess.

Time needed: 5–7 hours to read the chapter, re-run the code, and do exercises 1–4. The project, your own end-to-end pass, is another 6–10 hours.

Sections: 44.1 The question, and the plan · 44.2 Refitting the model (a recap, not a re-teach) · 44.3 From probability to rupees · 44.4 Building the call list: probability isn't the same as value · 44.5 Explaining two accounts · 44.6 Two functions, not six steps · 44.7 The one-page summary · 44.8 The Part 4 toolkit, applied

In `companion/ch44/` — *notebook:* `ch44-notebook.ipynb`

Part 5 — Data Engineering

Book 3 · Implementation · 8 chapters · roughly 139 hours in total

Ch	Chapter	Time	What you practise with
45	Data Ingestion & Integration	15–20 hours	4 Python script, notebook
46	Pipelines & Orchestration	16–20 hours	6 Python script, notebook
47	Data Quality, Observability & Contracts	14–18 hours	5 Python script, 2 SQL, 2 config, notebook
48	Big Data & Distributed Compute	14–18 hours	2 Python script, 92 dataset (parquet), notebook
49	Storage, Warehouses & Lakehouses	14–17 hours	2 Python script, 30 data (json), dataset (csv), 30 dataset (parquet), notebook
50	Streaming & Real-Time	15–19 hours	2 Python script, 5 data (json), notebook
51	Data Activation: Reverse ETL, APIs & System Integration	14–18 hours	4 Python script, 4 SQL, notebook
52	The Cloud, Containers & Infrastructure as Code	20–26 hours	4 Python script, 2 config, notebook, text

Chapter 45. Data Ingestion & Integration

You will learn to: name the kinds of source systems a data platform ingests from, and what makes each one hard · choose between full loads, incremental loads, and change data capture, and explain

what each one can miss · show, with a real test, why a "new rows only" load silently loses updates · compute and use SHA-256 and MD5 fingerprints · detect inserted, changed, and deleted rows with hashes, and skip files that haven't changed · read real change events from PostgreSQL's log · load messy CSV files without silent type errors, and reconcile what you loaded against the file · recognize schema changes before they break a load · pull data from a paginated, authenticated, rate-limited API with retries · decide when to build a connector and when to buy one · judge the legal and ethical limits of collecting web data.

Before you start: Chapter 2 (formats; the idea of a file fingerprint; what an API is). Chapter 12 (transactions, upsert, loading a CSV). Chapters 13 and 28 (SQL). Chapter 17 (the virtual environment and Jupyter). Chapter 18 (calling an API, .env files). Chapter 20 (idempotency with a run key). Chapter 26 section 26.0 (the terminal). Chapter 29 (Python as software; the CRM client). Chapter 32 (dbt) helps but isn't required.

Time needed: 15–20 hours of reading and practice, spread over two to three weeks. Plan four sittings: sections 45.1–45.4; sections 45.5–45.6 (hashes, upserts, change data capture); sections 45.7–45.8 (files and schema changes); sections 45.9–45.11 and the project.

Sections: 45.1 Where company data comes from · 45.2 Setting up the practice environment · 45.3 Full loads · 45.4 Incremental loads, and the update they miss · 45.5 Detecting change with hashes · 45.6 Change data capture · 45.7 Loading files reliably · 45.8 When the source changes shape · 45.9 Pulling data from APIs · 45.10 Build or buy: connectors versus custom code · 45.11 Web data: legal and ethical limits

In companion/ch45/ — *notebook:* ch45-notebook.ipynb · *Python script:* apply_day.py, make_files.py, mock_crm_api.py, reset_ch45.py

Chapter 46. Pipelines & Orchestration

You will learn to: explain what an orchestrator adds to scheduled scripts · describe a pipeline as a graph of assets with dependencies, schedules, and partitions · build Riverstone's daily pipeline in Dagster, from ingestion to a delivered Daily Sales Flash · prove a pipeline step is idempotent with a deliberate failure test, and show what happens when it isn't · rebuild history with a backfill without re-sending old reports · use retries for transient failures without hiding real bugs · stop delivery when the data fails its checks, and send an alert that tells someone what to do · handle late-arriving corrections · read and write the same pipeline in Airflow · choose an orchestrator, and run pipelines responsibly.

Before you start: Chapter 45 (ingestion, hashes, upserts, idempotency; your ingestion project), Chapter 20 (the Daily Sales Flash, cron, and scheduling), Chapter 29 (modules, decorators, generators, tests), Chapter 32 (dbt's DAG), Chapter 12 (transactions: BEGIN, COMMIT, ROLLBACK), Chapter 26, section 26.0 (the terminal), and Chapter 17 (the virtual environment and Jupyter).

Time needed: 16–20 hours of reading and practice, spread over two to three weeks, in four sittings: sections 46.1–46.3 (the ideas and the first run); 46.4–46.5 (idempotency, backfills, late data); 46.6–46.7 (retries, sensors, checks, alerts); 46.8–46.10 and the project.

Sections: 46.1 From scheduled scripts to pipelines · 46.2 The core ideas · 46.3 Building Riverstone's daily pipeline · 46.4 Idempotency, proven with a failure test · 46.5 Partitions and backfills · 46.6 Retries,

timeouts, and alerting · 46.7 Delivery only after the data passes its checks · 46.8 The same pipeline in Airflow · 46.9 Choosing an orchestrator · 46.10 Operating pipelines

In companion/ch46/ — *notebook*: ch46-notebook.ipynb · *Python script*: apply_day.py, ingest.py, make_files.py, mock_crm_api.py, and 2 more

Chapter 47. Data Quality, Observability & Contracts

You will learn to: name the dimensions of data quality and write a test for each · build a small test framework where every test is a query that returns the rows that break a rule · choose severities so that only real problems stop a pipeline · use write–audit–publish so bad data never reaches the tables people read · monitor freshness and volume, and spot unusual days against history · trace lineage to see which reports a broken table affects · write a data contract with a source owner and check it automatically · run a data incident: severity, communication, fix, and review · keep alerts few enough that people still read them · run the same rules in dbt, Great Expectations, and Soda, and know where observability platforms fit.

Before you start: Chapter 45 (ingestion and reconciliation), Chapter 46 (the orchestrated pipeline), Chapter 32 (dbt), and Chapter 14 (finding and fixing data problems as an analyst).

Time needed: 14–18 hours of reading and practice, spread over two to three weeks, in four sittings: sections 47.1–47.3; section 47.4; sections 47.5–47.7; sections 47.8–47.10 and the project.

Sections: 47.1 The dimensions of data quality · 47.2 Testing data inside the pipeline · 47.3 A check library for Riverstone · 47.4 Write–audit–publish · 47.5 Observability: freshness, volume, and unusual values · 47.6 Lineage: what breaks when this breaks · 47.7 Data contracts in practice · 47.8 Handling a data incident · 47.9 Alerts people still read · 47.10 The tools landscape

In companion/ch47/ — *notebook*: ch47-notebook.ipynb · *SQL*: ch47_queries_mysql.sql, ch47_queries_postgresql.sql · *Python script*: apply_day.py, ingest.py, make_files.py, mock_crm_api.py, and 1 more

Chapter 48. Big Data & Distributed Compute

You will learn to: install Spark and Polars and start your first Spark session · judge for yourself whether a dataset needs distributed computing at all · explain partitions, workers, the driver, and why moving data between machines is what costs · write PySpark that reads, filters, groups, joins, and writes Riverstone's sensor data · read a physical query plan with explain() and find the shuffle in it · use partition pruning, column pruning, and broadcast joins, and see each one change the plan · recognize skew and know the usual fixes · use Polars for the first time · compare DuckDB, Polars, and Spark on the same question and interpret the timings · avoid the common ways a Spark job surprises you · place Databricks, EMR, Dataproc, Fabric, Trino, and Ray on the map.

Before you start: Chapter 17 (the virtual environment and Jupyter). Chapter 18 (pandas, and Parquet). Chapter 26 section 26.0 (the terminal). Chapter 29 (Python as software). Chapter 45 (DuckDB from Python, and Parquet). Chapter 46 (pipelines and idempotent re-runs). Chapter 14's regular expressions help with one optional helper in section 48.4.

Time needed: 14–18 hours of reading and practice, spread over two to three weeks. Plan four sittings: section 48.0, installing Java, Spark, and Polars and starting your first Spark session (1–2 hours, more if Java fights you); sections 48.1–48.4; sections 48.5–48.6 (making jobs faster, Polars, three engines); section 48.7 onwards and the project.

Sections: 48.0 Setting up Spark, Polars, and the data · 48.1 Most "big data" isn't · 48.2 What "distributed" actually means · 48.3 PySpark on the sensor data · 48.4 Reading the plan, and finding the shuffle · 48.5 Making a job faster · 48.6 The same question in three tools · 48.7 Writing Spark that doesn't surprise you · 48.8 The wider landscape in one page

In companion/ch48/ — *notebook:* ch48-notebook.ipynb · *Python script:* make_sensor_data.py, time_three_tools.py · *dataset (parquet):* part-0.parquet, part-0.parquet, part-0.parquet, part-0.parquet, and 88 more

Chapter 49. Storage, Warehouses & Lakehouses

You will learn to: say where each kind of data belongs: files, object storage, databases, warehouses, lakes, lakehouses · explain row versus columnar storage, and show the difference in file size, read time, and what a query has to read · read a Parquet file's internals: row groups, column chunks, encodings, compression, statistics · explain ACID properly, and why plain files don't have it · turn a folder of Parquet into a real table with a transaction log, then overwrite one machine's data atomically, travel back to an earlier version, restore it, change the schema safely, and compact small files · choose partition columns and file sizes on purpose · explain how cloud warehouses separate storage from compute, and what actually drives their bills.

Before you start: Chapter 12 (transactions: BEGIN, COMMIT, ROLLBACK), Chapter 17 (the virtual environment and Jupyter), Chapter 45 (DuckDB from Python, files and Parquet), Chapter 46 (idempotent writes), Chapter 47 (write–audit–publish), and Chapter 48 (partitions, pruning, and the sensor dataset).

Time needed: 14–17 hours of reading and practice, spread over two to three weeks. Plan three sittings: sections 49.0–49.4 (setting up, where data sits, row versus columnar, inside a Parquet file, ACID); section 49.5 (table formats, the longest section); sections 49.6 onwards and the project.

Sections: 49.0 Setting up · 49.1 Where data sits · 49.2 Row versus columnar, measured · 49.3 Inside a Parquet file · 49.4 ACID, properly · 49.5 Table formats: a folder of files that behaves like a table · 49.6 Partitioning and file sizing · 49.7 Warehouses: storage and compute, separated · 49.8 What it costs · 49.9 Choosing a layout

In companion/ch49/ — *notebook:* ch49-notebook.ipynb · *Python script:* format_test.py, setup_ch49.py · *dataset (csv):* day.csv · *dataset (parquet):* part-00000-4c144aad-dbe5-4604-aba3-bdb1321f5077-c000.snappy.parquet, part-00000-74e9babf-433c-4cd2-ad1b-38e9e2a10618-c000.snappy.parquet, part-00000-b35be30a-f868-4c8f-a56b-d2c9ea8d033a-c000.snappy.parquet, part-00000-d6c2fa6d-4ea7-4262-9a1f-e1251c143095-c000.snappy.parquet, and 26 more

Chapter 50. Streaming & Real-Time

You will learn to: tell the difference between "real-time" as a business word and as an engineering commitment · explain topics, partitions, offsets, consumer groups, and why order is only guaranteed inside a partition · produce and consume events, commit offsets, and handle the duplicates that at-least-once delivery creates · run a real Kafka broker on your own computer, and read and write the Kafka client code you'll meet at work · count events in tumbling, sliding, and session windows, by hand and in Spark · build a Spark Structured Streaming job with event-time windows and a watermark · show what happens to late data, both when it's accepted and when it's dropped · keep a streaming sink from drowning in small files · monitor a stream: lag, watermark, state size, and dropped rows · decide when streaming is worth its cost, using Riverstone's plant monitoring case.

Before you start: Chapter 45 (ingestion, and its incremental-load watermark), Chapter 46 (idempotency and the delivery log), Chapter 47 (quality and alerts), Chapter 48 (Spark, and the sensor data this chapter replays), and Chapter 49 (table formats and compaction). Section 50.3's optional broker uses the terminal from Chapter 34.

Time needed: 15–19 hours of reading and practice, spread over two to three weeks. Plan four sittings: sections 50.0–50.2; section 50.3, plus 30–45 minutes if you run the optional Kafka broker; sections 50.4–50.5, the heart of the chapter; section 50.6 onwards and the project.

Sections: 50.0 Setting up · 50.1 What "real-time" actually means · 50.2 The log: topics, partitions, offsets, groups · 50.3 Delivery guarantees, and the duplicates you will get · 50.4 Streaming with Spark: the same code, running forever · 50.5 Event time, windows, and watermarks · 50.6 Writing a stream into a table · 50.7 Operating a stream · 50.8 Riverstone's plant monitoring case

In `companion/ch50/` — *notebook:* `ch50-notebook.ipynb` · *Python script:* `mini_log.py`, `sensor_events.py`

Chapter 51. Data Activation: Reverse ETL, APIs & System Integration

You will learn to: explain why a number sitting in a warehouse or a dashboard changes nothing on its own · describe reverse ETL and the kinds of fields companies sync back into operational systems · write to a system through an API with authentication, upserts, pagination, rate limits, and retries · make a write idempotent with an idempotency key, and prove a retried write doesn't double up · choose a system of record and a conflict rule before two systems can disagree · trigger a webhook, receive the event within seconds instead of polling, and check its signature · compare integration patterns: point-to-point, hub-and-spoke, message queues, and iPaaS · decide between custom code, orchestrator tasks, low-code tools, and enterprise platforms · explain why RPA is fragile, and when file-based integration (SFTP, EDI) is still the right answer · build an audit trail and a reconciliation report for writes, not only reads.

Before you start: Chapter 45 (ingestion, idempotency, APIs), Chapter 46 (orchestration and delivery), Chapter 47 (contracts and incidents), and Chapters 12 and 13 (SQL, including window functions).

Time needed: 14–18 hours of reading and practice, spread over two to three weeks.

Sections: 51.0 Setting up the practice environment · 51.1 The loop nobody finishes · 51.2 Computing what to sync · 51.3 Writing to a system through an API · 51.4 Making writes idempotent, and proving it · 51.5 System of record and conflict rules · 51.6 Webhooks: pushing instead of polling · 51.7 Reconciliation for writes · 51.8 Integration patterns · 51.9 Choosing how to build it · 51.10 When there's no API: RPA and file drops

In companion/ch51/ — *notebook:* ch51-notebook.ipynb · *SQL:* ch51_queries_mysql.sql, ch51_queries_postgresql.sql, follow_up_due.sql, lead_scores.sql · *Python script:* crm_sandbox.py, reset_ch51.py, sync_leads.py, webhook_receiver.py

Chapter 52. The Cloud, Containers & Infrastructure as Code

You will learn to: say what IaaS, PaaS, and SaaS rent you, and where the shared responsibility line falls for each · explain regions and availability zones · write a least-privilege access policy and read every field of it · plan a network with CIDR arithmetic you can do by hand · install Docker, build an image of Chapter 46's pipeline, and run it · run a small stack of containers with Docker Compose · read a Kubernetes CronJob, Deployment, and Service and say what each line does · write a small Terraform configuration, check it, and read its plan · build a CI/CD workflow and trace exactly which jobs run for a given event · keep secrets out of code and images · estimate what running a pipeline in the cloud actually costs · choose a deployment target that fits a company Riverstone's size.

Before you start: Chapter 26 (the terminal, Git, pull requests, .gitignore and .env, GitHub Actions). Chapter 29 (tests with pytest). Chapter 34 (environment variables, IP addresses, ports, private address ranges, and WSL on Windows). Chapter 45 (the practice source and CRM API), Chapter 46 (the pipeline this chapter deploys), Chapter 47 (checks and incidents), Chapter 49 (storage and its cost), Chapter 51 (webhooks).

Time needed: 20–26 hours of reading and practice, spread over three to four weeks. Docker (section 52.2) and Terraform (section 52.4) are hands-on and take the most time; read Kubernetes (section 52.3) for its vocabulary, and try it on your laptop only if you want to.

Sections: 52.0 Setting up · 52.1 Cloud fundamentals · 52.2 Containers · 52.3 Orchestrating containers · 52.4 Infrastructure as Code · 52.5 CI/CD · 52.6 Secrets, done properly · 52.7 Cost · 52.8 Choosing a deployment target

In companion/ch52/ — *notebook:* ch52-notebook.ipynb · *Python script:* ingest.py, riverstone_pipeline.py, run_flash.py, simulate_workflow.py

Part 6 — Production ML & Generative AI

Book 3 · Implementation · 7 chapters · roughly 121 hours in total

Ch	Chapter	Time	What you practise with
53	Deep Learning in Depth	17–20 hours	Python script, notebook
54	Generative AI & Large Language Models	16–20 hours	5 Python script, data (json), notebook, notes, 60 text
55	Building AI Applications: RAG, Agents & Evaluation	20–24 hours	4 Python script, data (json), notebook, 28 notes
56	MLOps: Making Models Survive Production	18–22 hours	4 Python script, data (json), notebook
57	LLMOps	16–20 hours	4 Python script, notebook
58	Intelligent Automation	16–20 hours	2 Python script, notebook
59	Industry Case Studies	6–8 hours	<i>reading only</i>

Chapter 53. Deep Learning in Depth

You will learn to: describe a neural network as a stack of very simple parts · follow one training step by hand, from forward pass to updated weights, and check it with a nudge · see what an optimizer such as Adam does differently from plain gradient descent · use the techniques that make training work (feature scaling, weight decay, initialization, batch normalization, dropout, learning-rate schedules, early stopping) · say what convolution and pooling do to an image, by computing them · build and judge a defect-detection model for Riverstone's moulding line, choosing its threshold from the cost of each kind of mistake, then let PyTorch learn the kernels · work through a transformer's attention step by step on four tokens · explain quantization, pruning, and distillation, and measure what quantization costs and saves · and say when deep learning is the wrong tool.

Before you start: Chapter 35 (vectors, matrix products, derivatives and gradient descent, cross-entropy), Chapter 36 (train/test splits, cross-validation, leakage), Chapter 37 (linear and logistic regression, ridge, gradient boosting, overfitting), Chapter 39 (confusion matrix, precision and recall, choosing a threshold by cost), Chapter 41 (tokens and word embeddings), Chapter 43 (a first look at deep learning: neurons, the chain rule, PyTorch, a small CNN). Helpful: Chapter 22's regression basics (section 22.10), Chapter 33 (Big-O).

Time needed: 17–20 hours, spread over three weeks, in three sittings: sections 53.0–53.3 (setup, a training step by hand, and the techniques), about 6 hours; sections 53.4–53.6 (convolution and the defect project), about 6 hours; sections 53.7–53.10 and the project, about 5 hours, plus the exercises.

Sections: 53.0 Setting up · 53.1 From regression to a network · 53.2 Training, worked by hand · 53.3 The techniques that make training work · 53.4 The architectures, and what each is for · 53.5 Convolution, step by step · 53.6 The project: defect detection on Riverstone's moulding line · 53.7 Attention, step by step · 53.8 Why transformers took over · 53.9 Quantization and compression · 53.10 When not to use deep learning

In [companion/ch53/](#) — *notebook:* [ch53-notebook.ipynb](#) · *Python script:* [generate_defect_images.py](#)

Chapter 54. Generative AI & Large Language Models

You will learn to: say what a language model actually computes, and why that explains both its fluency and its mistakes · train a byte-pair tokenizer on Riverstone's own text and count tokens the way a bill counts them · describe the three stages that turn raw pretraining into a model that follows instructions · reason about context windows, cost, and latency · set temperature and top-p deliberately, having worked them by hand and watched them change measured outputs · write prompts that work, and measure the improvement instead of asserting it · get structured output you can parse, validate, and retry · build embeddings and search with them · choose between prompting, retrieval, and fine-tuning · name the limits that decide whether a use case is safe to ship · and make your first call to a real hosted model.

Before you start: Chapter 53 (softmax, attention, quantization), Chapter 41 (tokens, TF-IDF, word embeddings), Chapter 42 (TruncatedSVD), Chapter 35 (the dot product and cosine similarity), Chapter 39 (evaluation and error analysis), Chapter 29 (modules, settings in .env, exit codes), Chapter 26 (the terminal, environment variables, Git).

Time needed: 16–20 hours, spread over three weeks.

Sections: 54.0 Setting up · 54.1 What the model computes · 54.2 Tokens: the units of everything · 54.3 How a chat model is built · 54.4 Context windows · 54.5 Sampling: temperature and top-p · 54.6 Prompting, measured · 54.7 Structured output you can trust · 54.8 Embeddings: text as coordinates · 54.9 Prompt, retrieve, or fine-tune? · 54.10 Multimodal models · 54.11 Limits and risks · 54.12 The landscape, and pricing a workload · 54.13 Your first real call (optional: needs an account)

In companion/ch54/ — *notebook:* ch54-notebook.ipynb · *Python script:* api_example.py, extraction.py, generate_order_emails.py, mock_llm.py, and 1 more

Chapter 55. Building AI Applications: RAG, Agents & Evaluation

You will learn to: explain why retrieval exists and when it beats a longer prompt · turn a folder of company documents into a searchable index, choosing a chunking strategy with numbers rather than taste · implement BM25 keyword search from scratch and combine it with vector search · measure retrieval with recall@k and mean reciprocal rank on your own questions · ground answers in retrieved text and cite the source · give an assistant tools it may call, and keep the decision to run them in your code · say plainly when an agent is the right shape and when a script is · evaluate a whole AI application with golden sets, rubrics, and human review · set guardrails, including against injection through your own documents · and decide what to log, what to cache, and what a question costs.

Before you start: Chapter 54 (tokens, prompting, structured output, embeddings), Chapter 41 (tokenizing text and TF-IDF), Chapter 42 (ranking metrics: hit rate@k, MRR, NDCG), Chapter 39 (evaluation: recall, precision, and choosing a threshold by cost), Chapter 53 (a threshold chosen with money, section 53.6), Chapter 29 (tested code, modules, classes), Chapter 33 (why an index beats a scan), and Chapter 14's regular expressions (section 14.2).

Time needed: 20–24 hours, spread over three weeks, in three sittings: sections 55.0–55.5 (setup, chunking, search, and measuring it), about 9 hours; sections 55.6–55.8 (grounding, refusal, tools,

agents), about 6 hours; sections 55.9–55.11 (evaluation, guardrails, shipping), about 4 hours; the project and exercises take the rest.

Sections: 55.0 Setting up · 55.1 Why retrieval, and not a longer prompt · 55.2 What is actually in the documents · 55.3 Chunking: the decision nobody measures · 55.4 Two ways to search · 55.5 Hybrid search, and measuring all of it · 55.6 Grounding, citations, and refusal · 55.7 Tools: when the answer isn't in a document · 55.8 Agents, without the hype · 55.9 Evaluating the whole application · 55.10 Guardrails · 55.11 Shipping it

In `companion/ch55/` — *notebook:* `ch55-notebook.ipynb` · *Python script:* `assistant.py`, `generate_corpus.py`, `retrieval.py`, `tools.py`

Chapter 56. MLOps: Making Models Survive Production

You will learn to: name the four ways a working model stops working · version the five things that make up "the model" · track experiments so every run can be compared instead of remembered · register a model and point a name at the version in use · package a model with the metadata that has to travel with it · serve it behind a real HTTP endpoint with validation, versioning, and a log line monitoring can read · choose between batch, online, shadow, canary, and blue-green · monitor in three layers, and know which layer sees which failure · compute drift with PSI and the KS statistic, and see why drift alarms and model decay are not the same thing · decide when to retrain, and what to check before shipping the retrain · run an incident and write the post-mortem · and judge how far up the MLOps maturity ladder your company should actually climb.

Before you start: Chapter 53 (the defect model this chapter operates, and its threshold chosen with money), Chapter 29 (tested, packaged Python; classes, decorators, type hints, logging), Chapter 34 (the command line, background jobs, HTTP and curl), Chapters 26 and 32 (CI: a check on every push), Chapter 39 (evaluation: recall, precision, a threshold chosen by cost), Chapter 30 (A/B tests), and Chapter 52 (containers).

Time needed: 18–22 hours, spread over three weeks, in three sittings: sections 56.0–56.5 (setup, versioning, tracking, packaging, and a running service), about 8 hours; sections 56.6–56.9 (deployment, monitoring, drift, and retraining), about 7 hours; sections 56.10–56.11, the project and the exercises take the rest.

Sections: 56.0 Setting up · 56.1 What breaks after launch · 56.2 Versioning: "the model" is five things · 56.3 Experiment tracking · 56.4 Packaging: what travels with the model · 56.5 Serving · 56.6 Deployment patterns · 56.7 Monitoring, in three layers · 56.8 Drift, measured · 56.9 Retraining · 56.10 When it goes wrong · 56.11 How far up the ladder to climb

In `companion/ch56/` — *notebook:* `ch56-notebook.ipynb` · *Python script:* `ci_model_check.py`, `serve.py`, `simulate_production.py`, `train.py`

Chapter 57. LLMOps

You will learn to: name what LLMOps adds to Chapter 56's discipline, and why the model you depend on isn't yours · keep prompts in a registry as versioned code with a score attached · run a golden set in CI with a floor that fails the build · survive the day a provider ships a new model version · build a

token meter and project a monthly bill · build a cache, and measure what it saves on real traffic · budget latency · retry, fall back, and degrade honestly when the provider fails · monitor a system that has no accuracy to monitor · keep logs that are useful and lawful · and run the human review loop that turns complaints into test cases.

Before you start: Chapter 54 (prompting, structured output, token costs, the order-extraction golden set), Chapter 55 (the support assistant, its test questions, guardrails), Chapter 56 (versioning, serving, monitoring, retraining, incidents), Chapter 29 (classes, retries and backoff, exit codes), Chapter 45 (hashes).

Time needed: 16–20 hours, spread over three weeks: sections 57.0 to 57.4 in the first, 57.5 to 57.8 in the second, and 57.9 to 57.11 with the project in the third.

Sections: 57.0 Setting up · 57.1 What changes, and what doesn't · 57.2 Prompts as code · 57.3 The golden set as a build step · 57.4 The day the provider changes the model · 57.5 Tokens, cost, and the meter · 57.6 Caching · 57.7 Latency · 57.8 Failure and fallback · 57.9 Monitoring without ground truth · 57.10 Safety and logs in production · 57.11 The human loop

In `companion/ch57/` — *notebook:* `ch57-notebook.ipynb` · *Python script:* `ci_eval.py`, `pipeline.py`, `provider.py`, `questions_stream.py`

Chapter 58. Intelligent Automation

You will learn to: tell the three generations of automation apart and pick the right one · choose what to automate using volume, variability, rules, and the cost of an error · assemble Chapters 54 to 57 into a pipeline that writes to a system of record · make writes idempotent, transactional, and auditable · design an approval threshold priced in money rather than guessed · classify exceptions so each one gets retried, held, escalated, or rejected · measure straight-through rate and silent error rate, which are the same system described two ways · do the ROI arithmetic honestly, including the review time and the cost of being wrong · roll out in stages with a gate at each one · and handle the conversation with the person whose job you just changed.

Before you start: Chapter 54 (extraction, and validation in section 54.7), Chapter 57 (prompt versions, tolerant parsing, the meter, fallbacks), Chapter 55 (tools that act), Chapter 49 (SQLite from Python, section 49.2), Chapter 12 (constraints and transactions, section 12.13), Chapter 29 (idempotency, your own exceptions, tests), Chapter 22 (confidence intervals).

Time needed: 16–20 hours, spread over three weeks, in three sittings: sections 58.0–58.4 (setup, choosing, and building the pipeline that writes), about 7 hours; sections 58.5–58.8 (gates, exceptions, measurement, money, rollout), about 6 hours; sections 58.9–58.10, the project and the exercises take the rest.

Sections: 58.0 Setting up · 58.1 Three generations of automation · 58.2 What to automate, and what to leave alone · 58.3 The pipeline, assembled · 58.4 Writing to a system of record · 58.5 The human in the loop · 58.6 Exceptions, classified · 58.7 Measuring it honestly · 58.8 Rolling it out · 58.9 The people part · 58.10 When not to automate

In `companion/ch58/` — *notebook:* `ch58-notebook.ipynb` · *Python script:* `erp.py`, `intake.py`

Chapter 59. Industry Case Studies

You will learn to: recognize the shape every data project shares, whatever the industry · read a case study for its constraints rather than its technology · see where the real work sits in nine different problems, and why it is so rarely the model · name the mistake each one made, what it cost, and how it was fixed · and judge your own next project against patterns that have already been through production.

Before you start: nothing new. This chapter draws on everything from Part 2 onwards, and each case names the chapters that built its pieces.

Time needed: 6–8 hours over a week: about 3 hours to read the nine cases carefully, an afternoon (3–4 hours) for the project, which maps a project of your own onto the frame, and an hour or two for the exercises.

Sections: 59.1 How to read a case study · 59.2 Case 1 · Manufacturing quality: the defect camera · 59.3 Case 2 · Demand planning at a mid-sized FMCG distributor · 59.4 Case 3 · B2B lead scoring at a software company · 59.5 Case 4 · Retail pricing at a regional chain · 59.6 Case 5 · Fraud detection at a bank · 59.7 Case 6 · Logistics routing for a distribution fleet · 59.8 Case 7 · Customer support automation at a subscription business · 59.9 Case 8 · Predictive maintenance on plant machinery · 59.10 Case 9 · Order to cash, end to end · 59.11 What the nine have in common · 59.12 What changes by industry, and what doesn't

Part 7 — Architecture & Leadership

Book 3 · Implementation · 8 chapters · roughly 90 hours in total

Ch	Chapter	Time	What you practise with
60	Designing Whole Systems	12–15 hours	notebook, 8 notes
61	Distributed Systems & Trade-offs	11–13 hours	Python script, notebook, 2 notes
62	Data Architecture Patterns	10–12 hours	3 notes
63	Automation Architecture & Governance	13–15 hours	dataset (csv), 2 notes
64	Security, Privacy, Governance & Responsible AI	15–18 hours	Python script, SQL, dataset (csv), notebook, 4 notes
65	FinOps: The Economics of Data Platforms	9–11 hours	Python script, 2 dataset (csv), notebook
66	Data Strategy, Maturity & Building Data Teams	9–11 hours	Python script, 2 dataset (csv), notebook
67	The Architect as Leader	3–4 hours	<i>reading only</i>

Chapter 60. Designing Whole Systems

You will learn to: explain what "architecture" means once a decision affects more than one team's tools · draw and read a C4 diagram at the zoom level a conversation actually needs · turn a vague wish ("make it fast", "make it secure") into a number someone can test against, including a p95 and a count of allowed failures · write an architecture decision record (ADR) that a stranger could read in two years and understand why · design a whole system by composing the pieces earlier parts of this book built separately · evolve a design under real constraints — budget, headcount, a legacy system nobody is allowed to touch · recognize the failure patterns of over-engineering, resume-driven design, and the big-bang rewrite.

Before you start: this chapter assembles systems you have already built. What it uses from each: - Chapter 16 (Power BI dashboards) and Chapter 20 (the Daily Sales Flash: sent by 07:30 IST on working days, and only after its four checks pass, section 20.11). - Chapter 21, section 21.3 (percentiles, and why service levels use them). - Chapters 45 to 47: the warehouse's raw, staging and mart layers; the Dagster pipeline that runs at 06:30 every morning (`riverstone_pipeline.py`); freshness judged against the previous working day (section 47.5). - Chapter 32, section 32.13 (the semantic layer: one tested definition of each metric). - Chapter 49 (the sensor archive as a Delta table, and the correction that failed without one) and Chapter 51 (reverse ETL into the CRM, PATCH versus PUT, systems of record). - Chapter 52 (AWS, region ap-south-1, the pipeline deployed as a scheduled container task). - Chapters 55 to 58: the support assistant and its refusal floor, the defect model's FastAPI service and shadow mode, and the PO-intake pipeline in assisted mode.

Time needed: 12–15 hours, spread over two weeks.

Sections: 60.1 What "architecture" means, practically · 60.2 Reading and drawing C4 diagrams · 60.3 Non-functional requirements: numbers, not adjectives · 60.4 Trade-offs and the architecture decision record · 60.5 The worked example: designing the Riverstone Analytics & AI Platform · 60.6 Evolving a design under real constraints

In `companion/ch60/` — *notebook:* `ch60-notebook.ipynb`

Chapter 61. Distributed Systems & Trade-offs

You will learn to: explain why any real system is a distributed system, and why that means failure is guaranteed, not merely possible · state the CAP theorem precisely enough to apply it, and explain what most people get wrong about it · use PACELC to reason about the trade-off that exists even when nothing has failed · place a real system on the consistency spectrum from strong to eventual, and say why that placement is a deliberate choice, not a defect · name and use the standard reliability toolkit — partitioning and sharding, replication, load balancing, caching, message queues, scaling up and out, quorums — and know what each one costs · design for failure by default: idempotency, timeouts, circuit breakers, graceful degradation · run a failure analysis on a real system, container by container, and find its actual single point of failure.

Before you start: Chapter 60's container diagram for the Riverstone Analytics & AI Platform is used throughout this chapter as the system under analysis. Chapters 45, 46, 47, 49, 50, 51, 52, 57 and 58 (idempotent loads, orchestration, freshness, storage, streaming, activation, deployment, caching, and

the PO-intake pipeline) are each revisited briefly, so you don't need to remember their details — only that they exist. The one simulation uses Python's random module (Chapter 29), enumerate() and list comprehensions (Chapter 17).

Time needed: 11–13 hours, spread over a week.

Sections: 61.1 Everything at scale is a distributed system · 61.2 The CAP theorem · 61.3 PACELC: the trade-off that exists even when nothing is broken · 61.4 The consistency spectrum · 61.5 The reliability toolkit · 61.6 Designing for failure by default · 61.7 Failure analysis: the Riverstone platform, container by container

In `companion/ch61/` — *notebook:* `ch61-notebook.ipynb` · *Python script:* `eventual_consistency_sim.py`

Chapter 62. Data Architecture Patterns

You will learn to: compare Lambda and Kappa on one concrete scenario (the plant-sensor stream Chapter 50 built), explain the trade-off each makes, and tell stream processing apart from a model answering requests · recognize the medallion architecture (bronze, silver, gold) as a name for a layering this book has already used since Chapter 45 · distinguish the centralized, data mesh, and data fabric organizational patterns, and know what each demands of a company · run an honest data mesh maturity check on a real organization, including the uncomfortable answer of "not yet" · describe what makes something a genuine data product rather than just a table with a name · explain how data contracts and a semantic layer are what make decentralized ownership survivable · apply Conway's Law as a design tool, not just an observation · match an architecture pattern to an organization's actual size and maturity, not to whichever pattern is fashionable this year.

Before you start: Chapter 60's container diagram is referenced directly — you don't need to reread it, but recognizing "the warehouse," "the semantic layer," and "the four-person data platform team" will help. Chapter 50 (the plant-sensor stream and its daily correction job), Chapter 45 (raw/staging/modeled), Chapter 47 (data contracts), Chapter 32 (the semantic layer, section 32.13), and Chapter 23 (metric definitions such as "active customer", section 23.13) are each revisited in one line before being renamed with this chapter's vocabulary.

Time needed: 10–12 hours, spread over a week. Plan three sittings: sections 62.1–62.3 (about 1.5 hours); sections 62.4–62.8 (about 1.5 hours); then the exercises (about 3 hours) and the project (4–6 hours).

Sections: 62.1 Zooming out again · 62.2 Lambda and Kappa, on one real scenario · 62.3 Medallion: a name for a layering you've already built · 62.4 Centralized, data mesh, and data fabric · 62.5 Data contracts and the semantic layer: the glue that makes decentralization survivable · 62.6 Is Riverstone ready for a data mesh? A maturity check, scored honestly · 62.7 What makes something a genuine data product · 62.8 Conway's Law, used as a design tool

Chapter 63. Automation Architecture & Governance

You will learn to: design the flow from source to delivered result as one architecture instead of a pile of independent scripts · discover and prioritize automation opportunities by value, risk, and effort, with a real worked ROI comparison · choose the right tool for a given job from a full decision matrix —

macro, scheduled script, BI subscription, low-code flow, orchestrated pipeline, integration platform, RPA, or an AI agent · apply a reference architecture that every automation in this book turns out to be a specialization of · decide between scheduled and event-driven designs · build the shared services every automation needs rather than reinventing them each time · assign ownership, write runbooks, and support what you've automated · find and control shadow IT before it becomes an unowned, business-critical liability · apply the controls — approvals, segregation of duties, audit trails, change management — that keep automation trustworthy at scale · retire an automation safely.

Before you start: this chapter governs automations built earlier in this book: Chapter 19 (VBA and Apps Script macros), Chapter 20 (the Daily Sales Flash), Chapter 46 (Dagster orchestration, Part 5), Chapter 51 (the CRM reverse-ETL sync, Part 5), and Chapter 58 (the PO-intake pipeline, Part 6). You don't need to reread any of them — each is reintroduced with the one fact this chapter needs from it.

Time needed: 13–15 hours, spread over a week and a half.

Sections: 63.1 One architecture, not a pile of scripts · 63.2 Process discovery and prioritization · 63.3 Choosing the right tool for the job · 63.4 A reference architecture for automation · 63.5 Scheduled versus event-driven design · 63.6 Shared services · 63.7 Ownership, runbooks, and support · 63.8 Controlling sprawl and shadow IT · 63.9 Controls: approvals, segregation of duties, audit trails, change management · 63.10 Retiring automations safely

In companion/ch63/ — *dataset (csv):* automation-inventory.csv

Chapter 64. Security, Privacy, Governance & Responsible AI

You will learn to: treat security, privacy, and fairness as design inputs decided before building, not incidents responded to afterward · apply the core security patterns — encryption, least privilege, authentication versus authorization, secrets management — to a real platform · design an identity and access model that answers "who can see what" with a specific, checkable table, and turn it into real roles, grants, a masked view and row-level security in PostgreSQL · build privacy in from the start: minimization, purpose limitation, retention, pseudonymization and anonymization, and know when differential privacy or federated learning earn their complexity · navigate the current regulatory landscape (India's DPDP Act, GDPR, the EU AI Act) well enough to know what applies and when to call a lawyer · formalize data governance — catalogs, lineage, ownership — as an organizational practice · run a real fairness audit and correctly diagnose proxy discrimination, where a model never uses a sensitive attribute yet still produces an unfair outcome · govern a model's whole lifecycle, including its model card and the rules for generative AI.

Before you start: this chapter closes an open risk from Chapter 60's design document (no access-control model was defined) and extends Chapter 63's controls section into full treatment. It also leans on: Chapter 12 (SQL, and the one line on GRANT and REVOKE), Chapter 22 (Welch's t-test), Chapter 39, sections 39.9 and 39.10 (fairness checks and model cards), Chapter 51 (the lead-score sync), Chapter 52, sections 52.1 and 52.6 (IAM, least privilege, secrets), Chapter 56 (MLOps), Chapter 57 (LLMOps logs) and Chapter 58 (PO intake). Each is revisited with the one fact this chapter needs from it.

Time needed: 15–18 hours over two weeks: about 7 for the sections and their code, 4 for the exercises, and 4–7 for the project.

Sections: 64.0 Setting up · 64.1 Governance as a design input · 64.2 Security fundamentals · 64.3 Identity and access patterns — closing Chapter 60's open risk · 64.4 Privacy by design · 64.5 The regulatory landscape · 64.6 Data governance, formalized · 64.7 A fairness audit, worked · 64.8 Model governance · 64.9 Closing the loop

In `companion/ch64/` — *notebook:* `ch64-notebook.ipynb` · *SQL:* `access_lab_setup.sql` · *Python script:* `build_ch64_files.py` · *dataset (csv):* `leads_scored_2025.csv`

Chapter 65. FinOps: The Economics of Data Platforms

You will learn to: explain how cloud billing actually works — pay-as-you-go, no minimum commitment, and the handful of meters that drive almost every bill · identify the real cost drivers in a data platform: warehouse compute and storage, orchestration, and AI serving and API calls · compute unit economics — cost per report, per prediction, per AI request — for a real platform · use tagging and showback to make a shared bill honest about who's spending what · make architecture decisions with cost as an explicit input, not an afterthought discovered on next month's invoice.

Before you start: this chapter puts a real price on the platform Chapters 60 through 64 designed, distributed, patterned, governed, and secured. It reuses numbers from earlier chapters: Chapter 49's sensor-archive estimate, Chapter 52's cloud vocabulary and its cost section, Chapter 54's LLM price tiers, the LLM costs Chapters 55 and 57 measured, Chapter 58's PO-intake review time, and Chapter 63's value table. It also returns to one specific number: Chapter 60's non-functional requirement that platform cost stay under ₹0.50 per 1,000 order lines processed — a target set before anyone had actually built a cost model to check it against.

Time needed: 9–11 hours, spread over a week.

Sections: 65.1 How cloud billing actually works · 65.2 Cost drivers in a data platform · 65.3 Unit economics — and the NFR nobody had checked · 65.4 Budgets, tagging, and showback · 65.5 Cost-aware architecture decisions

In `companion/ch65/` — *notebook:* `ch65-notebook.ipynb` · *Python script:* `build_ch65_files.py` · *dataset (csv):* `monthly_cost_model.csv`, `unit_economics.csv`

Chapter 66. Data Strategy, Maturity & Building Data Teams

You will learn to: write a data strategy that fits on one page and actually gets used · score an organization's data maturity honestly, across the dimensions that matter, not just the ones that flatter it · build a real business case for data platform investment, including the uncomfortable parts a case usually leaves out · choose a team structure — centralized, embedded, or hybrid — that fits the organization's actual size, and hire against a named gap rather than a vague sense that "we need more people" · decide when to build a capability and when to buy it, with a real framework rather than a preference · lead change in an organization that doesn't yet trust data by default.

Before you start: this chapter draws together every earlier Part 7 chapter into an organizational and strategic view: the platform (Chapter 60), its trade-offs (61), its architecture pattern (62), its governance (63), its security (64), and — directly — its real cost (65).

Time needed: 9–11 hours, spread over a week: about 2 hours for sections 66.1–66.3 and their code, 1 hour for sections 66.4–66.6, 3 hours for the exercises, and 3–5 hours for the project.

Sections: 66.1 A data strategy on one page · 66.2 Maturity models, scored honestly · 66.3 Building the business case — including the part it usually leaves out · 66.4 Hiring and structuring data teams · 66.5 Build versus buy · 66.6 Change management and data culture

In `companion/ch66/` — *notebook:* `ch66-notebook.ipynb` · *Python script:* `build_ch66_files.py` · *dataset (csv):* `maturity_scorecard.csv`, `roi_case.csv`

Chapter 67. The Architect as Leader

You will learn to: recognize the shift from individual technical contribution to leverage — impact through decisions and people rather than your own hours — as the change that actually makes someone an architect · align technical work to business strategy and think in portfolios, not just projects · tell the executive story: presenting trade-offs and outcomes to people who don't want the technical detail · use architecture decision records as institutional memory, not paperwork · lead people you don't manage, including three real scenarios worked through in detail: influencing without authority, saying no, and presenting to a board · understand team topologies and Conway's Law as tools for how you organize, not just how you diagnose · recognize judgment as the summit skill this book cannot hand you, and humility as its permanent companion · walk into a new architecture role with a real first-90-days plan.

Before you start: Chapters 60–66 in particular, plus Chapter 24 (requirements, storytelling and stakeholders). This is the last teaching chapter of the book, and it assumes everything before it — not as facts to recall, but as the raw material this chapter finally asks you to lead with, rather than simply do. Part 8 (interview preparation) and the Closing chapter follow it.

Time needed: 3–4 hours: about two hours to read, and an hour or two with the exercises. The project is the work of months, and living this chapter will take years.

Sections: 67.1 From technical excellence to leverage · 67.2 Strategy, business alignment, and portfolio thinking · 67.3 Communication: the bridge · 67.4 Leading people · 67.5 Judgment: the summit skill · 67.6 A first-90-days plan for a new architect · 67.7 The arc of this book

Part 8 — Be Interview Ready

Book 4 · Be Interview Ready · 19 chapters · roughly 104 hours in total

Ch	Chapter	Time	What you practise with
68	How Data Hiring Works	5 hours	<i>reading only</i>
69	The Extra-Points Method	4–6 hours	<i>reading only</i>
69A	Why This, Not That: Tool Choice & Judgement	3–4 hours	<i>reading only</i>
70	Excel, Google Sheets, VBA & BI Question Bank	5–7 hours	Excel workbook, Python script, dataset (csv)
71	SQL Question Bank	6 hours	2 SQL
72	Python & pandas Question Bank	4–6 hours	notebook
72A	Data Structures & Algorithms Question Bank	8–10 hours	notebook
72B	Data Cleaning & Wrangling Question Bank	7–9 hours	<i>reading only</i>
73	Statistics, Probability & Experimentation Bank	9–12 hours	notebook
74	Machine Learning Question Bank	8–10 hours	notebook
75	Product Sense, Metrics, Case Studies & Guesstimates	5 hours	<i>reading only</i>
76A	Data Analyst & Data Scientist Question Bank	4–5 hours	<i>reading only</i>
76B	Business Analyst Question Bank	5–3 hours	<i>reading only</i>
77	Data Engineering & Data System Design Bank	5–6 hours	2 SQL, notebook
78	Automation & Integration Question Bank	5–4 hours	notebook
79	GenAI, LLM & MLOps Question Bank	5–6 hours	notebook
80	Architecture & Leadership Question Bank	3–4 hours	notebook
81	Behavioral, HR & Offer Conversations	2–3 hours	<i>reading only</i>
82	Take-Home Assignments & Mock Interviews	2–3 hours	Python script, 2 SQL, notebook

Chapter 68. How Data Hiring Works

You will learn to: see the full hiring process from the other side of the table, so nothing in it surprises you · write a CV that passes both the screening software and the human who reads it next · know what each interview round actually tests, for whichever of the ten roles you're aiming at · build a portfolio and a LinkedIn profile that do real work for you, not just sit there · use referrals properly, without asking for a favor you haven't earned · read pay figures across roles before the offer conversation · run a focused 30/60/90-day preparation plan instead of "studying everything."

Before you start: Chapter 7 (the ten roles and tracks); Chapter 8, especially §8.5 (decoding a job description) and §8.6 (reading salary figures); and Chapter 9, especially §9.1 (estimating your timeline from Time needed lines) and §9.5 (the seven-part portfolio piece). This chapter doesn't repeat them; it builds on them.

Time needed: 2.5–3.5 hours to read; allow 3–5 hours more for the project. After that it's a reference you return to at each stage of a real search.

Sections: 68.1 The whole process, gate by gate · 68.2 Cracking the resume screen: the software gate, then the human gate · 68.3 What each round actually tests · 68.4 LinkedIn hiring: how recruiter search actually works, and how to be found · 68.5 Referrals, used properly · 68.6 Pay: reading the numbers before the offer gate · 68.7 Preparing in 30, 60, or 90 days · 68.8 Worked examples, in three groups · 68.9 Four resumes, matched to their JDs

Chapter 69. The Extra-Points Method

You will learn to: understand the five dimensions that interview rubrics score, whatever the question · give a "strong" answer instead of a merely correct one, and know the difference · use twelve specific moves that turn a strong answer into an outstanding one, each shown before and after on a short example · carry a question through all three answer tiers yourself, live, under time pressure · recognize the red flags and over-corrections that cost points even when the technical content is right.

Before you start: the method needs nothing technical: this chapter is about how you answer, not what you know. The exercises draw on Chapters 4 and 12–51; each one names where its content is taught, so skip any you haven't studied. The method works alongside whichever question bank you use next (Chapters 70–82).

Time needed: about 1½ hours to read; 4–6 hours to drill all twenty exercises (or 1–2 hours for the five nearest your role); return to it before every interview.

Sections: 69.1 How interviewers actually score · 69.2 The shape of a strong answer · 69.3 The twelve extra-point moves · 69.4 One question, all three tiers · 69.5 The same moves, under different rounds · 69.6 What not to do

Chapter 69A. Why This, Not That: Tool Choice & Judgement

Before you start: nothing technical. This chapter is about choosing, not using, so it can be read before the tool chapters rather than after. Where a question names a technique, it says where the book teaches

it. Chapter 69 (the three answer tiers and the twelve extra-point tags) is the method every answer here is written against.

Time needed: 3–4 hours to read and answer each question aloud; 1 hour for a revision pass. Shorter than the tool banks, and the questions in it are asked more often than any of them.

Chapter 70. Excel, Google Sheets, VBA & BI Question Bank

You will learn to: answer the spreadsheet, automation, and BI questions that come up across screening calls, live exercises, and case interviews for analyst, BA, BI, and automation-track roles · check every formula answer on a small practice table you can type in two minutes · read a broken macro and fix it live · critique a dashboard the way a hiring manager would · walk out with a 20-question final-week revision list.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapters 10, 11, 15, 16 and 19, with a few links to Chapter 12 (joins, UNION ALL, GROUP BY). This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its Learn it in line sends you to the section that teaches it.

Time needed: 5–7 hours for a first pass (about 5 minutes per core question, 1–2 minutes per rapid-fire row), plus 1 hour for the final-week list. Section 70.9 adds about an hour and is best done with a spreadsheet open, since four of its questions ask you to confirm a behaviour in your own copy of Excel.

Sections: 70.0 The practice table, and how to use this bank · 70.1 Formulas, lookups, and logic · 70.2 Pivot tables and data analysis · 70.3 Power Query · 70.4 Google Sheets: QUERY, ARRAYFORMULA, and IMPORTRANGE · 70.5 VBA and Excel macros · 70.6 Google Apps Script · 70.7 DAX and Power BI modeling · 70.8 Dashboard critiques · 70.9 Predict the output: what the grid does behind the number

In [companion/ch70/](#) — *Python script:* [build_ch70_files.py](#) · *Excel workbook:* [ch70_practice.xlsx](#) · *dataset (csv):* [ch70_practice.csv](#)

Chapter 71. SQL Question Bank

You will learn to: answer the SQL questions that come up across screening calls, live-coding rounds, and take-home exercises for every data role · reason through the classic NULL and join traps that catch experienced candidates, not just beginners · write window-function solutions to the second-highest, top-N-per-group, running-total, streak and retention problems that recur across companies · know exactly where PostgreSQL and MySQL disagree · predict the output of a query cold, from the warm-up cases to the brain-racking ones, which is the round that cannot be talked around · walk out with a 24-question final-week revision list.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point tags). The questions test Chapters 12 and 13 (SQL), Chapter 14 (cleaning), Chapter 27 (NTILE) and Chapter 28 (recursive CTEs, window frames, query plans, indexes, materialized views), plus Chapter 49, section 49.4 (ACID). This chapter tests those skills; it doesn't teach them again.

Time needed: 4½–6 hours for a first pass (about 5 minutes per core question, 1 minute per rapid-fire row); 8–10 hours more to run every core question yourself in both databases; 1 hour for the final-week

list. Section 71.11 is worth its own sitting of 1½–2 hours, answering each question out loud before reading on.

Sections: 71.1 Core concepts: SELECT, WHERE, and JOIN · 71.2 Basic questions that are trickier than they look · 71.3 Aggregation: GROUP BY and HAVING · 71.4 Subqueries, CTEs, and EXISTS vs. IN · 71.5 Window functions: the classics · 71.6 Classic problems: duplicates, gaps-and-islands, retention · 71.7 Schema, constraints, and transactions · 71.8 PostgreSQL vs. MySQL: where the dialects actually differ · 71.9 Query optimization and indexes · 71.10 Live-coding walk-throughs · 71.11 Predict the output: from basic to brain-racking

In companion/ch71/ — *SQL:* ch71_queries_mysql.sql, ch71_queries_postgresql.sql

Chapter 72. Python & pandas Question Bank

You will learn to: answer the Python and pandas questions that come up across screening calls, live-coding rounds, and take-homes for every data role · recognize the classic Python gotchas (mutable defaults, is vs ==, late-binding closures) before they bite you live · write idiomatic, vectorized pandas instead of slow, easy-to-get-wrong loops · debug broken pandas code the way a live round actually tests you.

Before you start: Chapter 17 (Python from zero) and Chapter 18 (pandas), which teach almost everything here; Chapter 29 (classes, decorators, generators, keyword-only arguments) and Chapter 33 (generators and memory) for section 72.3; Chapter 69 for the answer tiers and the twelve extra-point tags. Chapter 71's SQL answers are the twins of this chapter's merge questions.

Time needed: about 4–6 hours to run every snippet yourself; 1 hour for a revision pass. Sections 72.8 to 72.10 add about 2 hours and are best done with a notebook open; they are the NumPy, pandas-depth and code-quality questions that come up once the interviewer has decided you know the basics.

Sections: 72.1 Python fundamentals: the gotchas that catch experienced candidates · 72.2 Predict the output: basic to advanced tricky questions · 72.3 Functions, generators, and decorators · 72.4 pandas fundamentals: Series, DataFrames, and selection · 72.5 pandas: grouping, joining, and reshaping · 72.6 pandas: performance, missing data, strings, and dates · 72.7 Debugging and live-coding walk-throughs · 72.8 NumPy, and why pandas behaves the way it does · 72.9 pandas at interview depth · 72.10 Writing Python someone else can run

In companion/ch72/ — *notebook:* ch72-notebook.ipynb

Chapter 72A. Data Structures & Algorithms Question Bank

Before you start: do Chapter 33 first (The Computer Science You Actually Need): it teaches almost every idea in this bank. You also need Chapter 17 (Python from zero), Chapter 29, section 29.5 (writing a class with __init__ and self), and Chapter 69 (the three answer tiers and the twelve extra-point tags). Chapter 72's Python gotchas (mutable defaults, is versus ==) come back here twice.

Time needed: 8–10 hours to run every snippet and answer each question aloud; 1 hour for a revision pass. Section 72A.10, the predict-the-output round, is worth its own sitting of about 2 hours, answering

each snippet out loud before reading on. Add 3–4 hours if the Chapter 33 sections named in the Learn-it-in lines are new to you.

In `companion/ch72a/` — *notebook*: `ch72a-notebook.ipynb`

Chapter 72B. Data Cleaning & Wrangling Question Bank

Before you start: do Chapter 14 first (Cleaning & Wrangling Real Data). This bank is the interview practice for that chapter's method, and it uses the same files. You also need Chapter 17 (Python from zero), Chapter 25 (pandas), and Chapter 69 (the three answer tiers and the twelve extra-point tags). Chapter 72's pandas gotchas come back here three times.

Time needed: 7–9 hours to work through with the files open, plus 1 hour for a revision pass. Sections 72B.4 and 72B.7 are the two that most often decide an interview; give each its own sitting.

Chapter 73. Statistics, Probability & Experimentation Bank

You will learn to: answer probability puzzles that show up across every data role's interviews, and explain why the surprising answer is correct, not just what it is · reason correctly about distributions, p-values, and confidence intervals, including the ways almost everyone misinterprets them at first · design an A/B test's sample size before running it, and debug one that's already gone wrong · spot Simpson's paradox and correlation-masquerading-as-causation in real-looking data.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapter 21 (probability and distributions), Chapter 22 (confidence intervals, tests, A/B basics, confounders, Simpson's paradox, and regression basics in section 22.10), Chapter 30 (inference, power, experiment design, the sample-ratio check) and Chapter 31 (causal inference without experiments). This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its Learn it in line sends you to the section that teaches it.

Time needed: about 9–12 hours for a first pass, running every cell and saying each answer aloud (the simulations take time to run and to understand); 1 hour for the final-week list. Section 73.8 adds about an hour and is best done in one sitting.

Sections: 73.0 The setup cell, and how to use this bank · 73.1 Core probability concepts · 73.2 Distributions · 73.3 Hypothesis testing fundamentals · 73.4 A/B test design · 73.5 A/B test debugging · 73.6 Causal inference and Simpson's paradox · 73.7 Live-coding and live-analysis walk-throughs · 73.8 Predict the number: the arithmetic that quietly goes wrong

In `companion/ch73/` — *notebook*: `ch73-notebook.ipynb`

Chapter 74. Machine Learning Question Bank

You will learn to: answer the ML questions that come up across screening calls, live-coding rounds, and case interviews for Data Scientist, ML Engineer, and analytics-adjacent roles · explain bias and variance, not just define them · spot data leakage before a model's score fools you · read a confusion

matrix, an ROC curve, and a calibration plot the way an interviewer actually wants · debug a model that "worked in training and broke in production."

Before you start: Part 4 (Chapters 35–43), which teaches every idea in this bank; Chapter 53, section 53.3 (early stopping) and Chapter 56 (monitoring, drift, training-serving skew) for section 74.6; Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its Learn it in line sends you to the section that teaches it.

Time needed: 8–10 hours for a first pass: about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, and about an hour to run the four code demos yourself. Section 74.8 adds about 1½ hours and is worth a sitting of its own, with the code running beside you. Plus 1 hour for the final-week list.

Sections: 74.1 Basic-but-tricky ML questions · 74.2 Bias, variance, and the shape of a good model · 74.3 Metrics, calibration, and thresholds · 74.4 Data leakage · 74.5 Feature engineering · 74.6 Model debugging · 74.7 ML case studies and live-coding walk-throughs · 74.8 Predict the number: what the library did that you did not ask for

In [companion/ch74/](#) — *notebook:* [ch74-notebook.ipynb](#)

Chapter 75. Product Sense, Metrics, Case Studies & Guesstimates

You will learn to: structure an ambiguous business case the way an interviewer actually wants, instead of jumping straight to an answer · diagnose a metric that moved, systematically, not by guessing at causes · design a KPI dashboard that answers real decisions, not just displays numbers · size a market or estimate a quantity with a defensible structure, not a guessed final number.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapters 3 (KPIs, dashboards), 4 (percentage points, estimation), 5 (precise questions, issue trees, MECE), 22 (A/B design, Simpson's paradox), 23 (KPI trees, diagnosing a change, guardrails) and 24 (turning an ask into a question), with a few links to Chapters 15, 16, 25 and 30. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its Learn it in line sends you to the section that teaches it.

Time needed: 3½–5 hours to read and drill every question once, out loud; 2–3 hours more for the project. Section 75.7 is arithmetic rather than structure, and rewards being done with a pen rather than read.

Sections: 75.1 The general case framework · 75.2 Diagnosing a metric that moved · 75.3 Designing metrics and dashboards · 75.4 Product sense and decision cases · 75.5 Guesstimates: structured estimation · 75.6 Full cases, talked through live · 75.7 Predict the number: metric arithmetic that is not what it looks like

Chapter 76A. Data Analyst & Data Scientist Question Bank

You will learn to: answer the role-specific questions a Data Analyst or Data Scientist interview actually asks, distinct from the skill-testing banks elsewhere in this part (SQL, Python, stats, ML) · walk

an interviewer through a real project end to end, at the right level of depth for each role · handle the "what's the difference between a DA and a DS" question, and the harder version, "which one are you actually best suited for" · survive a portfolio deep-dive without getting caught flat-footed on a detail you glossed over.

Before you start: Chapter 7, §7.2 (the tracks and the ten roles) · Chapter 8, §8.5 (decoding a job description) · Chapter 24, §24.4–24.8 (bottom line up front, the memo, handling pushback) · Chapter 25, §25.1 (the business analyst on a data team) · Chapters 36–39 (the lead-scoring project behind §76A.3) · Chapter 44 (the churn call list behind §76A.6) · Chapter 69 (the three answer tiers and the twelve extra-point moves).

Time needed: 4–5 hours to read and drill every question aloud; 3–4 hours for the project (two five-line summaries, one defended metric, and rehearsed walkthroughs). Sections 76A.7 to 76A.10 are the situational half and are best drilled out loud with someone else reading the situation to you.

Chapter 76B. Business Analyst Question Bank

You will learn to: answer the requirements-gathering, process-mapping, and documentation questions that come up in BA interviews · tell a BRD, FRD, and SRS apart without hesitating · write a user story with real acceptance criteria, not a vague wish · map a process the way a BA actually would, with decision points and swimlanes · handle "can you just change the number?" and other stakeholder pushback without folding or getting defensive.

Before you start: Chapter 24 (all of it: turning an ask into a question, business rules, stakeholders, and pushback) and Chapter 25 (all of it: the business analyst track), which between them teach every technique this bank tests; Chapter 26, section 26.10 (Agile, Scrum, and the backlog); Chapter 3, section 3.2 (order 5001's journey from enquiry to cash, which the worked examples use); and Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its Learn it in line sends you to the section that teaches it.

Time needed: 2.5–3 hours to read and drill once (about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row), plus 4–6 hours for the project: two interviews, a swimlane map, a story, and a requirement.

Chapter 77. Data Engineering & Data System Design Bank

You will learn to: answer the pipeline, data-modeling, and system-design questions that come up in Data Engineer interviews · design a batch or streaming pipeline live, under time pressure, the way an interviewer actually wants · reason correctly about idempotency, partitioning, and schema evolution, not just define the terms · walk through a full system design case end to end, stating trade-offs out loud.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Part 5 (Chapters 45–52: ingestion, pipelines, data quality, distributed compute, storage, streaming, activation and deployment) and Chapter 28 (query plans, grain, the star schema, slowly changing dimensions). They also link to Chapter 12, section 12.13 (the upsert), Chapter 33 (queues and graphs), Chapter 61 (partitioning and sharding), and the two banks this one leans on, Chapter 71 (SQL)

and Chapter 72A (complexity). This chapter tests those skills; it doesn't teach them again, with one exception: Q77-018 builds topological sort step by step, because no earlier chapter does.

Time needed: 5–6 hours for a first pass (about 10 minutes per core question, including running its code, and 1–2 minutes per rapid-fire row), plus 1 hour for the final-week list. Section 77.8 adds about 1½ hours and is best done with a notebook open.

Sections: 77.1 Pipeline fundamentals · 77.2 Basic-but-tricky data engineering questions · 77.3 Data modeling for pipelines and warehouses · 77.4 Orchestration and scheduling · 77.5 Scale, partitioning, and performance · 77.6 Data quality and monitoring · 77.7 Full system design walk-throughs · 77.8 Predict the output: what happens to rows on the way in

In companion/ch77/ — *notebook:* ch77-notebook.ipynb · *SQL:* ch77_queries_mysql.sql, ch77_queries_postgresql.sql

Chapter 78. Automation & Integration Question Bank

You will learn to: choose the right automation approach for a given problem, macro, script, low-code tool, or RPA, instead of defaulting to whichever one you know best · design report and alert automations that actually get read · reason correctly about APIs, webhooks, and reverse ETL · build retry and idempotency logic that survives a flaky upstream system · investigate an automation that silently stopped working, the way an interviewer actually wants.

Before you start: Chapter 69 (the three answer tiers and the twelve extra-point moves). The questions test Chapters 19–20 (spreadsheet and report automation), 45 and 51 (APIs, webhooks, reverse ETL), 58 and 63 (intelligent automation, automation architecture), with retries from Chapter 29, section 29.9 and the circuit breaker from Chapter 57, section 57.8. This chapter tests those skills; it doesn't teach them again.

Time needed: 3.5–4 hours for a first pass (about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, and 30 minutes to run the four code demos yourself); 30 minutes for the final-week list. Section 78.7 adds about an hour.

Sections: 78.1 Choosing the right automation approach · 78.2 Basic-but-tricky automation questions · 78.3 Report and alert automation · 78.4 APIs, webhooks, and reverse ETL · 78.5 Failure handling and monitoring for automations · 78.6 Full design case · 78.7 Predict the number: schedules, retries, and defaults

In companion/ch78/ — *notebook:* ch78-notebook.ipynb

Chapter 79. GenAI, LLM & MLOps Question Bank

You will learn to: answer the LLM fundamentals, RAG design, and evaluation questions that come up in GenAI-adjacent Data Scientist, ML Engineer, and AI Engineer interviews · reason correctly about embeddings, chunking, and retrieval, not just name the components · explain how a model actually survives production (registries, serving, monitoring, retraining) · design a RAG assistant and a model-serving platform end to end, stating trade-offs out loud.

Before you start: Chapters 54–57 (Generative AI and LLMs; RAG, agents and evaluation; MLOps; LLMOps), which teach every idea in this bank; Chapter 35, section 35.2 for the dot product and cosine similarity; Chapter 41, section 41.7 for word embeddings; Chapter 74 for the classical-ML side of monitoring; Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its Learn it in line sends you to the section that teaches it.

Time needed: 5–6 hours for a first pass: about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, 20 minutes for each design case, and about 45 minutes to run the four code demos yourself. Section 79.9 adds about 1¼ hours. Plus 45 minutes for the final-week list.

Sections: 79.1 LLM fundamentals · 79.2 Basic-but-tricky GenAI questions · 79.3 Retrieval-Augmented Generation (RAG) · 79.4 Agents and tool use · 79.5 Evaluating AI applications · 79.6 MLOps: making models survive production · 79.7 LLMOps · 79.8 Full design cases · 79.9 Predict the number: the arithmetic behind an AI system

In `companion/ch79/` — *notebook:* `ch79-notebook.ipynb`

Chapter 80. Architecture & Leadership Question Bank

You will learn to: reason about distributed-systems trade-offs the way a senior interview actually probes them · make and defend a build-vs-buy or architecture decision with real trade-offs stated, not a confident guess · answer governance, security, and cost questions at the level an architect owns them · handle leadership scenarios (influencing without authority, saying no, presenting to a board) live · walk through a full architecture design case end to end.

Before you start: Part 7 (Chapters 60–67), which teaches almost every idea in this bank; Chapter 20, section 20.14 and Chapter 23, section 23.9 for the time-saving and payback arithmetic in Q80-016; Chapter 24, sections 24.7 and 24.8 for presenting to executives and handling pushback; and Chapter 69 for the three answer tiers and the twelve extra-point tags. This chapter tests those skills; it doesn't teach them again. When you can't answer a question, its Learn it in line sends you to the section that teaches it.

Time needed: 3–4 hours to read and drill once (about 10 minutes per core question answered aloud, 1–2 minutes per rapid-fire row, and 15 minutes to run Q80-016's cells yourself), plus 2–3 hours for the project: an ADR, an ROI, a pushback, and a roadmap. Section 80.7 adds about an hour and is whiteboard arithmetic — do it with a pen, not by reading.

Sections: 80.1 Distributed systems trade-offs · 80.2 Basic-but-tricky architecture questions · 80.3 Data architecture decisions · 80.4 Security, governance, and cost · 80.5 Leadership scenarios · 80.6 Full architecture design cases · 80.7 Predict the number: capacity, availability, and the arithmetic of a design

In `companion/ch80/` — *notebook:* `ch80-notebook.ipynb`

Chapter 81. Behavioral, HR & Offer Conversations

You will learn to: answer any behavioral question using the STAR method without sounding scripted · build a small story bank from your own experience that covers most behavioral themes with five to

eight real stories, not forty different ones · ask questions that actually reveal something about the role, not filler · answer "what's your expected CTC?", read an offer's breakup, and negotiate without either accepting the first number or overplaying a weak hand.

Before you start: Chapter 8, §8.6 (reading salary figures; CTC and in-hand pay) · Chapter 68 (the hiring process gate by gate, and §68.6 on pay before the offer gate) · Chapter 69 (the three answer tiers and the twelve extra-point tags) · Chapter 76A (talking about your own projects). The worked examples reuse your portfolio projects from Chapters 27 and 44.

Time needed: 2–3 hours to read and say every core answer aloud once; 4–6 hours for the project (building your own story bank, three STAR outlines, and timed practice). Sections 81.8 to 81.10 add about 2 hours; 81.9 covers the Indian hiring conversation, which most interview material leaves out entirely.

Sections: 81.1 The STAR method · 81.2 Building a story bank · 81.3 Classic behavioral questions, worked in full · 81.4 Basic-but-tricky HR questions · 81.5 Questions to ask interviewers · 81.6 Handling offers · 81.7 Handling a hostile follow-up, live · 81.8 The questions you will actually be asked · 81.9 The Indian hiring conversation · 81.10 Hard moments in the room

Chapter 82. Take-Home Assignments & Mock Interviews

You will learn to: judge whether a take-home assignment is fair before you start it · complete a take-home the way a strong candidate actually would, not just technically correctly, but with the judgment and communication a reviewer is scoring · sit through a realistic mock interview and hear what a strong answer and a weaker one sound like at the same turn · read interviewer scoring notes and understand what actually moved the score.

Before you start: Chapter 69 (the five-dimension rubric and the twelve extra-point tags). The take-homes use SQL from Chapters 12 and 13, the lead model from Chapters 36, 37 and 39, and the pipeline ideas from Chapters 45–47. The mocks draw on the banks in Chapters 71, 74, 76A, 76B, 77 and 81. This chapter rehearses those skills; it doesn't teach them again, and each section says where each one is taught.

Time needed: 2–3 hours to read the chapter and run every query and cell; about 10–12 hours if you also do all three take-homes against the clock (2 + 3 + 2 hours), compare them with the scoring tables, and record one mock.

Sections: 82.0 Before you start: is this a fair take-home? · 82.1 Take-home assignment: Data Analyst · 82.2 Take-home assignment: Data Scientist · 82.3 Take-home assignment: Data Engineer · 82.4 Mock interview: Data Analyst (entry level) · 82.5 Mock interview: Data Scientist (mid level) · 82.6 Mock interview: Data Engineer (mid level)

In `companion/ch82/` — *notebook:* `ch82-notebook.ipynb` · *SQL:* `ch82_queries_mysql.sql`, `ch82_queries_postgresql.sql` · *Python script:* `lead_data.py`

Closing

Book 3 • Implementation • 1 chapters • roughly 4 hours in total

Ch	Chapter	Time	What you practise with
83	The Long Game	4–5 hours	<i>reading only</i>

Chapter 83. The Long Game

You will learn to: see the real arithmetic of this book, in hours and in years, and plan against it rather than against a feeling • choose a weekly pace that survives a bad month • tell the difference between reading that helps and reading that hides • build depth in one place and literacy everywhere else • pick your own stopping point on the career tree and stop there without apology • separate what will still be true in ten years from what will not • find the feedback that no book can supply • recognize the four plateaus that arrive after the first job, which look nothing like the first one • keep the only skill that never goes out of date.

Before you start: the book. Chapter 9 in particular, which this chapter is the far end of.

Time needed: about an hour to read, and 4–5 hours for the project, spread over one week. The rest of it takes years, which is the subject.

Sections: 83.1 The arithmetic, stated plainly • 83.2 Consistency beats intensity • 83.3 Study just enough to build, then build • 83.4 Go deep, then broad • 83.5 Choose your own summit • 83.6 The fundamentals are durable; the tools are not • 83.7 You cannot do this alone • 83.8 The plateaus that come later • 83.9 The meta-skill: learning how to learn • 83.10 What this book could not give you

The topic index

Every term the book defines, alphabetically, with the chapter that teaches it. Built from each chapter's own Key terms list. A term taught in more than one place lists them all, earliest first.

3,497 terms.

Term	Taught in
%*	Ch 18
&&	Ch 26
+=	Ch 17
10,000-hour rule	Ch 9
100% stacked bar	Ch 15
15 significant digits	Ch 70
1904 epoch	Ch 70
2⁵³ in JSON	Ch 78
2⁵³ precision limit	Ch 77
3-2-1 rule	Ch 2
429 Too Many Requests	Ch 78
68–95–99.7 rule	Ch 21
95th percentile	Ch 56
__debug__ and -O	Ch 72
__init__	Ch 18
__name__	Ch 29

A

Term	Taught in
A/B test	Ch 22, Ch 30
ABAC (attribute-based access control)	Ch 64
ABC analysis	Ch 13
absolute difference	Ch 30
absolute reference	Ch 10
absolute versus normalized scores	Ch 55
absolute vs. relative reference	Ch 70
absorbed	Ch 31
acceptance criteria	Ch 25
accepted_values	Ch 32
access control	Ch 25
access token	Ch 51
access-control matrix	Ch 64
accrual accounting	Ch 23
accumulating snapshot	Ch 28
accumulator	Ch 11
accuracy	Ch 1, Ch 35, Ch 39
ACID (atomicity, consistency, isolation, durability)	Ch 49
action	Ch 48
action (add, remove, metaData,	Ch 49

Term	Taught in
protocol, commitInfo)	
action title	Ch 15
activate	Ch 17
activation function	Ch 43, Ch 53
actor	Ch 58
Adam	Ch 35, Ch 43, Ch 53
adapter	Ch 32
Adaptive Card	Ch 20
adaptive query execution (AQE)	Ch 48
addition rule	Ch 21
additive model	Ch 40
adequacy decision	Ch 64
ADF statistic	Ch 40
adjacent knowledge	Ch 81
adjusted Rand index (ARI)	Ch 38
adjusted R²	Ch 74
administrator (admin) rights	Ch 16
admitting limits	Ch 69
adoption	Ch 59
ageing report	Ch 12
agent	Ch 55, Ch 79
agglomerative	Ch 38
aggregate feature	Ch 36
aggregate function	Ch 12
aggregate inside a window function - --	Ch 71
Agile	Ch 25, Ch 26

Term	Taught in
AI assistant (All terms are defined in the Glossary, Appendix A.) ---	Ch 6, Ch 26
AI engineer	Ch 7
AIC	Ch 40
airline model	Ch 40
alert	Ch 20, Ch 46
alert budget	Ch 59
alert fatigue	Ch 20, Ch 47, Ch 56, Ch 77, Ch 78
alert grouping	Ch 47
alerting	Ch 63
algorithm	Ch 33
alias	Ch 12, Ch 26, Ch 56
.all() / .any()	Ch 72
allocation rule	Ch 65
ALLSELECTED	Ch 16
alpha	Ch 30, Ch 40
alpha (α)	Ch 37
alt text	Ch 15, Ch 16, Ch 20
ALTER TABLE	Ch 12
alternating least squares (ALS)	Ch 42
alternative constructor	Ch 29
alternative flow	Ch 25
alternative hypothesis	Ch 22
ambiguous truth value	Ch 72
amortisation	Ch 23
amortised $O(1)$	Ch 72A
amortized complexity	Ch 72A

Term	Taught in
analysis plan	Ch 27
analytics engineer	Ch 7
analytics engineering	Ch 32
anchor	Ch 28
anchoring	Ch 5
anisotropy	Ch 79
annotation	Ch 15
anomaly	Ch 47
anomaly detection	Ch 38
anonymization	Ch 64
ANOVA	Ch 22, Ch 30
Anscombe's quartet	Ch 15, Ch 73
anti-join	Ch 12, Ch 14, Ch 47, Ch 71
AP system	Ch 61
Apache Hudi	Ch 49
Apache Iceberg	Ch 49
API	Ch 2, Ch 18
API integration	Ch 58
API key	Ch 2, Ch 54
API key vs. OAuth	Ch 78
API rate limit	Ch 78
app	Ch 16
append queries	Ch 11
applicant tracking system (ATS)	Ch 68
Application	Ch 19
applied steps	Ch 11
.apply	Ch 18
apply	Ch 52
.apply(axis=1)	Ch 72

Term	Taught in
approval threshold	Ch 58
approvals	Ch 63
approximate against exact match (range_lookup)	Ch 70
approximate match	Ch 11
approximate nearest neighbour (HNSW, IVF)	Ch 79
approximate-nearest-neighbour (ANN) index	Ch 55
Apps Script	Ch 19
Apps Script execution limit	Ch 70
Apriori	Ch 38
architecture	Ch 60
architecture decision record	Ch 8
Architecture Decision Record (ADR)	Ch 80
architecture decision record (ADR)	Ch 60
architecture decision record (institutional memory)	Ch 67
argparse	Ch 20
args / *kwargs	Ch 72
argument	Ch 10, Ch 12, Ch 17, Ch 19, Ch 26
argument (\$1, \$@)	Ch 34
ARIMA	Ch 40

Term	Taught in
ARN	Ch 52
array	Ch 18, Ch 19
ARRAYFORMULA	Ch 11, Ch 70
Arrow table	Ch 49
as-is	Ch 25
as-is vs. to-be	Ch 76B
ASCII	Ch 2
assert against raise	Ch 72
assertion	Ch 29
asset	Ch 46
asset check	Ch 46
assets	Ch 23
assign	Ch 18
assignment	Ch 17
assignment step	Ch 38
assisted mode	Ch 58
associative array	Ch 34
assumption	Ch 5
astimezone	Ch 20
at-least-once	Ch 50
at-least-once / at-most-once / exactly-once delivery	Ch 77
at-least-once webhook delivery	Ch 51
at-most-once	Ch 50
atomicity	Ch 61
attachment	Ch 20
attention	Ch 53
attribute	Ch 17, Ch 18, Ch 19, Ch 29
attribute/column	Ch 12

Term	Taught in
attribution (first-touch, last-touch, multi-touch)	Ch 23
audit (All terms are defined in the Glossary, Appendix A.) ---	Ch 11
audit log	Ch 58
audit schema	Ch 47
audit trail	Ch 51, Ch 57, Ch 63
audit trail (All terms are defined in the Glossary, Appendix A.) ---	Ch 25, Ch 64
auditability	Ch 69A
augmented Dickey–Fuller test	Ch 40
authentication	Ch 64
authentication token	Ch 45
authorization	Ch 64
auto-commit	Ch 12
auto_arima	Ch 40
autocorrelation	Ch 40
autocorrelation function (ACF)	Ch 40
autograd	Ch 43
automation analyst	Ch 7
automation architecture	Ch 63
automation inventory	Ch 63
automation ladder	Ch 20
AutoML (All terms are defined	Ch 37

Term	Taught in
in the Glossary, Appendix A.) ---	
autoregressive (AR)	Ch 40
AutoSave	Ch 10
AutoSum	Ch 10
availability	Ch 2, Ch 61
availability ("nines")	Ch 60
availability bias	Ch 5
availability in series	Ch 80
availability zone	Ch 52
average	Ch 4
average against worst case	Ch 72A
average customer lifetime (1/r)	Ch 75
average handle time	Ch 75
average of averages	Ch 4, Ch 21
average order value (AOV)	Ch 3
awk	Ch 34
axis	Ch 18

B

Term	Taught in
B-tree	Ch 28
backend	Ch 52
backfill	Ch 46
backfilling	Ch 77
background data	Ch 39
background job	Ch 34
background verification	Ch 68, Ch 81
backlog	Ch 26
backoff cap	Ch 78
backpressure	Ch 77
backpropagation	Ch 43, Ch 53
backtesting	Ch 40
backup	Ch 2
bag of words	Ch 41
bagging	Ch 37
balance	Ch 31
balance sheet	Ch 23
band (tier) table	Ch 11
bandwidth	Ch 31
banker's rounding	Ch 12
banker's rounding (round half to even)	Ch 17
banker's rounding against half-away-from-zero	Ch 70
bar chart	Ch 15
bare except	Ch 72
base case	Ch 33, Ch 72A
base class	Ch 18

Term	Taught in
base effect	Ch 4
base rate	Ch 21, Ch 35, Ch 39, Ch 73, Ch 74
base rate fallacy	Ch 21
base salary	Ch 8
base task	Ch 43
base value	Ch 39
base64	Ch 20
BaseException	Ch 72
baseline	Ch 25, Ch 36, Ch 47, Ch 59, Ch 69A
baseline (for a claimed metric)	Ch 76A
bash	Ch 34
basket (transaction)	Ch 38
.bat file	Ch 18
batch	Ch 50, Ch 53
batch API	Ch 54
batch circuit breaker	Ch 58
batch layer	Ch 62
batch normalization	Ch 53
batch serving	Ch 56
batch vs. streaming	Ch 77
Bayes' rule	Ch 21, Ch 37
Bayes' theorem	Ch 73
Bayesian A/B testing	Ch 30
Bayesian optimization	Ch 37
BCELoss / BCEWithLogitsLoss	Ch 43
before-and-after	Ch 31
behavioral theme	Ch 81

Term	Taught in
Benjamini-Hochberg	Ch 22, Ch 30
Bernoulli distribution	Ch 35
Bessel's correction	Ch 73
beta	Ch 30, Ch 40
bfloat16	Ch 53, Ch 79
BI (business intelligence)	Ch 7
BI analyst	Ch 8
BI developer	Ch 7
bias	Ch 37, Ch 40, Ch 43, Ch 53
bias-variance trade-off	Ch 74
big bang deployment	Ch 56
big-bang rewrite	Ch 60
Big-O	Ch 33
Big-O notation	Ch 72A
bigram	Ch 41
bill of materials (BOM)	Ch 28
billings	Ch 3
bin	Ch 15
binary	Ch 2
binary classification	Ch 36
binary search	Ch 33, Ch 72A
binary search tree (BST)	Ch 72A
Binomial distribution	Ch 73
binomial distribution	Ch 21, Ch 35
birthday problem	Ch 73

Term	Taught in
bisect	Ch 33, Ch 72A
bit	Ch 2, Ch 35
bit-packing	Ch 49
bitmap scan	Ch 28
blameless review	Ch 47
blended against paid CAC	Ch 75
blended CAC	Ch 23
blind spot sampling	Ch 59
Block Kit	Ch 20
blocking (deduplication)	Ch 77
blocking check	Ch 46
blue-green deployment	Ch 56
blue/green tables	Ch 47
BM25	Ch 55
body	Ch 2
Bonferroni correction	Ch 22, Ch 30, Ch 73
bookings	Ch 3
bookmark	Ch 16
Boolean	Ch 1
boolean mask	Ch 18
bootstrap	Ch 22
bootstrap sample	Ch 37
border point	Ch 38
bottom line up front (BLUF)	Ch 24
bound parameters	Ch 18
boundary burst	Ch 78
bounded staleness	Ch 61
box plot	Ch 15

Term	Taught in
Boyce–Codd normal form (BCNF)	Ch 28
BPMN	Ch 25, Ch 76B
branch	Ch 26
BRD	Ch 25, Ch 76B
breach	Ch 47
breach intimation	Ch 64
breadth-first search	Ch 33
breadth-first search (BFS)	Ch 72A
break	Ch 17
break-even	Ch 58
break-even per account	Ch 44
break-even threshold	Ch 39
break-glass access	Ch 64
breakpoint	Ch 19
bridge table	Ch 12
Brier score	Ch 39
broadcast join	Ch 48
broadcasting	Ch 38, Ch 72
broker	Ch 50
bronze / silver / gold	Ch 62
BST invariant	Ch 72A
bubble sort	Ch 72A
bucket	Ch 33, Ch 49
budget alert	Ch 65
build cache	Ch 52
build context	Ch 52
build versus buy	Ch 66

Term	Taught in
build vs. buy	Ch 80
build-against-upkeep	Ch 69A
bunching	Ch 31
business alignment	Ch 67
business analyst	Ch 7, Ch 25
business case	Ch 66
business impact	Ch 69
business intelligence	Ch 16
business judgment	Ch 69
business key	Ch 50, Ch 58
business limit	Ch 14
business metric	Ch 56
business requirement	Ch 25, Ch 76B
business rule	Ch 24, Ch 76B
business system	Ch 3
business-question-first narrative structure	Ch 76A
busy hour	Ch 80
buy-out	Ch 81
byte	Ch 2
byte-order mark (BOM)	Ch 45
byte-order mark (BOM) ---	Ch 77
byte-pair encoding	Ch 54
bytes	Ch 20
BytesIO	Ch 20
ByVal / ByRef	Ch 19

C

Term	Taught in
C (inverse regularization)	Ch 37
C4 diagram	Ch 80
C4 model	Ch 60
CAC	Ch 23
CAC payback period	Ch 75
cache	Ch 48
cache invalidation	Ch 57, Ch 61
caching	Ch 61
caching (LLM cost)	Ch 79
CAGR (compound annual growth rate)	Ch 4
CALCULATE	Ch 11, Ch 16, Ch 70
calculated column	Ch 11, Ch 16
calculated column vs. measure (DAX)	Ch 70
calculated field (pivot)	Ch 70
calculated table	Ch 16
CALCULATETABLE	Ch 16
Calculation	Ch 19
calculation mode	Ch 10
calibrated probabilities	Ch 37
CalibratedClassifierCV	Ch 39
calibration	Ch 39, Ch 74, Ch 76A
calibration (All terms are defined in the Glossary, Appendix A.) ---	Ch 36
calibration within groups	Ch 39
caliper	Ch 31
call stack	Ch 33
canary release	Ch 56
cancellation rate	Ch 3

Term	Taught in
candidate key	Ch 28
CAP theorem	Ch 61, Ch 80
capacity constraint	Ch 39, Ch 44
capital expenditure (capex)	Ch 23
capping	Ch 36
career switcher	Ch 8
career tree	Ch 8, Ch 83
CASE	Ch 12
case study	Ch 59
CASE without ELSE	Ch 71
cash conversion cycle	Ch 23
cash flow statement	Ch 23
cast	Ch 12
catalog	Ch 49
CatBoost	Ch 37
catch-all topic	Ch 41
catchup	Ch 46
categorical dtype	Ch 18
categorical palette	Ch 15
category	Ch 14
category dtype	Ch 72
causal claim	Ch 31
causation	Ch 5, Ch 22
CDC (Change Data Capture)	Ch 77
ceiling	Ch 57
cell	Ch 10, Ch 17
cell address	Ch 10
Cells	Ch 19
censoring	Ch 36
center of excellence	Ch 63

Term	Taught in
Central Limit Theorem	Ch 73
central limit theorem	Ch 21
centralized ownership	Ch 62
centralized team	Ch 7, Ch 66
centred moving average	Ch 40
centroid	Ch 38
CERT-In	Ch 64
CFO / CFI / CFF	Ch 23
chain of thought	Ch 54
chain rule	Ch 43, Ch 53
chain-linked (sequential) decomposition	Ch 23
chained assignment	Ch 18, Ch 72
ChainedAssignmentError	Ch 72
chance	Ch 5
change data capture (CDC)	Ch 45
change management	Ch 63, Ch 66
change notice	Ch 47
change request	Ch 76B
chaos test (All terms are defined in the Glossary, Appendix A.) ---	Ch 61
chargeback	Ch 65
CHECK	Ch 12
check	Ch 20
check (Soda) (All terms are defined in the Glossary, Appendix A.) ---	Ch 47
check cell	Ch 11
check strategy	Ch 32
checkpoint	Ch 50
checksum	Ch 28, Ch 34, Ch 45
cherry-picking	Ch 15

Term	Taught in
chi-square test	Ch 22
chi-square test of independence	Ch 30
chmod	Ch 34
choropleth map	Ch 15
ChrW	Ch 19
chunk	Ch 55
chunk overlap	Ch 79
chunking	Ch 18
chunking (fixed-size, sentence, section)	Ch 79
chunking strategy	Ch 55
chunksize (large-file reading)	Ch 72
churn rate	Ch 23
churn risk	Ch 51
CI/CD	Ch 52
CIA triad	Ch 2
CIDR notation	Ch 52
circuit breaker	Ch 57, Ch 61, Ch 78
circular reference	Ch 10
citation	Ch 55
citation rate	Ch 57
citation/grounding	Ch 79
claim	Ch 5
clarifying question	Ch 24, Ch 69
clasp (All terms are defined in the Glossary, Appendix A.) ---	Ch 19
class	Ch 18, Ch 29
class attribute	Ch 18, Ch 72
class imbalance	Ch 39, Ch 53
class method	Ch 29

Term	Taught in
class weight	Ch 37
class weights	Ch 39
classes_	Ch 36
classification	Ch 37
clause	Ch 12
clawback	Ch 81
cleaning log	Ch 14
client	Ch 2, Ch 54
clone	Ch 26
closing the loop	Ch 69
closure	Ch 72
cloud	Ch 2
cloud billing	Ch 65
cloud scheduler	Ch 20
cluster	Ch 52
cluster profiling	Ch 38
clustered standard errors	Ch 31
clustering	Ch 38
clutter	Ch 15
CNN	Ch 53
co-authoring	Ch 10
COALESCE	Ch 12
COALESCE / IFNULL	Ch 71
code diagram	Ch 60
code review	Ch 26, Ch 29
coefficient	Ch 37
coefficient of variation	Ch 21
coefficient sign flip	Ch 74
cognitive bias	Ch 5
Cohen's d	Ch 30
Cohen's h	Ch 30, Ch 73

Term	Taught in
cohort	Ch 13
cold-start problem	Ch 42
collaboration	Ch 69
collaborative filtering	Ch 42
collation	Ch 12
collation (All terms are defined in the Glossary, Appendix A.) ---	Ch 14
collect	Ch 48
collections	Ch 3
collectively exhaustive	Ch 5
collision	Ch 33
color vision deficiency	Ch 15
column / field / variable / attribute	Ch 1
column chart	Ch 15
column chunk	Ch 49
column pruning	Ch 48, Ch 77
column storage	Ch 69A
column-level lineage	Ch 47
columnar format	Ch 77
columnar storage	Ch 49, Ch 77
Columns area	Ch 11
ColumnTransformer	Ch 36
combine files	Ch 11
command	Ch 17
command substitution	Ch 34
comment	Ch 10, Ch 17
commit	Ch 26, Ch 49
COMMIT / ROLLBACK	Ch 12
commit hash	Ch 26
commit history as evidence	Ch 27

Term	Taught in
committed offset	Ch 50
common cause	Ch 5
common table expression (CTE)	Ch 13
compaction	Ch 50
compaction (OPTIMIZE)	Ch 49
comparison group	Ch 27
comparison operator	Ch 17
competence plateau	Ch 83
compiled SQL	Ch 32
complement rule	Ch 21
completeness	Ch 1, Ch 25
complexity	Ch 33
component	Ch 35
component diagram	Ch 60
composite index	Ch 28, Ch 71
composite key	Ch 12
composite model	Ch 16
composition	Ch 18
compound growth rate	Ch 4
compound score	Ch 41
compounding	Ch 4, Ch 75, Ch 83
compression	Ch 2
compression codec (Snappy, zstd, gzip)	Ch 49
compute	Ch 49
compute meter	Ch 65
concat	Ch 18, Ch 72
CONCAT_WS	Ch 71
concept drift	Ch 56, Ch 74
condition	Ch 17
conditional expression	Ch 17

Term	Taught in
conditional formatting	Ch 10
conditional independence	Ch 37
conditional probability	Ch 4, Ch 21, Ch 73
conditional saving	Ch 66
confidence	Ch 5, Ch 38
confidence interval	Ch 22, Ch 27, Ch 30, Ch 73
confidence level	Ch 22
confidence threshold	Ch 55
confidence weighting	Ch 42
confidentiality	Ch 2
config()	Ch 32
confirmation bias	Ch 5
confirmation loop	Ch 58
conflict rule	Ch 51
conformed dimension	Ch 28
confound (in a resume claim)	Ch 76A
confounded elasticity	Ch 59
confounder	Ch 22, Ch 27, Ch 73
confounding	Ch 31
confusion matrix	Ch 39, Ch 53, Ch 74
confusion matrix (multi-class)	Ch 41
confusion of the inverse	Ch 21
connect timeout against read timeout	Ch 78
Connected Sheets	Ch 11
connection string	Ch 45
connection URL	Ch 18
consensus	Ch 61
consent manager	Ch 64
consistency	Ch 1, Ch 61

Term	Taught in
consistency over intensity	Ch 83
consistency spectrum	Ch 61
constant folding	Ch 72A
constant time	Ch 33
constraint	Ch 12
consumer	Ch 50
consumer group	Ch 50
consumer lag	Ch 77
container	Ch 52
container diagram	Ch 60
containment	Ch 47
contamination	Ch 38
content backlog	Ch 55, Ch 59
content gap	Ch 57
content-based filtering	Ch 42
Content-ID (cid:)	Ch 20
context attachment	Ch 55
context diagram	Ch 60
context manager (with)	Ch 17
context transition	Ch 16
context window	Ch 41, Ch 53, Ch 54, Ch 79
continue	Ch 17
continuous	Ch 1
continuous deployment	Ch 52
continuous integration	Ch 26, Ch 32, Ch 52
contrast ratio	Ch 15
contribution margin	Ch 23, Ch 75
control	Ch 30
control group	Ch 22
convergence	Ch 35
conversion rate	Ch 13

Term	Taught in
convolution	Ch 43, Ch 53
convolutional neural network (CNN)	Ch 43
Conway's Law (All terms are defined in the Glossary, Appendix A.) ---	Ch 62
Conway's Law (Chapter 62)	Ch 66, Ch 67
Copilot (All terms are defined in the Glossary, Appendix A.) ---	Ch 16
.copy()	Ch 18
Copy-on-Write	Ch 72
copy-on-write	Ch 72
copy-paste integration	Ch 3, Ch 25
core point	Ch 38
core skill	Ch 8
corpus	Ch 55
corpus version key	Ch 57
correction rate	Ch 58
correctness	Ch 69
correlated subquery	Ch 12, Ch 71
correlation	Ch 5, Ch 15, Ch 22
correlation matrix	Ch 22
cosine similarity	Ch 35, Ch 54, Ch 55, Ch 79
cosine similarity (recommenders)	Ch 42
cosine similarity (text)	Ch 41
cost	Ch 28
cost of an error	Ch 58
cost of goods sold (COGS)	Ch 23
cost per request	Ch 57
cost-based threshold	Ch 76A

Term	Taught in
cost-based threshold (reused)	Ch 82
count(*) vs. count(col)	Ch 71
count(DISTINCT col)	Ch 71
Counter	Ch 17, Ch 72A
counter-offer	Ch 81
counter-offer (from your current employer)	Ch 81
counterfactual	Ch 31, Ch 76A
covariance	Ch 35
covariance matrix	Ch 35
covariate smoothness	Ch 31
coverage	Ch 29, Ch 73
covering index	Ch 28, Ch 71
CP system	Ch 61
CP vs. AP	Ch 80
cProfile	Ch 33
Cramér's V	Ch 30
CREATE DATABASE	Ch 12
CREATE TABLE	Ch 12
CreateObject	Ch 19
credential vault	Ch 63, Ch 64
credible applicant	Ch 8
credit hold	Ch 51
criterion	Ch 10
critical value	Ch 40
CRM	Ch 3
cron	Ch 20, Ch 34, Ch 46
cron expression	Ch 46
cron step syntax (* / n)	Ch 78
CRON_TZ	Ch 20
CronJob	Ch 52

Term	Taught in
crontab	Ch 20
crore	Ch 4
cross join	Ch 12, Ch 47
cross-encoder	Ch 55, Ch 79
cross-entropy	Ch 35, Ch 53
cross-field rule	Ch 14
cross-filter direction	Ch 16
cross-fitting	Ch 36
cross-validation	Ch 36
CrossEntropyLoss	Ch 43
CSV	Ch 2
CSV extract	Ch 20
CSV quoting and escaping	Ch 77
CSV UTF-8	Ch 10
csv.DictReader	Ch 17
csv.DictWriter	Ch 17
CTC (cost to company)	Ch 8
CTC breakup	Ch 81
CTE (WITH)	Ch 71
cte_max_recursion_depth	Ch 28
cumulative / running total	Ch 4
cumulative distribution	Ch 56
cumulative distribution function (cdf)	Ch 21, Ch 35
CUPED	Ch 73
curl	Ch 34
currency text	Ch 14
current assets / liabilities	Ch 23
current ratio	Ch 23
CURRENT ROW	Ch 13
CurrentRegion	Ch 19
curse of dimensionality	Ch 37, Ch 74, Ch 79

Term	Taught in
cursor	Ch 45
custom exception	Ch 29
custom format	Ch 10
customer health score	Ch 51
customer lifetime value (LTV)	Ch 23
cut	Ch 34
cut-off	Ch 3, Ch 31
cutting the tree	Ch 38
cycle	Ch 40, Ch 46
CYCLE clause	Ch 28
cycle detection	Ch 33

D

Term	Taught in
daemon	Ch 46
DAG (dependency graph)	Ch 32
DAG (Directed Acyclic Graph)	Ch 77
Daily Sales Flash	Ch 7
damped trend	Ch 40
dashboard	Ch 3, Ch 7, Ch 16
data	Ch 1
data activation	Ch 51
data analyst	Ch 7
Data Analyst vs. Data Scientist (role distinction)	Ch 76A
data architect	Ch 7
data architecture pattern	Ch 62
data as a product	Ch 62
data augmentation	Ch 53
data catalog	Ch 62, Ch 64
data center	Ch 2
data class	Ch 29
data class (@dataclass)	Ch 18
data cleaning	Ch 14
data contract	Ch 8, Ch 45, Ch 47, Ch 77
data contract (Chapter 47)	Ch 62
data culture	Ch 66
data dictionary	Ch 1
data drift	Ch 36, Ch 56, Ch 74
data engineer	Ch 7
data fabric	Ch 62

Term	Taught in
Data Fiduciary	Ch 64
data governance	Ch 80
data governance analyst	Ch 7
data hash	Ch 56
data incident	Ch 47
data lake	Ch 49
data leakage	Ch 36, Ch 59
data lineage	Ch 77
data mesh	Ch 62
data mesh maturity	Ch 62
data minimization	Ch 64
data model	Ch 11
data observability	Ch 77
data preparation	Ch 14
Data Principal	Ch 64
data processor	Ch 64
data product	Ch 25, Ch 62
Data Protection Board of India	Ch 64
data quality	Ch 1
data quality analyst	Ch 7
data scanned	Ch 49
data scientist	Ch 7
data steward	Ch 7
data stewardship	Ch 64
data strategy	Ch 66
data structure	Ch 33
data table	Ch 11
data test	Ch 47
data test (reused)	Ch 82
data transfer meter	Ch 65
data type	Ch 1, Ch 12

Term	Taught in
data type detection	Ch 11
data URI	Ch 20
data validation	Ch 10
data versioning	Ch 56
data visualization	Ch 15
data warehouse	Ch 3, Ch 11
data warehouse vs. data lake vs. lakehouse	Ch 77
data-ink ratio	Ch 15
data-quality report	Ch 14
database	Ch 2
database engine	Ch 69A
database server	Ch 2, Ch 69A
Databricks	Ch 48
DataFrame	Ch 18
Dataproc	Ch 48
dataset	Ch 1
date and time	Ch 1
date dimension (recap)	Ch 28
date grouping	Ch 11
date literal	Ch 12
date parts	Ch 36
date serial number	Ch 10, Ch 70
date spine / calendar table	Ch 13
date table	Ch 11, Ch 16
DATEDIFF	Ch 12
DATESINPERIOD	Ch 16
datetime	Ch 17
datetime index	Ch 18
datum	Ch 1
DAU/MAU stickiness	Ch 75

Term	Taught in
DAX	Ch 11, Ch 16
DAX Studio	Ch 16
DBeaver	Ch 12
DBMS	Ch 12
DBSCAN	Ch 38
dbt build	Ch 32
dbt Cloud	Ch 32
dbt Core	Ch 32
dbt run	Ch 32
dbt test	Ch 32, Ch 47
dbt_project.yml	Ch 32
dbt_scd_id	Ch 32
dbt_utils	Ch 32
dbt_valid_from	Ch 32
dbt_valid_to	Ch 32
DCL	Ch 12
DCL (Data Control Language)	Ch 64
DDL	Ch 12
ddof (degrees of freedom)	Ch 73
de-duplication key	Ch 50
dead-letter path	Ch 50
dead-letter queue	Ch 77
Debug.Print	Ch 19
Decimal	Ch 17
decision boundary	Ch 43
decision log	Ch 27
decision point (gateway)	Ch 76B
decision rights	Ch 3, Ch 5
decision rule	Ch 30
decision tree	Ch 37

Term	Taught in
decision-first framing	Ch 59
declared columns	Ch 45
decomposition	Ch 40, Ch 54
decomposition (count × size)	Ch 5
decorator	Ch 18, Ch 29, Ch 33, Ch 72
deduplication	Ch 13, Ch 36
deep copy	Ch 72
deep network	Ch 53
deepcopy	Ch 72A
DEFAULT	Ch 12
default argument	Ch 44
default evaluated at definition	Ch 72
default frame (RANGE vs. ROWS)	Ch 71
default NA markers	Ch 77
default value	Ch 17
defaultdict	Ch 17
defaultdict.__missing__	Ch 72A
Definition of Done	Ch 26, Ch 76B
definitional difference	Ch 76A
deflection rate	Ch 59
degraded mode	Ch 57
degrees of freedom	Ch 22
DELETE	Ch 12
delete anomaly	Ch 28
deliberate practice	Ch 9
delivery challan / delivery note	Ch 3
delivery log	Ch 46
delivery tool	Ch 20
Delta Lake	Ch 49

Term	Taught in
demographic parity	Ch 39
dendrogram	Ch 38
denominator	Ch 4
denominator of the search	Ch 27
denormalisation	Ch 69A
denormalization	Ch 28
DENSE_RANK	Ch 13
density	Ch 38, Ch 42
density curve	Ch 15
department	Ch 3
dependency	Ch 46
dependency group	Ch 29
Deployment	Ch 52
deployment pipeline	Ch 16
depreciation	Ch 23
depth & edge cases	Ch 69
depth and breadth	Ch 83
depth-first search	Ch 33
depth-first search (DFS)	Ch 72A
dequantization	Ch 53
deque	Ch 33, Ch 72A
derivative	Ch 35
derived table	Ch 12
describe	Ch 18
descriptive question	Ch 5
descriptive statistics	Ch 21
determinism	Ch 69A
dev dependency	Ch 29
deviation	Ch 21
diagnostic case	Ch 75
diagnostic question	Ch 5

Term	Taught in
dialect	Ch 69A
dict.get	Ch 72A
dictionary	Ch 17, Ch 33
dictionary comprehension	Ch 17
dictionary encoding	Ch 49
difference in approach	Ch 81
difference-in-differences	Ch 31
differencing	Ch 40
differential privacy	Ch 64
Digital Omnibus on AI	Ch 64
DIKW	Ch 1
Dim	Ch 19
dimension	Ch 28, Ch 35
dimension table	Ch 16
dimension table (recap)	Ch 28
dimensional modeling	Ch 28
dimensionality reduction	Ch 38
DIO	Ch 23
Dir	Ch 19
direct multi-horizon	Ch 40
directed acyclic graph (DAG)	Ch 46
DirectQuery	Ch 16
dirty read	Ch 71
disagree and commit	Ch 76A, Ch 81
discount band	Ch 27
discounted cumulative gain (DCG)	Ch 42
discoverability	Ch 62
discrete	Ch 1

Term	Taught in
disparate impact	Ch 64
disparate treatment	Ch 64
display rounding against ROUND	Ch 70
distillation	Ch 53, Ch 54
DISTINCT	Ch 12
distinct count	Ch 11
DISTINCTCOUNT	Ch 16
distributed computing	Ch 48
distributed system	Ch 61
distributed trace	Ch 57
distribution	Ch 15, Ch 21
diverging palette	Ch 15
DIVIDE	Ch 11, Ch 16
dividend	Ch 23
division by zero	Ch 71
DML	Ch 12
DNS	Ch 2, Ch 34
Do While	Ch 19
Docker	Ch 52
Docker Compose	Ch 52
Dockerfile	Ch 52
.dockerignore	Ch 52
docs site	Ch 32
docstring	Ch 17, Ch 29
document frequency	Ch 41, Ch 55
document metadata	Ch 55
document-topic distribution	Ch 41
domain ownership	Ch 62
donor pool	Ch 31
donut chart	Ch 15

Term	Taught in
door	Ch 8
dot plot	Ch 15
dot product	Ch 35
double unary (--)	Ch 11
downcasting	Ch 18
%~dpo	Ch 18
DPDP Act (Digital Personal Data Protection Act)	Ch 64
DPDP Rules, 2025	Ch 64
DPO	Ch 23, Ch 54
drift	Ch 28, Ch 40, Ch 52
drift (All terms are defined in the Glossary, Appendix A.) ---	Ch 39
drift detection	Ch 79
drill-down	Ch 16
drill-through	Ch 16
driver	Ch 12, Ch 48
DROP	Ch 12
dropna=False on groupby	Ch 72
dropout	Ch 53
dry run	Ch 34, Ch 49
DSO	Ch 23
.dt accessor	Ch 18, Ch 72
dtype	Ch 18
dtype downcasting	Ch 72
dtype= at read time	Ch 77
dual-axis chart	Ch 15
DuckDB	Ch 18, Ch 45, Ch 48
due date	Ch 3
dummy (0/1) variable	Ch 22

Term	Taught in
dummy variable	Ch 31
dummy variable trap	Ch 37
durability vs. availability	Ch 80
durable fundamentals	Ch 83
dynamic array	Ch 11
dynamic array (FILTER, UNIQUE)	Ch 70
dynamic programming	Ch 33
dynamic programming (top-down / bottom-up)	Ch 72A

E

Term	Taught in
e	Ch 35
e-commerce platform	Ch 3
early bad news	Ch 81
early stopping	Ch 37, Ch 53, Ch 74
EBIT	Ch 23
EBITDA	Ch 23
edge	Ch 33
edge case	Ch 69
EDI (Electronic Data Interchange)	Ch 51
editable install	Ch 29
effect size	Ch 22, Ch 27, Ch 30
effective step size	Ch 79
egress	Ch 65
egress (All terms are defined in the Glossary, Appendix A.) ---	Ch 49
eigenvalue	Ch 35
eigenvector	Ch 35
elastic net	Ch 37
elbow method	Ch 38
elicitation	Ch 25, Ch 76B
ELT	Ch 32
EmailMessage	Ch 20
embedded (decentralized) team	Ch 7
embedded team	Ch 66
embedding	Ch 54, Ch 55, Ch 79
empty-set aggregate	Ch 71
EMR	Ch 48

Term	Taught in
EnableEvents	Ch 19
encapsulation	Ch 18
encoding	Ch 2, Ch 15, Ch 17, Ch 49
encryption at rest	Ch 2, Ch 64
encryption in transit	Ch 2, Ch 64
End(xlUp)	Ch 19
endpoint	Ch 56
enterprise integration platform	Ch 51
entity-relationship (ER) diagram	Ch 12
entropy	Ch 35, Ch 37
entry point	Ch 18, Ch 29
enumerate	Ch 17
.env	Ch 26
.env file	Ch 18, Ch 20, Ch 29
.env.example	Ch 26
env_var	Ch 32
environment variable	Ch 17, Ch 18, Ch 20, Ch 26, Ch 29, Ch 34
ephemeral model	Ch 32
epoch	Ch 43, Ch 53
eps	Ch 38
equal opportunity	Ch 39
equity	Ch 23
equivalence test	Ch 73
erasure coding	Ch 80
ERP	Ch 3
Err	Ch 19
Err.Raise	Ch 19
error	Ch 10
error budget	Ch 80
error value	Ch 10

Term	Taught in
ERRORLEVEL	Ch 18
errors="coerce"	Ch 72
escalation	Ch 54
escalation queue	Ch 57
escaping	Ch 19
estimate	Ch 22
estimator (model)	Ch 36
ETL	Ch 32
ETL vs. ELT	Ch 77
EU AI Act	Ch 64
Euclidean distance	Ch 35
evaluation floor	Ch 57
evaluation set	Ch 54
event log	Ch 50
event object	Ch 19
event study	Ch 31
event time	Ch 50
event-driven automation	Ch 63
eventual consistency	Ch 61, Ch 80
exact duplicate	Ch 14
exact match	Ch 10, Ch 11
exact-match cache	Ch 57
exactly-once	Ch 50
Excel serial date	Ch 14
Excel workbook (.xlsx)	Ch 2
ExcelWriter	Ch 18
exception	Ch 17, Ch 29
exception path	Ch 25
exception queue	Ch 58
exception report	Ch 20

Term	Taught in
exception taxonomy	Ch 58
excess kurtosis	Ch 21
exchange	Ch 48
EXCLUDE	Ch 28
exclusion restriction	Ch 31
execution policy	Ch 17
executive presentation	Ch 24
executive storytelling	Ch 67
executive summary	Ch 44
executor	Ch 46
EXISTS vs. IN	Ch 71
exit code	Ch 17, Ch 18, Ch 20, Ch 26, Ch 29, Ch 34, Ch 52
expectation (Great Expectations)	Ch 47
expected counts	Ch 30
expected CTC	Ch 68, Ch 81
expected margin at risk	Ch 44
expected value	Ch 21, Ch 39, Ch 63, Ch 73
expense	Ch 23
experiment tracking	Ch 56
experiment versus inference	Ch 59
EXPLAIN	Ch 28
EXPLAIN / ANALYZE	Ch 71
EXPLAIN ANALYZE	Ch 28
explain()	Ch 48
explainable by design	Ch 64
explained variance	Ch 38
explanatory (independent) variable	Ch 22
explanatory chart	Ch 15

Term	Taught in
explicit feedback	Ch 42
exploding a BOM	Ch 28
exploratory chart	Ch 15
exponential	Ch 33
exponential backoff	Ch 29, Ch 45, Ch 57, Ch 78
exponential smoothing	Ch 40
exponentially weighted mean	Ch 40
export	Ch 26, Ch 34
ExportAsFixedFormat	Ch 19
expression	Ch 12, Ch 48
expression index	Ch 28
extension	Ch 2, Ch 17
external validation	Ch 38
extra-point moves	Ch 69
extrapolation	Ch 22

F

Term	Taught in
F-statistic	Ch 30
f-string	Ch 17
F1 score	Ch 39, Ch 74
Fabric capacity (F SKU)	Ch 16
fact	Ch 5, Ch 28
fact table	Ch 16
fact table (recap)	Ch 28
fact table grain	Ch 77
factorial	Ch 21
factorization	Ch 41
fail fast	Ch 46
failing rows	Ch 47
failover	Ch 80
failure alert	Ch 20
failure analysis	Ch 61
failure test	Ch 46
fair take-home	Ch 82
fairness	Ch 39
fairness audit	Ch 64
fake	Ch 29
fallback model	Ch 57
false discovery rate	Ch 22, Ch 30
false negative	Ch 39
false positive	Ch 39
false positive rate	Ch 39
false precision	Ch 4
false-positive rate	Ch 21, Ch 73
falsifiable	Ch 5

Term	Taught in
family-wise error rate	Ch 22, Ch 30
fan-out	Ch 12, Ch 14, Ch 18, Ch 80
fan-out (join cardinality)	Ch 71
fancy indexing	Ch 72
fast proxy metric	Ch 56
fast-forward	Ch 26
fast/slow pointers (Floyd's cycle detection)	Ch 72A
feature	Ch 35, Ch 36
feature engineering	Ch 36
feature hashing	Ch 74
feature importance	Ch 37, Ch 74
feature map	Ch 43, Ch 53
feature matrix (X)	Ch 35
feature scaling	Ch 53
feature selection	Ch 37
feature store	Ch 56, Ch 79
feature versioning	Ch 56
federated governance	Ch 62
federated learning	Ch 64
feedback	Ch 9
feedback loop	Ch 39, Ch 56, Ch 59, Ch 83
Fermi estimate / guesstimate	Ch 4
Fermi estimation	Ch 75
FETCH FIRST ... WITH TIES	Ch 71
few-shot	Ch 54
field separator	Ch 34
field shift	Ch 77

Term	Taught in
figure and axes	Ch 18
file	Ch 2
FileDialog	Ch 19
FileSystemObject	Ch 19
fill down	Ch 14
fill handle	Ch 10
fill rate	Ch 23
FILTER	Ch 11, Ch 13
filter	Ch 10
FILTER (WHERE ...)	Ch 71
filter context	Ch 11, Ch 16, Ch 70
filter view	Ch 10
Filters area	Ch 11
find	Ch 34
find and replace	Ch 10
FINDINGS file	Ch 27
fine-tuning	Ch 43, Ch 54
fine-tuning vs. prompting	Ch 79
FinOps	Ch 65
first in first out	Ch 33
first normal form (1NF)	Ch 28
first-90-days plan (architect)	Ch 80
first-party data	Ch 1
FIRST_VALUE	Ch 13
Fisher's exact test	Ch 30
fit	Ch 36
fit for use (All terms are defined in the Glossary, Appendix A.) ---	Ch 1

Term	Taught in
fit once, decide many times	Ch 44
.fit()	Ch 35
fitted value	Ch 22
five-dimension rubric	Ch 69
fixed cost	Ch 23
fixed effects	Ch 31
fixed vs variable pay	Ch 81
fixed window against token bucket	Ch 78
fixed-width integer overflow	Ch 72
fixture	Ch 29
Flash Fill	Ch 10
flattening	Ch 1
float32 / float64	Ch 43
floating alias	Ch 57
floating point	Ch 2
floating-point comparison	Ch 47
floating-point number	Ch 17
flowchart	Ch 25
fluency hours	Ch 83
fold	Ch 36, Ch 40
folder / directory	Ch 2
follow-up-due flag	Ch 51
footer (metadata)	Ch 49
for	Ch 17
For Each	Ch 19
for loop	Ch 34
forecast residual	Ch 40
foreign key	Ch 12, Ch 47

Term	Taught in
form control	Ch 19
Format Painter	Ch 10
format specification	Ch 17
formatted Excel	Ch 20
formatter	Ch 29
formatter (All terms are defined in the Glossary, Appendix A.) ---	Ch 17
formula	Ch 10
formula (statsmodels)	Ch 30
formula bar	Ch 10
formula rule	Ch 10
forward pass	Ch 43, Ch 53
forward-facing reason	Ch 81
FP-Growth	Ch 38
fraction vs percentage	Ch 14
frame	Ch 13
frame type	Ch 28
FRD	Ch 25, Ch 76B
FreeFile	Ch 19
frequency encoding	Ch 74
fresher	Ch 8
freshness	Ch 25, Ch 47, Ch 60
frozen layers	Ch 43
frozen=True	Ch 18
FrozenInstanceError	Ch 18
frozenset	Ch 38
frozenset ---	Ch 72A
full load	Ch 45
full outer join	Ch 12

Term	Taught in
full refresh	Ch 32
full-batch training	Ch 43
fully-loaded cost	Ch 65
Function	Ch 19
function	Ch 10, Ch 12, Ch 17, Ch 34
functional dependency	Ch 28
functional requirement	Ch 25, Ch 60, Ch 76B
functools.wraps	Ch 72
funnel	Ch 13, Ch 23
funnel multiplication	Ch 75
fuzzy duplicate	Ch 14
fuzzy merge	Ch 14

G

Term	Taught in
gamma	Ch 40
gap analysis	Ch 25, Ch 76B
gap explanation	Ch 81
gaps and islands	Ch 13
gaps-and-islands	Ch 71
garbled text (mojibake)	Ch 2
garden of forking paths	Ch 22, Ch 27
gate	Ch 58
GDPR	Ch 64
generate_series	Ch 13
generator	Ch 29, Ch 33, Ch 72
generator expression	Ch 33
generic test	Ch 32
geometric mean	Ch 75
get with default	Ch 17
GET, POST, PUT, PATCH	Ch 51
GETPIVOTDATA	Ch 11
getValues / setValues	Ch 19
getValues()/setValues() batching	Ch 70
Gherkin	Ch 25
gigabyte (GB)	Ch 2
Gini impurity	Ch 37
git add	Ch 26
Git Bash	Ch 26
git branch	Ch 26
git check-ignore	Ch 26
git commit	Ch 26
git commit --amend	Ch 26

Term	Taught in
git config	Ch 26
Git Credential Manager	Ch 26
git diff	Ch 26
git init	Ch 26
git log	Ch 26
git merge	Ch 26
git pull	Ch 26
git push	Ch 26
git restore	Ch 26
git restore --staged	Ch 26
git revert	Ch 26
git rm --cached	Ch 26
git status	Ch 26
git switch	Ch 26
GitHub Actions	Ch 52
.gitignore	Ch 26
Given/When/Then	Ch 25
Given/When/Then (Gherkin)	Ch 76B
glob	Ch 17
global importance	Ch 39
globbing	Ch 34
Gmail API	Ch 20
GmailApp	Ch 19
Go To Special	Ch 11
Goal Seek	Ch 11
golden set	Ch 54, Ch 55, Ch 57, Ch 79
Goodhart's law	Ch 23
goodness-of-fit test	Ch 30
Google Form	Ch 10
Google Visualization API Query Language	Ch 11

Term	Taught in
graceful degradation	Ch 61
gradient	Ch 35, Ch 53
gradient boosting	Ch 37
gradient descent	Ch 35, Ch 53
grain	Ch 1, Ch 10, Ch 12, Ch 14, Ch 16, Ch 25
grain (recap)	Ch 28
GRANT	Ch 64
grantee	Ch 64
graph	Ch 33
graphical perception	Ch 15
greedy against lazy quantifier	Ch 72
greedy algorithm	Ch 33
greedy decoding	Ch 54, Ch 79
greenfield	Ch 60
grep	Ch 34
grid search	Ch 37, Ch 39
gross margin	Ch 3, Ch 12
gross profit / margin	Ch 23
ground truth	Ch 59
ground-truth delay	Ch 56
grounding	Ch 54, Ch 55
group	Ch 34
GROUP BY	Ch 12
GROUPBY	Ch 11
groupby	Ch 18
groupby().agg()	Ch 72
grouped (clustered) bar	Ch 15
GroupKFold	Ch 74
GROUPS frame	Ch 28
guardrail	Ch 55, Ch 79

Term	Taught in
guardrail metric	Ch 23, Ch 30, Ch 75
gzip (All terms are defined in the Glossary, Appendix A.) ---	Ch 34

H

Term	Taught in
habit questions	Ch 5
half-life	Ch 40
half-open date range	Ch 18, Ch 77
half-open date window	Ch 23
half-open validity period	Ch 77
hallucination	Ch 54, Ch 79
handoff	Ch 25, Ch 76B
handover note	Ch 20, Ch 63
harmonic mean	Ch 39
hash	Ch 2
hash (row hash, file hash)	Ch 45
hash collision	Ch 74
hash function	Ch 33
hash join	Ch 28, Ch 33
hash map / set	Ch 72A
hash table	Ch 33
hashable	Ch 72A
HAVING	Ch 12
HCL	Ch 52
head	Ch 34, Ch 43, Ch 72A
header	Ch 2
headers	Ch 10
health check	Ch 52, Ch 56
heap	Ch 28, Ch 33, Ch 72A
heapq	Ch 33, Ch 72A
heartbeat (dead-man's switch)	Ch 77

Term	Taught in
heatmap	Ch 15
heavy-tailed distribution	Ch 73
hedged request	Ch 80
hexdigest	Ch 45
hidden file	Ch 26
hidden layer	Ch 43, Ch 53
hidden partitioning	Ch 49
hidden skill	Ch 8
hidden state	Ch 69A
hierarchical clustering	Ch 38
hierarchy	Ch 28
high cardinality	Ch 36
high-risk AI system	Ch 64
highlight color	Ch 15
hiring-manager round	Ch 68
histogram	Ch 15
hit rate	Ch 57
hit rate@k	Ch 42
Hive-style partitioning	Ch 48
HLOOKUP	Ch 11
HMAC (event signature)	Ch 51
holdout	Ch 76A
holdout group	Ch 73
Holt's method	Ch 40
Holt–Winters	Ch 40
home folder	Ch 26

Term	Taught in
honest limitation	Ch 76A
honest timeline	Ch 9
horizon	Ch 40
horizontal vs. vertical scaling	Ch 80
host and port	Ch 20
host key fingerprint	Ch 34
hours saved (All terms are defined in the Glossary, Appendix A.) ---	Ch 20
HRMS	Ch 3
HTML	Ch 19
HTML email	Ch 20
HTMLBody	Ch 19
HTTP	Ch 34
HTTP / HTTPS	Ch 2
HTTP 422	Ch 56
HTTP 429	Ch 29
hub-and-spoke integration	Ch 51
human in the loop	Ch 58
human review	Ch 55
human review loop	Ch 57
humility	Ch 67
hybrid (hub and spoke) team	Ch 66
hybrid recommender	Ch 42
hybrid search	Ch 55
hyperparameter	Ch 35, Ch 37, Ch 39

Term	Taught in
hypothesis	Ch 5
hypothesis test	Ch 30

I

Term	Taught in
IaaS	Ch 52
IAM (identity and access management)	Ch 52
ICE plot	Ch 39
id_str	Ch 78
ideal DCG	Ch 42
idempotency	Ch 20, Ch 57, Ch 58, Ch 61, Ch 77
idempotency key	Ch 29, Ch 46, Ch 50, Ch 51, Ch 58, Ch 60, Ch 78
idempotent	Ch 28, Ch 29, Ch 45, Ch 46
idempotent load (reused)	Ch 82
identifier	Ch 1
identity	Ch 52
IDENTITY / AUTO_INCREMENT	Ch 71
identity against equality (is / ==)	Ch 72A
identity column / AUTO_INCREMENT	Ch 12
IEEE 754	Ch 72A
if __name__ == "__main__"	Ch 17
if/elif/else	Ch 17
IFNA against IFERROR	Ch 70
.iloc	Ch 18
image	Ch 52
Immediate window	Ch 19
immutability	Ch 33, Ch 72A
immutable	Ch 29
impact analysis	Ch 47
implicit commit	Ch 12

Term	Taught in
implicit feedback	Ch 42
import	Ch 17
Import mode	Ch 16
Import mode vs. DirectQuery	Ch 70
IMPORTRANGE	Ch 11, Ch 70
impossible date	Ch 14
imputation	Ch 14, Ch 36
in-degree	Ch 77
in-hand pay	Ch 8
in-hand pay (Chapter 8)	Ch 81
in-order/pre-order/post-order traversal	Ch 72A
incident	Ch 56
income statement	Ch 23
incoming webhook	Ch 20
inconsistent formula	Ch 11
incremental load	Ch 45
incremental model	Ch 32
incremental refresh	Ch 16
incremental strategy	Ch 32
independence	Ch 4, Ch 21, Ch 31
independent failure	Ch 80
independent vs. mutually exclusive events	Ch 73
INDEX	Ch 11
index	Ch 17, Ch 18, Ch 28, Ch 55
index (in-memory)	Ch 33
index scan	Ch 28

Term	Taught in
index-only scan	Ch 28
INDEX/MATCH	Ch 70
indicator	Ch 18
indicator=	Ch 72
indicator=True	Ch 72
indispensable plateau	Ch 83
individual contributor (IC)	Ch 80
individual-contributor track ---	Ch 76A
inertia	Ch 38
inference	Ch 30
influence without authority	Ch 67
influencing without authority	Ch 80
info	Ch 18
information	Ch 1
information gain	Ch 35
informative missingness	Ch 36
Infrastructure as Code (IaC)	Ch 52
ingestion	Ch 45
inheritance	Ch 18, Ch 29
initialization (He, Xavier)	Ch 53
injection monitoring	Ch 57
inline styles	Ch 20
inner join	Ch 12
input contract	Ch 56
input monitoring	Ch 56
input validation	Ch 56

Term	Taught in
InputBox	Ch 19
Inquire	Ch 11
INSERT	Ch 12
insert anomaly	Ch 28
insertion order	Ch 72A
insight	Ch 1
installable trigger	Ch 70
installer	Ch 12
instance	Ch 18, Ch 29
instrumental variable	Ch 31
intake route	Ch 76A
integer division	Ch 71
integration	Ch 3
integration engineer	Ch 7
integration test	Ch 29
integrity	Ch 2
intelligent automation	Ch 58
interaction	Ch 36
interaction feature	Ch 74
interaction matrix	Ch 42
interaction term	Ch 31
intercept	Ch 22, Ch 37
interference between units	Ch 31
intermediate layer	Ch 32
internet	Ch 2
interpolation method	Ch 21
interquartile range	Ch 21
interquartile range (IQR)	Ch 15

Term	Taught in
INTERSECT / EXCEPT	Ch 12
INTERVAL	Ch 13
interval	Ch 1
interviewer scoring notes	Ch 82
inventory turns	Ch 23
inverse document frequency	Ch 55
inverse document frequency (IDF)	Ch 41
inverse regularisation strength (C)	Ch 74
INVEST	Ch 26, Ch 76B
invoice	Ch 3
IP address	Ch 2, Ch 34
iPaaS (integration platform as a service)	Ch 51, Ch 63
IQR rule	Ch 21
IS DISTINCT FROM	Ch 71
is vs. ==	Ch 72
is_incremental()	Ch 32
.isna() / .fillna() / .dropna()	Ch 72
ISO 8601 date	Ch 1
isolation forest	Ch 38
isolation level	Ch 71
isotonic regression	Ch 39
issue key	Ch 26
issue tree	Ch 5
IST	Ch 14
item set	Ch 38

Term	Taught in
item-based collaborative filtering	Ch 42
iterator (AVERAGEX)	Ch 16

J

Term	Taught in
Jaccard similarity	Ch 14
Java runtime	Ch 48
JAVA_HOME	Ch 48
Jinja	Ch 32
jitter	Ch 15, Ch 29, Ch 78
jitter (full, decorrelated)	Ch 78
Job	Ch 52
job	Ch 46, Ch 52
job description (JD)	Ch 8
job-ready (All terms are defined in the Glossary, Appendix A.) ---	Ch 27
job-ready path	Ch 6
join key	Ch 14
join kind	Ch 11
join type	Ch 18
JSON	Ch 1, Ch 2, Ch 17
JSON Lines	Ch 51
JSON mode	Ch 54
json_normalize	Ch 18
judgment	Ch 9, Ch 67
judgment (All terms are defined in the Glossary, Appendix A.) ---	Ch 83
junk dimension	Ch 28, Ch 77
Jupyter	Ch 17

K

Term	Taught in
k-distance plot	Ch 38
k-fold	Ch 36
k-means	Ch 38
k-means++	Ch 38
k-nearest neighbors	Ch 37
Kafka	Ch 50
Kahn's algorithm	Ch 77
Kanban	Ch 26
Kappa architecture	Ch 62
kappa architecture	Ch 80
keep_default_na	Ch 77
KEEPFILTERS	Ch 16
kernel	Ch 17, Ch 37
kernel (filter)	Ch 43, Ch 53
key	Ch 8, Ch 50, Ch 53
key (object storage)	Ch 49
key and value	Ch 17
key pair	Ch 34
key-management service (KMS)	Ch 64
keyword argument	Ch 17
keyword flag	Ch 36
keyword match	Ch 68
keyword-only argument	Ch 29

Term	Taught in
keyword-only arguments (*)	Ch 72
kilobyte (KB)	Ch 2
knowledge	Ch 1, Ch 9
knowledge cutoff	Ch 54
known_hosts	Ch 34
Kolmogorov-Smirnov statistic	Ch 56
KPI	Ch 3
KPI definition	Ch 3
KPI tile	Ch 20
KPI tree	Ch 23
KPI tree (All terms are defined in the Glossary, Appendix A.) ---	Ch 3
KS statistic (D)	Ch 79
Kubernetes	Ch 52
kurtosis	Ch 21
KV cache	Ch 79

L

Term	Taught in
label and selector	Ch 52
label delay	Ch 59, Ch 74
label scarcity	Ch 59
labelled outcome	Ch 69A
ladder plateau	Ch 83
LAG	Ch 13
lag	Ch 40, Ch 50
LAG / LEAD	Ch 71
lag feature	Ch 40
lagging indicator	Ch 3, Ch 75
lakehouse	Ch 49
lakh	Ch 4
LAMBDA	Ch 11
lambda	Ch 18
Lambda architecture	Ch 62
lambda architecture	Ch 80
Laplace smoothing	Ch 41
large language model	Ch 54
lasso (L1)	Ch 37
last in first out	Ch 33
Last Run Result	Ch 18
last-updated timestamp	Ch 45
last-write-wins	Ch 61
LAST_VALUE	Ch 13
late binding	Ch 19, Ch 72
late-arriving data	Ch 46
latency	Ch 50

Term	Taught in
latency budget	Ch 55, Ch 57
latency/consistency trade-off	Ch 61
lateness	Ch 50
Latent Dirichlet Allocation (LDA)	Ch 41
latent factors	Ch 42
latent semantic analysis	Ch 54
lawful basis	Ch 64
layer	Ch 43, Ch 52, Ch 53
lazy evaluation	Ch 33
lazy execution	Ch 48
lazy frame	Ch 48
LEAD	Ch 13
lead	Ch 3
lead score	Ch 51
lead to cash	Ch 3
leader	Ch 61
leader election	Ch 61
leader-follower (primary-replica) replication	Ch 80
leading indicator	Ch 3, Ch 75
leading underscore	Ch 18
leading zeros	Ch 10
leaf	Ch 37
leaf page	Ch 28
leakage	Ch 8
learned attribute (trailing _)	Ch 36
learned representation	Ch 43
learning curve	Ch 37, Ch 74

Term	Taught in
learning how to learn	Ch 83
learning plateau (Chapter 9)	Ch 83
learning rate	Ch 35, Ch 37, Ch 53
learning-rate schedule	Ch 53
least privilege	Ch 2, Ch 52, Ch 76A
least privilege (Chapter 52)	Ch 64
least squares	Ch 22, Ch 37
leave-one-out evaluation	Ch 42
left join	Ch 12
LEFT JOIN / INNER JOIN	Ch 71
legitimate uses	Ch 64
lemma	Ch 41
lemmatization	Ch 41
length (norm)	Ch 35
length normalization	Ch 55
less	Ch 34
LET	Ch 11
level-order traversal	Ch 72A
levels of measurement	Ch 1
leverage	Ch 23, Ch 67, Ch 73, Ch 83
liabilities	Ch 23
liabilities-to-equity	Ch 23
library	Ch 69A
lifecycle policy	Ch 65
lift	Ch 38
lift over baseline	Ch 76A

Term	Taught in
LightGBM	Ch 37
Like	Ch 19
likelihood	Ch 21, Ch 35, Ch 37
line chart	Ch 15
lineage	Ch 47, Ch 64
lineage graph	Ch 32
linear probability model	Ch 31
linear regression	Ch 37
linear regression for inference	Ch 30
linear search	Ch 33
linear time	Ch 33
linearithmic	Ch 33
linkage (Ward, complete, average, single)	Ch 38
linked list	Ch 72A
linter	Ch 17, Ch 29
linting	Ch 32
liquid clustering	Ch 49
list	Ch 17, Ch 33
list comprehension	Ch 17, Ch 72
Little's Law	Ch 80
live connection	Ch 16
liveness	Ch 56
liveness probe	Ch 52
LLM (large language model)	Ch 7
LLM-as-judge	Ch 55, Ch 79
LLMOps	Ch 57
load balancer	Ch 52
load balancing	Ch 61

Term	Taught in
load log	Ch 45
load shedding	Ch 80
load status	Ch 47
load to	Ch 11
load_dotenv	Ch 20
loadings	Ch 35, Ch 38
.loc	Ch 18
.loc vs. .iloc	Ch 72
local effect	Ch 31
local explanation	Ch 39
local mode	Ch 48
local versus global structure	Ch 38
locale	Ch 10, Ch 11, Ch 14
locale date parsing ---	Ch 70
localhost	Ch 12, Ch 34
lock file	Ch 19
lock-in	Ch 69A
lockfile	Ch 29
log base 2 (log₂)	Ch 35
log difference	Ch 31
log file	Ch 20
log file handler	Ch 18
log handler	Ch 20
log level	Ch 20, Ch 29
log loss (binary cross-entropy)	Ch 35
log scale	Ch 15
log transform	Ch 36, Ch 40
log-likelihood	Ch 35
log-log model	Ch 37

Term	Taught in
log-odds	Ch 30, Ch 37
logarithm	Ch 31, Ch 35
logarithmic time	Ch 33
logger	Ch 20, Ch 29
logging	Ch 18, Ch 20, Ch 29
logical (execution) date	Ch 77, Ch 78
logical date (Airflow ds)	Ch 46
logical execution order	Ch 12
logical value	Ch 10
logistic function	Ch 31
logistic regression	Ch 30, Ch 36, Ch 37
logit	Ch 43
logits	Ch 54
lollipop chart	Ch 15
long data	Ch 11
Long versus Integer	Ch 19
look-back window	Ch 45, Ch 46
lookahead	Ch 55
lookback window	Ch 32
lookbehind	Ch 55
lookup	Ch 10
lookup array	Ch 11
loop	Ch 17
LoRA	Ch 54
LoRA / QLoRA	Ch 79
loss	Ch 31
loss function	Ch 35, Ch 53
lossless	Ch 2
lossy	Ch 2

Term	Taught in
low-code	Ch 20
lru_cache	Ch 33, Ch 72A
LSTM	Ch 53
LTS release	Ch 12
LTV:CAC	Ch 23, Ch 75

M

Term	Taught in
M language	Ch 11
macro	Ch 19, Ch 32
macro and weighted F1	Ch 74
macro average	Ch 41
macro F1	Ch 39, Ch 41
macro recorder	Ch 19
macro security	Ch 19
macro vs. script vs. low-code vs. RPA	Ch 78
made to stock	Ch 3
MAE	Ch 39
MailApp	Ch 19
main flow	Ch 25
main() returning an exit code	Ch 18
Make	Ch 20
malformed input	Ch 58
managed connector	Ch 45
manifest	Ch 49, Ch 52
manipulation check	Ch 31
Mann-Whitney U	Ch 22, Ch 30
manual matching	Ch 25
many-to-many	Ch 12
.map	Ch 18
MAPE	Ch 39, Ch 40
mapping table (crosswalk)	Ch 14
margin	Ch 37
margin of error	Ch 22
marginal cost	Ch 65
mark as date table	Ch 16

Term	Taught in
Mark of the Web	Ch 19
Markdown	Ch 18, Ch 26
market basket analysis	Ch 38
mart	Ch 32
MASE	Ch 40
masked view	Ch 64
masking	Ch 53
MATCH	Ch 11
match key	Ch 14
match mode	Ch 11
match rate	Ch 14
matching	Ch 31
matching without replacement	Ch 31
materiality	Ch 20
materialization	Ch 32
materialize	Ch 46
materialized view	Ch 28, Ch 77
matplotlib	Ch 18
matrix	Ch 35
matrix factorization	Ch 42
matrix times matrix	Ch 35
matrix-times-vector (@)	Ch 35
maturity model	Ch 66
maturity score	Ch 62, Ch 66
maturity stage	Ch 66
max_depth	Ch 37
max_iter	Ch 36
maximum likelihood estimate (MLE)	Ch 35
MD5	Ch 45
mean	Ch 1, Ch 4, Ch 21

Term	Taught in
mean absolute error (MAE)	Ch 37
mean average precision (MAP)	Ch 42
mean decrease in impurity	Ch 74
mean of ratios against ratio of means	Ch 73
mean reciprocal rank	Ch 55
mean reciprocal rank (MRR)	Ch 42
mean squared error (MSE)	Ch 35
mean-centering	Ch 35
measure	Ch 11, Ch 16, Ch 28
measured floor	Ch 66
MECE	Ch 5
MECE (Mutually Exclusive, Collectively Exhaustive)	Ch 75
medallion architecture	Ch 62
median	Ch 1, Ch 4, Ch 8, Ch 10, Ch 21, Ch 47
megabits per second (Mbps)	Ch 2
megabyte (MB)	Ch 2
melt	Ch 18
memoization	Ch 33, Ch 72A
memory (RAM)	Ch 2
memory_usage(deep=True)	Ch 72
mentor	Ch 9
mentorship	Ch 83
MERGE	Ch 28
merge	Ch 18, Ch 54
merge (join types)	Ch 72
merge commit	Ch 26
merge conflict	Ch 26
merge fan-out	Ch 72

Term	Taught in
merge join	Ch 28
merge queries	Ch 11
merge sort	Ch 72A
Merge vs. Append	Ch 70
merge_asof	Ch 18
message queue	Ch 61
message queue / event bus	Ch 51
meta-analysis	Ch 9
metadata	Ch 1, Ch 56
metadata filter	Ch 55
method	Ch 17, Ch 18, Ch 29, Ch 34
metric	Ch 3, Ch 56
metric definition	Ch 23
MetricFlow (All terms are defined in the Glossary, Appendix A.) ---	Ch 32
micro-batch	Ch 50
Microsoft Fabric	Ch 16
Microsoft Graph	Ch 20
Microsoft Store	Ch 16
migration	Ch 12, Ch 28
MIME type	Ch 20
min-heap	Ch 72A
min_samples	Ch 38
min_samples_leaf	Ch 37
mini-batch	Ch 53
minimum detectable effect	Ch 22, Ch 30
minimum detectable effect (MDE)	Ch 73
MIS (management information system)	Ch 8
missing at random	Ch 14

Term	Taught in
missing completely at random	Ch 14
missing indicator	Ch 36
missing not at random	Ch 14
missing value	Ch 14, Ch 36
missing value / NULL	Ch 1
missing-value indicator	Ch 74
mix effect	Ch 23
mixed reference	Ch 10
ML engineer	Ch 7
MLOps	Ch 7, Ch 56
MLOps maturity ladder	Ch 56
MLP	Ch 53
mobile layout	Ch 16
mock interview	Ch 82
mode	Ch 1, Ch 4, Ch 21
model	Ch 7, Ch 32
model alias	Ch 54
model artifact	Ch 56
model card	Ch 39
model card (All terms are defined in the Glossary, Appendix A.) ---	Ch 53
model card (Chapter 39)	Ch 64
model contract	Ch 47
model governance	Ch 64
model lifecycle	Ch 56
model registry	Ch 56, Ch 64, Ch 79
model submission	Ch 82
model version	Ch 56
model-assisted intake	Ch 58
modeled layer	Ch 45

Term	Taught in
MODIFY COLUMN	Ch 12
module	Ch 17, Ch 19, Ch 29
momentum	Ch 53
monitoring	Ch 39
monitoring (absence of success)	Ch 78
monotonic association	Ch 73
monotonic transform	Ch 74
month key	Ch 10
month-end clamping	Ch 77
month-over-month	Ch 13
month-over-month growth	Ch 4
monthly projection	Ch 57
Monty Hall problem	Ch 73
move tag	Ch 69
moving average	Ch 13, Ch 40
moving average (MA) term	Ch 40
multi-agent system	Ch 79
multi-factor authentication (MFA)	Ch 2
multi-head attention	Ch 53
multicollinearity	Ch 37, Ch 74
multimodal	Ch 54
multinomial Naive Bayes	Ch 41
multipart message	Ch 20
multiple comparisons	Ch 22, Ch 73
multiplication rule	Ch 21
multiplicative model	Ch 40
must-have	Ch 8
must-have coverage	Ch 8, Ch 68
mutable default argument	Ch 18, Ch 72
mutation during iteration	Ch 72A

Term	Taught in
mutually exclusive	Ch 5, Ch 21
MVCC	Ch 69A, Ch 71
mypy	Ch 29
MySQL	Ch 12
<=> (MySQL NULL-safe equals)	Ch 71

N

Term	Taught in
n choose k	Ch 21
n-gram	Ch 41
n8n	Ch 20
n_init	Ch 38
Naive Bayes	Ch 37
naive forecast	Ch 40
naive practice	Ch 9
name box	Ch 10
named aggregation	Ch 18, Ch 72
named argument	Ch 19
named range	Ch 10
named window (WINDOW)	Ch 13
namedValues	Ch 19
namespace	Ch 52
NaN	Ch 72A
nanmax and friends	Ch 72
narrow operation	Ch 48
NaT	Ch 18
nat	Ch 35
natural (business) key	Ch 28
natural experiment	Ch 73
natural frequencies	Ch 73
natural log	Ch 31
natural log (ln)	Ch 35
NDCG (normalized discounted cumulative gain)	Ch 42
near-orthogonality in high dimensions	Ch 79

Term	Taught in
nearest neighbors (embedding space)	Ch 41
negative R²	Ch 74
nested function	Ch 33
nested loop join	Ch 28
net margin	Ch 23
net profit (PAT)	Ch 23
Net Promoter Score (NPS)	Ch 23
network effect (interference)	Ch 73
network partition	Ch 61
neural network	Ch 43
neuron	Ch 43, Ch 53
next-token prediction	Ch 54
nice-to-have	Ch 8
nines of availability	Ch 80
ninety-second scan	Ch 27
nn.Linear	Ch 43
nn.Module	Ch 43
nn.Sequential	Ch 43
"no data today"	Ch 20
no unmeasured confounding	Ch 31
no-data batch	Ch 50
node	Ch 33, Ch 37, Ch 52, Ch 72A
node ID	Ch 29
node selection (+, state:modified)	Ch 32
noise (residual)	Ch 40
noise floor	Ch 74
noise points	Ch 38
nominal	Ch 1

Term	Taught in
non-data row	Ch 14
non-determinism	Ch 54
non-deterministic LIMIT	Ch 71
non-functional requirement	Ch 25, Ch 76B
non-functional requirement (NFR)	Ch 60
non-negative matrix factorization (NMF)	Ch 41
non-response bias	Ch 22
nonexistent and ambiguous local times	Ch 77
Normal / log-normal distribution	Ch 73
normal approximation	Ch 22
normal distribution	Ch 21, Ch 35
normal form	Ch 28
normalisation	Ch 69A
normalization	Ch 12
normalize	Ch 14
North Star metric	Ch 23, Ch 75
NOT EXISTS	Ch 71
NOT IN NULL trap	Ch 71
NOT NULL	Ch 12
not_null	Ch 32
notebook	Ch 17
notebook → script → package	Ch 69A
notice period	Ch 81
notification service	Ch 63
NotImplementedError	Ch 18
novelty effect	Ch 30, Ch 73

Term	Taught in	
np.array_equal / np.allclose	Ch 72	
np.inf	Ch 18	
np.nan	Ch 18	
np.select	Ch 18	
np.where	Ch 18	
NPS scale (–100 to +100)	Ch 75	
.npy file	Ch 53	
NR and FNR	Ch 34	
NTILE	Ch 27	
NTILE (recap)	Ch 28	
NULL	Ch 12	
null hypothesis	Ch 22, Ch 30, Ch 73	
NULL propagation in		Ch 71
null result	Ch 27	
nullable Int64	Ch 72	
nullable integer (Int64)	Ch 18, Ch 77	
NULLIF	Ch 71	
NULLS FIRST / NULLS LAST	Ch 71	
number	Ch 1, Ch 10	
number format	Ch 10	
numbers stored as text	Ch 70	
NUMERIC vs floating point	Ch 12	
numerical gradient check	Ch 43	
NumPy	Ch 18	

O

Term	Taught in
$O(1)/O(\log n)/O(n)/O(n \log n)/O(n^2)/O(2^n)$	Ch 72A
OAuth 2.0	Ch 51
object identity	Ch 72
object model	Ch 19
object storage	Ch 49
object-level security	Ch 16
observability	Ch 47
observed and expected counts	Ch 22
ODBC	Ch 11
odds	Ch 30, Ch 37
odds ratio	Ch 30, Ch 37
OEE	Ch 23
offer negotiation	Ch 81
Office Scripts	Ch 19, Ch 70
official documentation	Ch 6
offset	Ch 50
OIDC	Ch 52
Okabe–Ito palette	Ch 15
ON DELETE CASCADE	Ch 12
ON DELETE CASCADE / NO ACTION	Ch 71
On Error	Ch 70
On Error GoTo	Ch 19

Term	Taught in
on-demand pricing	Ch 65
on-premises data gateway	Ch 16
one message per slide	Ch 24
one-hot encoding	Ch 36, Ch 74
one-page analysis memo	Ch 24
one-page strategy	Ch 66
one-page summary (take-home)	Ch 82
one-sided test	Ch 22
one-tailed vs. two-tailed test	Ch 73
one-to-many	Ch 11, Ch 12
one-vs-rest	Ch 41
online serving	Ch 56
online serving (Chapter 56)	Ch 62
online vs. batch serving	Ch 79
open-weight model	Ch 54
openpyxl	Ch 18
operating cycle	Ch 23
operating expenses (opex)	Ch 23
operating point	Ch 53, Ch 59
operator	Ch 10
opinion	Ch 5
optimistic concurrency	Ch 49

Term	Taught in
optimizer	Ch 53
option	Ch 17, Ch 20, Ch 26
Option Explicit	Ch 19
Optuna Also met in the exercises and the story: validation curve	Ch 37
orchestrator	Ch 46, Ch 79
order	Ch 3
ORDER BY col IS NULL	Ch 71
order of magnitude	Ch 4
order to cash	Ch 3
ordered-set aggregate	Ch 28
ordinal	Ch 1
ordinal encoding	Ch 36
ordinal GROUP BY	Ch 71
orientation	Ch 7
others	Ch 34
OTIF	Ch 23
out-of-fold predictions	Ch 53
outcome monitoring	Ch 56
outcome-driven field	Ch 36
outlier	Ch 14, Ch 21
output mode (append, update, complete)	Ch 50

Term	Taught in
output normalization	Ch 57
OVER	Ch 13
over-correction ---	Ch 69
over-differencing	Ch 40
over-partitioning	Ch 49
overdue	Ch 3
overfitting	Ch 37, Ch 53, Ch 74
overlap	Ch 55, Ch 79
overlap (common support)	Ch 31
overlapping runs ---	Ch 78
overplotting	Ch 15
override with reason	Ch 59
oversampling	Ch 39
owner	Ch 34
ownership	Ch 59, Ch 63

P

Term	Taught in
p-hacking	Ch 22
p-value	Ch 22, Ch 30, Ch 73
p95 (95th percentile)	Ch 60
PaaS	Ch 52
PACELC	Ch 61, Ch 80
package	Ch 17, Ch 29, Ch 32
packaging	Ch 56
padding	Ch 43, Ch 53
page	Ch 28
pagination	Ch 29
pagination (offset, cursor)	Ch 45
paging	Ch 18
paired design	Ch 73
paired test	Ch 22
pandas	Ch 18
pandas DataFrame	Ch 72
pandas Series	Ch 72
panel consistency ---	Ch 81
parallel trends	Ch 31
parameter	Ch 11, Ch 17, Ch 56
parameter-efficient fine-tuning	Ch 54
parameters	Ch 53
parameters (weights)	Ch 35
parametrize	Ch 29
Pareto principle	Ch 13
Parquet	Ch 2, Ch 18, Ch 45, Ch 49
part of speech	Ch 41

Term	Taught in
partial autocorrelation function (PACF)	Ch 40
partial correlation	Ch 73
partial dependence	Ch 39
partial dependency	Ch 28
partial derivative	Ch 35
partial index	Ch 28
partition	Ch 46, Ch 48, Ch 50
PARTITION BY	Ch 13, Ch 71
partition key	Ch 46
partition overwrite	Ch 46
partition pruning	Ch 48, Ch 77
partition tolerance	Ch 61
partitioning	Ch 49, Ch 61, Ch 77
partitioning vs. sharding	Ch 80
passphrase	Ch 34
password manager	Ch 2
paste special	Ch 10
paste values	Ch 10
PATH	Ch 34
path	Ch 2
path (absolute, relative)	Ch 26
path object	Ch 17
path parameter	Ch 51
path set	Ch 33
pathlib	Ch 17
pattern profile	Ch 14
payback period	Ch 23
payment	Ch 3
payment terms	Ch 3

Term	Taught in
PCA	Ch 38
pd.cut	Ch 18
PDF	Ch 2, Ch 20
peak factor	Ch 80
Pearson	Ch 22
Pearson against Spearman	Ch 73
peeking	Ch 22, Ch 30
peeking (repeated significance testing)	Ch 73
peer group	Ch 28, Ch 71
penalty	Ch 37
PEP 8	Ch 17
per-call cost	Ch 69A
percent change	Ch 4
percent of	Ch 4
percent point function (ppf)	Ch 21
percent-encoding against form-encoding (quote / quote_plus)	Ch 78
percentage	Ch 4
percentage point	Ch 4
percentage points against relative percent	Ch 75
percentile	Ch 8, Ch 21
percentile interpolation method ---	Ch 73
percentile threshold	Ch 20
PERCENTILE_CONT (recap)	Ch 28
PERCENTILE_DISC	Ch 28

Term	Taught in
Performance analyzer	Ch 16
periodic snapshot	Ch 28
perishable tools	Ch 83
permission bits	Ch 34
permission policy	Ch 52
permutation importance	Ch 37, Ch 39, Ch 74
permuted-target check	Ch 74
perplexity	Ch 38
persistence	Ch 20
persistence rule	Ch 40
personal data	Ch 1, Ch 64
Personal Macro Workbook	Ch 19, Ch 70
petabyte (PB)	Ch 2
phishing	Ch 2
physical plan	Ch 48
picking list	Ch 3
pickle risk	Ch 56
pie chart	Ch 15
PII (personally identifiable information)	Ch 64
pilot (as a decision-testing step)	Ch 75
pinned model version	Ch 57
pinned requirements	Ch 29
pip	Ch 17
pipe	Ch 34
pipeline	Ch 7, Ch 23, Ch 36, Ch 46
Pipeline ---	Ch 74

Term	Taught in
pipeline version	Ch 58
pivot	Ch 13
pivot chart	Ch 11
pivot table	Ch 11, Ch 70
pivot_table	Ch 18, Ch 72
PIVOTBY	Ch 11
PivotCache	Ch 19
placebo test	Ch 31
placeholder	Ch 58
placeholder value	Ch 14
plan	Ch 52
plateau	Ch 9
Platt scaling	Ch 37, Ch 39
Pod	Ch 52
point anomaly	Ch 40
point-in-time features	Ch 59
point-in-time join	Ch 28
point-to-point integration	Ch 51
Poisson distribution	Ch 21, Ch 35, Ch 73
polarity score	Ch 41
Polars	Ch 18, Ch 48
policy	Ch 52, Ch 64
polling	Ch 60, Ch 63
polling vs. webhook	Ch 78
polymorphism	Ch 18
pooled rate	Ch 73
pooled standard deviation	Ch 31
pooled standard error	Ch 30
pooling	Ch 53

Term	Taught in
pooling (max pooling)	Ch 43
popularity baseline	Ch 42
population	Ch 21, Ch 30
population against sample standard deviation	Ch 73
population stability index	Ch 56
population stability index (PSI)	Ch 79
port	Ch 12, Ch 34
port forwarding	Ch 34
Porter stemmer	Ch 41
portfolio	Ch 9
portfolio deep-dive	Ch 76A
portfolio piece	Ch 9, Ch 68
portfolio project	Ch 27
portfolio thinking	Ch 67
POS	Ch 3
positional argument	Ch 20
positional encoding	Ch 53
post-hoc test	Ch 30
post-mortem	Ch 56
post-training quantization	Ch 53
posterior	Ch 21, Ch 37
PostgreSQL	Ch 12
power	Ch 22
Power Automate	Ch 19, Ch 20
Power BI Desktop	Ch 16
Power BI Service	Ch 16
Power Pivot	Ch 11
Power Query	Ch 11, Ch 69A, Ch 70

Term	Taught in
power rule	Ch 35
power–interest grid	Ch 24
PR-AUC	Ch 74
PR-AUC (average precision)	Ch 39
practical significance	Ch 22
practice log	Ch 9
pre-attentive attributes	Ch 15
pre-change fit	Ch 31
pre-registration	Ch 22
precise question	Ch 5
precision	Ch 39, Ch 53, Ch 74
precision as displayed	Ch 70
precision@k	Ch 42
precision–recall curve	Ch 39
precondition	Ch 72A
predicate overwrite	Ch 49
predicate pushdown	Ch 69A, Ch 77
predict	Ch 36
predict_proba	Ch 36
prediction interval	Ch 40
prediction moment	Ch 36
predictive parity	Ch 39
predictive question	Ch 5
preference tuning	Ch 54
Premium Per User	Ch 16
prescriptive question	Ch 5
pretrained model	Ch 43
pretraining	Ch 54

Term	Taught in
preventing a class of error	Ch 76A
previous working day	Ch 47
primary data	Ch 1
primary key	Ch 12
primary metric	Ch 22, Ch 30
principal component	Ch 35
principal component analysis (PCA)	Ch 35
principle of least privilege	Ch 80
print area	Ch 10
print titles (All terms are defined in the Glossary, Appendix A.) ---	Ch 10
printf	Ch 34
prior	Ch 21, Ch 37
prior (All terms are defined in the Glossary, Appendix A.) ---	Ch 30
privacy budget (epsilon)	Ch 64
privacy by design	Ch 64
private address range	Ch 34
private key	Ch 34
Pro	Ch 16
probability	Ch 4, Ch 21
probability density function (pdf)	Ch 21
probability distribution	Ch 35
probability mass function (pmf)	Ch 21

Term	Taught in
problem framing (DS)	Ch 76A
procedure	Ch 19
process discovery	Ch 63
process redesign	Ch 58
processing time	Ch 50
producer	Ch 50
product backlog	Ch 26
product backlog vs. sprint backlog	Ch 76B
Product Owner	Ch 26, Ch 76B
product quantisation	Ch 79
product-decision case	Ch 75
profile	Ch 32
profiles.yml	Ch 32
profiling	Ch 14, Ch 18, Ch 33
profit & loss (P&L)	Ch 23
profit curve	Ch 39
program	Ch 17
project	Ch 9
project frame	Ch 59
promoter / passive / detractor	Ch 75
prompt	Ch 17, Ch 54
prompt caching	Ch 54, Ch 55, Ch 57
prompt injection	Ch 54, Ch 79
prompt injection through documents	Ch 55
prompt prefix caching ---	Ch 79
prompt registry	Ch 57
prompt version	Ch 57

Term	Taught in
prompt versioning	Ch 79
proof of delivery (POD)	Ch 3
propensity score	Ch 31
PropertiesService	Ch 19
@property	Ch 18
property	Ch 29
Prophet	Ch 40
proportion	Ch 30
protected attribute	Ch 64
provider	Ch 52
provider upgrade	Ch 57
provisioned storage	Ch 65
proxy	Ch 64
proxy discrimination	Ch 64
proxy metric	Ch 57
pruning	Ch 53
pseudonymization	Ch 64
public key	Ch 34
publish swap	Ch 47
pull request	Ch 26, Ch 29
pure function	Ch 29
purpose limitation	Ch 64
push the work to the data	Ch 69A
pushback	Ch 24
pushed filter	Ch 48
pwd, ls, cd	Ch 17
PyPI	Ch 17
pyproject.toml	Ch 29
pyramid principle	Ch 24

Term	Taught in
PySpark	Ch 48
pytest	Ch 29
Python	Ch 17
Python install manager	Ch 17
PyTorch	Ch 43
$p^{\wedge}$ (p-hat)	Ch 22

Q

Term	Taught in
Q&A	Ch 16
QLoRA	Ch 54
quadratic	Ch 33
QUALIFY	Ch 13
qualitative / categorical	Ch 1
quantified impact	Ch 76A
quantile	Ch 18
quantisation (int8, int4)	Ch 79
quantitative	Ch 1
quantization	Ch 53
quantization-aware training	Ch 53
quarantine	Ch 14
quarterly access review	Ch 64
quartile	Ch 15, Ch 21, Ch 27
QUERY	Ch 11
query	Ch 11, Ch 18, Ch 53
QUERY (Sheets)	Ch 70
query folding	Ch 16
query language	Ch 69A
query parameter	Ch 29
query plan	Ch 28, Ch 48
queue	Ch 33
queue (FIFO)	Ch 72A

Term	Taught in
queueing collapse ---	Ch 80
quicksort	Ch 72A
quorum	Ch 61
quota	Ch 19
quota attainment	Ch 23
quote / quotation	Ch 3
quoting	Ch 34

R

Term	Taught in
RACI (Responsible, Accountable, Consulted, Informed)	Ch 24
Raft	Ch 61
RAG (Retrieval-Augmented Generation)	Ch 79
raise_for_status	Ch 18
RAM (memory)	Ch 6
random forest	Ch 37
random number generator	Ch 21
random sample	Ch 21
random search	Ch 37
random split	Ch 36
randomization	Ch 73
randomization unit	Ch 22, Ch 30
Range	Ch 19
range	Ch 10, Ch 17, Ch 21
RANK	Ch 13
rank	Ch 54
rank-based metric	Ch 74
ransomware	Ch 2
rate	Ch 4
rate (λ)	Ch 21
rate limit	Ch 2, Ch 18, Ch 29, Ch 57
rate limit (429)	Ch 45
ratio	Ch 1, Ch 4
raw / staging / modeled	Ch 62
raw layer	Ch 45

Term	Taught in
raw string	Ch 72
raw, staging, mart layers	Ch 47
Ray (All terms are defined in the Glossary, Appendix A.) ---	Ch 48
RBAC (role-based access control)	Ch 64
RBF kernel	Ch 37
re-keying	Ch 3, Ch 25
re.match against re.search	Ch 72
read replica	Ch 69A, Ch 76A
read-only tool	Ch 55
read-your-writes	Ch 61
read_csv parameters	Ch 18
reader pathway	Ch 8
readiness	Ch 56
readiness probe	Ch 52
README	Ch 26
README first paragraph	Ch 27
README sheet	Ch 11
real evidence	Ch 69
reasoning out loud	Ch 81
recall	Ch 53, Ch 74
recall (sensitivity, true positive rate)	Ch 39
recall@k	Ch 42, Ch 55, Ch 79
receivables / accounts receivable	Ch 3
recency	Ch 5
recipient list	Ch 20

Term	Taught in
recommender system	Ch 42
reconciliation	Ch 10, Ch 12, Ch 14, Ch 25, Ch 27, Ch 47, Ch 60, Ch 76A
reconciliation / manual matching	Ch 3
reconciliation check	Ch 45
reconciliation criterion	Ch 76B
reconstructing a trade-off ---	Ch 69A
recovery asymmetry	Ch 75
recruiter call	Ch 68
recursion	Ch 33, Ch 72A
recursion limit	Ch 33
RecursionError	Ch 72A
RECURSIVE	Ch 28
recursive case	Ch 33
recursive CTE	Ch 28, Ch 71
recursive forecast	Ch 40
recursive part	Ch 28
red flag (interview)	Ch 69
redirection	Ch 34
ref	Ch 32
reference architecture	Ch 63
reference category	Ch 30
referential integrity	Ch 12, Ch 14
referral	Ch 68
refresh	Ch 11
refreshable	Ch 20
refusal	Ch 55, Ch 79
refusal rate	Ch 57
region	Ch 52

Term	Taught in
regional endpoint	Ch 57
registry	Ch 52
regression	Ch 22, Ch 36, Ch 37
regression adjustment	Ch 31
regression check	Ch 56
regression discontinuity	Ch 31
regression test (LLM)	Ch 79
regression to the mean	Ch 5, Ch 22
regular expression (regex)	Ch 14
regularization	Ch 37
regularization (L1/L2)	Ch 74
rehearsal (back-test)	Ch 44
rejected rows	Ch 45
RELATED	Ch 11
related image	Ch 20
relational database	Ch 12
relationship	Ch 11, Ch 16
relationships	Ch 32
relative lift	Ch 30
relative reference	Ch 10
relevance	Ch 31
reliability curve	Ch 39
relieving letter	Ch 81
ReLU	Ch 53
ReLU (rectified linear unit)	Ch 43
remote	Ch 26

Term	Taught in
remove duplicates	Ch 10
REMOVEFILTERS	Ch 16
reorder flag	Ch 51
repair	Ch 14
repeat contact	Ch 41
repeatability	Ch 69A
repeatable migration	Ch 28
REPL	Ch 17
replay	Ch 50, Ch 51
replica	Ch 50, Ch 52, Ch 61
replication	Ch 61
replication factor	Ch 80
replication slot	Ch 45
report	Ch 3, Ch 16
report automation	Ch 78
report flow map	Ch 20
reporting date	Ch 20
repository	Ch 26
repository secret	Ch 32
repr	Ch 17
reproducibility ---	Ch 72
request	Ch 2
request/API meter	Ch 65
requests	Ch 18
requirements gathering	Ch 24
requirements traceability matrix	Ch 25, Ch 76B
requirements.txt	Ch 17
requires_grad	Ch 43
reranking	Ch 55, Ch 79

Term	Taught in
resample	Ch 18
resampling with replacement	Ch 22
reserved pricing	Ch 65
reserved word	Ch 13
residual	Ch 22, Ch 37, Ch 73
residual connection	Ch 53
residual fitting	Ch 37
residual plot	Ch 22
resolved-and-satisfied	Ch 59
resource requests and limits	Ch 52
response	Ch 2
response (dependent) variable	Ch 22
Responsible AI review	Ch 80
restore	Ch 49
Resume	Ch 19
resume screen	Ch 68
resume scrutiny	Ch 76A
resume-driven design	Ch 60
retention	Ch 13, Ch 49, Ch 50, Ch 64
retention period	Ch 57
retention policy	Ch 80
retention rate	Ch 23
retraining trigger	Ch 56, Ch 79
retrieval practice	Ch 9
retrieval-augmented generation	Ch 54

Term	Taught in
retrieval-augmented generation (RAG)	Ch 55
retrospective	Ch 26
retry	Ch 20, Ch 29
retry loop	Ch 54
retry policy	Ch 46
retry queue	Ch 33
retry with backoff	Ch 18
Retry-After	Ch 29, Ch 45, Ch 78
return	Ch 17
return array	Ch 11
return on assets (ROA)	Ch 23
return on equity (ROE)	Ch 23
return on investment (ROI)	Ch 66
RETURNING / LAST_INSERT_ID()	Ch 12
revenue	Ch 3, Ch 23
reverse causation	Ch 5, Ch 22
reverse ETL	Ch 51, Ch 78
reverse ETL (data activation)	Ch 7
reverse percentage	Ch 4
reversibility	Ch 5
review capacity	Ch 59
review catch rate	Ch 58
review from memory	Ch 6
reviewer interface	Ch 58
REVOKE	Ch 64
RFM	Ch 38

Term	Taught in
ribbon	Ch 10
RICE	Ch 75
ridge regression (L2)	Ch 37
right join	Ch 12
right to erasure (right to be forgotten)	Ch 64
right-censoring	Ch 13
right-skewed	Ch 4
RLHF	Ch 54
RMSE	Ch 39
RNN	Ch 53
ROAS	Ch 23
robots.txt	Ch 45
robust baseline	Ch 40
ROC curve	Ch 39
ROC-AUC	Ch 36, Ch 39, Ch 74
ROI	Ch 58
ROI/payback period	Ch 80
role	Ch 7, Ch 52, Ch 64
rollback	Ch 56, Ch 58
rolling	Ch 18
rolling feature	Ch 40
rolling origin	Ch 40
rolling z-score	Ch 40
root cause	Ch 25, Ch 76B
root mean squared error	Ch 31
rounding	Ch 4
row / record	Ch 1
ROW / ROWS	Ch 11
row context	Ch 16

Term	Taught in
row group	Ch 49
row normalization	Ch 41
row-level filtering	Ch 20
row-level security	Ch 16
row-level security (RLS)	Ch 64, Ch 70
row-oriented storage	Ch 49
row/record	Ch 12
ROW_NUMBER	Ch 13
ROW_NUMBER / RANK / DENSE_RANK	Ch 71
Rows area	Ch 11
ROWS vs RANGE	Ch 13
RPA	Ch 58
RPA (robotic process automation)	Ch 7, Ch 51, Ch 63
RPA developer	Ch 7
rsync	Ch 34
rubric	Ch 55
Ruff (All terms are defined in the Glossary, Appendix A.) ---	Ch 29
rule of 72	Ch 4
rule of 72 ---	Ch 75
rule of three	Ch 58
rule-of-thumb baseline	Ch 36
run	Ch 56
run failure sensor	Ch 46
run history	Ch 20
run ID	Ch 58

Term	Taught in
run id	Ch 51
run key	Ch 20, Ch 46
run log	Ch 63
run-length encoding	Ch 49
runbook	Ch 46, Ch 47, Ch 56, Ch 63
running total	Ch 13, Ch 71
running variable	Ch 31
RuntimeError: dictionary changed size	Ch 72A
runway	Ch 68
R²	Ch 39
R² (coefficient of determination)	Ch 22, Ch 37

S

Term	Taught in
s and σ	Ch 21
SaaS	Ch 2, Ch 52
safety stock	Ch 40
sales cycle	Ch 23
salt	Ch 64
salting	Ch 48
SAMEPERIODLASTYEAR	Ch 16
sample	Ch 21, Ch 30
Sample Ratio Mismatch (SRM)	Ch 73
sample size calculation	Ch 30
sample versus population (ddof)	Ch 21
sample-ratio mismatch	Ch 30
sampled audit	Ch 56
sampling	Ch 54
sampling error	Ch 21, Ch 22
sanity check	Ch 4
sargable	Ch 28
SARIMA	Ch 40
save rate	Ch 44
scalar subquery	Ch 71
scale and maintenance	Ch 69
scale invariance	Ch 74
scaled dot-product attention	Ch 53
scaling	Ch 35
scaling (standardization)	Ch 36
scaling out (horizontal scaling)	Ch 61

Term	Taught in
scaling up (vertical scaling)	Ch 61
scatter plot	Ch 15
SCD Type 1/2/3	Ch 77
Scenario Manager	Ch 11
schedule	Ch 46
scheduled	Ch 20
scheduled automation	Ch 63
scheduled refresh	Ch 16
schema	Ch 12, Ch 45, Ch 50, Ch 56
schema (PostgreSQL)	Ch 12
schema change (schema drift)	Ch 45
schema drift	Ch 76A, Ch 77
schema enforcement	Ch 49
schema evolution	Ch 49
schema history table	Ch 28
schema merge	Ch 49
schema registry	Ch 50
schema validation	Ch 54
scientific notation	Ch 18, Ch 35
scientific-notation display	Ch 70
scikit-learn	Ch 35, Ch 36
scope	Ch 17, Ch 51, Ch 83
scope creep	Ch 76A
score normalization	Ch 55
.score()	Ch 74
scoring=	Ch 74
scp	Ch 34
screen scraping	Ch 58
ScreenUpdating	Ch 19

Term	Taught in
script	Ch 17, Ch 29
Scripting.Dictionary	Ch 19
Scrum	Ch 26
Scrum Master	Ch 26
SDLC	Ch 76B
seaborn	Ch 18
SEARCH DEPTH FIRST	Ch 28
search mode	Ch 11
seasonal differencing	Ch 40
seasonal index	Ch 40
seasonal naive	Ch 40
seasonal order	Ch 40
seasonality	Ch 40
second normal form (2NF)	Ch 28
secondary data	Ch 1
Secret (Kubernetes)	Ch 52
secrets management (Chapter 52)	Ch 64
secrets manager	Ch 29, Ch 52
security group	Ch 52
seed	Ch 21
seen set	Ch 33
segment effect	Ch 27
segment-level baseline	Ch 42
segmentation	Ch 58
segregation of duties	Ch 63, Ch 80
Select Case	Ch 19
SELECTEDVALUE	Ch 16
selection	Ch 5, Ch 31
selection bias	Ch 22, Ch 73
selective	Ch 28

Term	Taught in
selective reporting	Ch 27
self	Ch 18, Ch 29
self-aware weakness	Ch 81
self-correction	Ch 81
self-healing load	Ch 47
self-join	Ch 12
self-reported salary	Ch 8
self-serve	Ch 20
self-serve platform	Ch 62
semantic cache	Ch 57
semantic layer	Ch 32
semantic layer (Chapter 32)	Ch 62
semantic model (dataset)	Ch 16
semantic search	Ch 54
semi-structured data	Ch 1
senior analyst	Ch 8
sensitivity	Ch 21, Ch 73
sensor	Ch 46
sensor (orchestration)	Ch 77
sentiment analysis	Ch 41
sentiment lexicon	Ch 41
separation of storage and compute	Ch 49
SEQUENCE	Ch 11
sequential palette	Ch 15
sequential scan	Ch 28
sequential scan vs. index scan	Ch 71
sequential testing	Ch 22, Ch 30
Series	Ch 18
series	Ch 10

Term	Taught in
server	Ch 2
serverless	Ch 52
Service	Ch 52
service account	Ch 20, Ch 51, Ch 64
service bond	Ch 81
service container	Ch 32
service level	Ch 21
service metrics	Ch 56
service-level agreement (SLA)	Ch 46, Ch 75
serving layer	Ch 62
session	Ch 29
Set	Ch 19
set	Ch 17, Ch 33
set -euo pipefail	Ch 34
set ordering	Ch 72A
SettingWithCopyWarning (legacy)	Ch 72
severity (error, warning)	Ch 47
severity levels (S1, S2, S3)	Ch 47
SFTP	Ch 51
SGD	Ch 43
SHA-256	Ch 45
sha256sum	Ch 34
shadow IT	Ch 63
shadow mode	Ch 56
shadow mode (Chapter 56)	Ch 60
shadow reporting	Ch 76A
shadow rollout	Ch 58
shadow system	Ch 3, Ch 25
shallow copy	Ch 72, Ch 72A

Term	Taught in
SHAP	Ch 37, Ch 39
shape	Ch 18, Ch 35
shape rule	Ch 35
Shapley value	Ch 39
sharding	Ch 61, Ch 77
share / proportion	Ch 4
shared mailbox	Ch 20
shared responsibility model	Ch 52
shared responsibility model (Chapter 52)	Ch 64
shared services	Ch 63
shared vocabulary	Ch 62
shared weights	Ch 53
shebang	Ch 34
sheet protection	Ch 10
shell	Ch 26, Ch 34
shell built-in	Ch 34
shift	Ch 18
Show Values As	Ch 11
showback	Ch 65
showback vs. chargeback	Ch 80
shuffle	Ch 48
shuffle=False	Ch 74
side effect	Ch 46
sigmoid	Ch 37, Ch 43
sign-off	Ch 25
significance band	Ch 40
significance level (α)	Ch 22
Significant Data Fiduciary	Ch 64
significant figures	Ch 4

Term	Taught in
signposting	Ch 69
silent error rate	Ch 58, Ch 59
silent failure	Ch 78
silhouette score	Ch 38
similarity	Ch 14
simple and installable triggers	Ch 19
simple linear regression	Ch 22
simple-first	Ch 69
Simpson's paradox	Ch 22, Ch 73
single point of failure	Ch 80
single point of failure (SPOF)	Ch 61
single point of failure (undocumented automation)	Ch 78
single-writer lock	Ch 69A
singular test	Ch 32, Ch 47
sink	Ch 50
skew	Ch 15, Ch 21, Ch 48
skewed data	Ch 35
skill	Ch 9
skills matrix	Ch 8
slice	Ch 17
slicer	Ch 11, Ch 16
slide title as a finding	Ch 24
sliding-window pattern	Ch 72A
SLO against SLA	Ch 80
slope	Ch 22
slope and intercept	Ch 35
slope of an activation	Ch 43
slot lag	Ch 45

Term	Taught in
slowly changing dimension (SCD)	Ch 28
small files	Ch 50
small files problem	Ch 48, Ch 49
small multiples	Ch 15
small-integer cache	Ch 72A
small-numbers bias	Ch 5
Smart Fill	Ch 10
smoke test	Ch 57
smoothing	Ch 37
SMOTE	Ch 39
SMTP	Ch 20
snapshot	Ch 32, Ch 49
snapshot isolation	Ch 61
snowflake schema	Ch 28
soft delete	Ch 12
softmax	Ch 41, Ch 43, Ch 53, Ch 54
softmax temperature	Ch 79
software	Ch 29
software development life cycle (SDLC)	Ch 25
Solver	Ch 11
SORT	Ch 11
sort	Ch 10, Ch 34
sort key	Ch 33
sort stability (kind=)	Ch 72
sort-merge join	Ch 48
SORTBY	Ch 11
sorting	Ch 33
source	Ch 32
source freshness	Ch 32
source system	Ch 45

Term	Taught in
source()	Ch 32
space complexity	Ch 72A
spaced repetition	Ch 9
Spark web interface	Ch 48
sparkline	Ch 15
SparkSession	Ch 48
sparse matrix	Ch 36
sparse matrix (csr_matrix)	Ch 42
sparsity	Ch 42
Spearman	Ch 22
specificity	Ch 39
speed layer	Ch 62
spike	Ch 26, Ch 76B
spill range	Ch 11
spill reference (#)	Ch 11
#SPILL!	Ch 11
split	Ch 37
splittable compression	Ch 77
spread	Ch 21
spreadsheet application	Ch 10
SpreadsheetApp	Ch 19
sprint	Ch 26
sprint backlog	Ch 26
SQL	Ch 7
SQL dialect	Ch 12
SQL injection	Ch 18, Ch 29
SQLAlchemy engine	Ch 18
SQLFluff	Ch 32
SQLite	Ch 18
squared error	Ch 53
src/ layout	Ch 29

Term	Taught in
SRS	Ch 25, Ch 76B
ss	Ch 34
SSH	Ch 34
SSH server	Ch 34
~/.ssh/config	Ch 34
stability	Ch 38
stable sort	Ch 33, Ch 72A
stack	Ch 33
stack (LIFO)	Ch 72A
stack plateau	Ch 83
stacked bar	Ch 15
stacking	Ch 37
stage (deprecated)	Ch 56
staging area	Ch 26
staging layer	Ch 32, Ch 45
staging table	Ch 12, Ch 14
stakeholder	Ch 24
stakeholder mapping	Ch 24
stakeholder register	Ch 24, Ch 76B
staleness	Ch 28
staleness limit	Ch 47
Standard Contractual Clauses	Ch 64
standard deviation	Ch 17, Ch 21, Ch 73
standard error	Ch 21, Ch 22, Ch 30, Ch 34
standard input	Ch 34
standard library	Ch 17
standard output	Ch 34
standardization	Ch 22, Ch 35
standardize	Ch 14
standardized mean difference	Ch 31

Term	Taught in
standup	Ch 26
STAR method (Situation, Task, Action, Result)	Ch 81
star schema	Ch 16, Ch 70
star schema (recap)	Ch 28
STARTTLS	Ch 20
state	Ch 50
state file	Ch 52
stated assumption	Ch 69
statement	Ch 12, Ch 19, Ch 52
statement timeout	Ch 76A
stating the assumption	Ch 76A
stationarity	Ch 40
statistical power	Ch 30, Ch 73
statistical significance	Ch 27
statistics	Ch 28
statistics (min/max, distinct count)	Ch 49
statsmodels	Ch 22
status bar	Ch 10
status code	Ch 2, Ch 18, Ch 34
stdout and stderr	Ch 20
stemming	Ch 41
step limit	Ch 55, Ch 79
stochastic gradient descent	Ch 35
stop switch	Ch 20
stop words	Ch 41
stopping condition	Ch 76A
stopping rule	Ch 22, Ch 30
storage (SSD, hard disk)	Ch 2
storage meter	Ch 65

Term	Taught in
storage tiering	Ch 65
story bank	Ch 81
story point	Ch 26
.str accessor	Ch 18, Ch 72
straight-through processing	Ch 58
straight-through rate	Ch 59
strangler pattern	Ch 69A
stratified k-fold	Ch 36
stratified sample	Ch 21
StratifiedKFold against KFold	Ch 74
stratify	Ch 36, Ch 74
stream replay	Ch 62
streaming	Ch 50, Ch 57
streaming serving	Ch 56
stride	Ch 53
string	Ch 17
STRING_AGG / GROUP_CONCAT	Ch 71
strip plot	Ch 15
strong consistency	Ch 61
strptime/strftime	Ch 17
structure & communication	Ch 69
structured data	Ch 1
structured log	Ch 57
structured logging	Ch 56
structured output	Ch 54
Structured Streaming	Ch 50
study	Ch 9
study hours	Ch 6
study log	Ch 6

Term	Taught in
study plan	Ch 6
stump	Ch 37
Sub	Ch 19
subclass	Ch 18
subnet	Ch 52
subproblem	Ch 33
subquery	Ch 12
SUBTOTAL	Ch 10
sudo	Ch 34
SUMIFS/COUNTIFS	Ch 70
SUMPRODUCT	Ch 70
SUMX	Ch 11
superuser	Ch 34
supervised fine-tuning	Ch 54
supervised learning	Ch 36, Ch 37
support	Ch 38, Ch 39, Ch 41
support desk / ticketing system	Ch 3
support vector	Ch 37
support vector machine	Ch 37
surprise	Ch 35
surrogate key	Ch 28, Ch 77
survival rate	Ch 75
survivorship bias	Ch 5, Ch 22
swimlane diagram	Ch 25, Ch 76B
symbol	Ch 54
symbol map	Ch 15
sync	Ch 2
synthetic control	Ch 31
sys.argv	Ch 17
system of record	Ch 45, Ch 51, Ch 58, Ch 60

Term	Taught in
system of record / source of truth	Ch 3
system prompt	Ch 54

T

Term	Taught in
t-distribution	Ch 22, Ch 30
T-shaped	Ch 83
t-SNE	Ch 38
t-test vs. z-test	Ch 73
table	Ch 1, Ch 10, Ch 12, Ch 32
table format	Ch 49
table layout	Ch 20
table reference	Ch 10
tag	Ch 19, Ch 52
tagging	Ch 65
tail	Ch 21, Ch 34
tail latency	Ch 80
take-home assignment	Ch 68, Ch 82
tanh	Ch 43
target	Ch 32, Ch 36
target (y)	Ch 35
target encoding	Ch 36
target encoding leakage	Ch 74
target leakage	Ch 36, Ch 74
task	Ch 46
Task Scheduler	Ch 18, Ch 20
TCL	Ch 12
teaching as learning	Ch 83
team topology	Ch 66, Ch 67
Teams Workflows	Ch 20
technical debt	Ch 67
tee	Ch 34

Term	Taught in
temperature	Ch 54, Ch 79
templater	Ch 32
temporal leakage	Ch 74
temporary credentials	Ch 52
temporary table	Ch 13
tensor	Ch 43
terabyte (TB)	Ch 2
term frequency (TF)	Ch 41
term saturation	Ch 55
terminal	Ch 17, Ch 26
Terraform	Ch 52
test	Ch 29, Ch 34
test (condition)	Ch 10
test client	Ch 56
test double	Ch 29
test mail server (aiosmtpd)	Ch 20
test set	Ch 36
testable	Ch 5
text (string)	Ch 10
text / string	Ch 1
text classification	Ch 41
Text to Columns	Ch 10
TF-IDF	Ch 41
the 1900 leap-year bug	Ch 70
the request underneath the request	Ch 76A

Term	Taught in
the two-minute version	Ch 27
theme	Ch 16
third normal form (3NF)	Ch 28
third-party data	Ch 1
{{ this }}	Ch 32
thousands separator	Ch 14
three-layer monitoring	Ch 56
three-valued logic	Ch 12
three-valued logic (true / false / unknown)	Ch 71
threshold	Ch 20, Ch 35, Ch 39, Ch 53
threshold re-tuning	Ch 56
threshold sensitivity	Ch 27
thumbs-down rate	Ch 57
thundering herd	Ch 78
ticket	Ch 3
tidy data	Ch 18
tie-breaker	Ch 13, Ch 71
tier	Ch 8
time complexity	Ch 72A
time intelligence	Ch 11, Ch 16
time series	Ch 18, Ch 40
time series split	Ch 36

Term	Taught in
time to first token	Ch 57
time travel	Ch 49
time zone	Ch 14, Ch 20
time zone (UTC, IST)	Ch 46
time-based split	Ch 36
time-based window	Ch 40
timeit	Ch 33
timeline	Ch 11
timeliness	Ch 1, Ch 25
timeout	Ch 18, Ch 29, Ch 45, Ch 46, Ch 57, Ch 61
Timer	Ch 19
TimeSeriesSplit	Ch 74
timestamp strategy	Ch 32
timestamp with time zone	Ch 14
Timsort	Ch 33, Ch 72A
TLS	Ch 64
to-be	Ch 25
to_sql	Ch 18
token	Ch 41, Ch 54, Ch 79
token meter	Ch 57
tokenization	Ch 41
tokenizer	Ch 54
tolerance	Ch 25
tolerant parsing	Ch 57
TOML	Ch 29
tool	Ch 55

Term	Taught in
tool decision matrix	Ch 63
tool schema	Ch 55
tool timeline	Ch 6
tool use	Ch 79
tool whitelist	Ch 55
tooltip page	Ch 16
top N per group	Ch 13
top-down vs. bottom-up estimate	Ch 75
top-k	Ch 54
top-N table	Ch 39
top-p (nucleus sampling)	Ch 54
top-p against top-k	Ch 79
topic	Ch 50
topic drift	Ch 57
topic modeling	Ch 41
topic-word distribution	Ch 41
topological sort	Ch 77
total addressable market (TAM)	Ch 75
total cost of ownership (TCO)	Ch 80
total deadline	Ch 78
total pay	Ch 8
total probability	Ch 21
TOTALYTD	Ch 11, Ch 16
touch time	Ch 58
trace	Ch 57

Term	Taught in
trace precedents	Ch 10
traceback	Ch 17
tracemalloc	Ch 33, Ch 72
tracing (LLM)	Ch 79
track	Ch 7
tracked file	Ch 26
tracking store	Ch 56
trade-off	Ch 60, Ch 69
trailing space	Ch 10
train-test contamination	Ch 36
trained sentiment classifier	Ch 41
training set	Ch 36
training-serving skew	Ch 74, Ch 76A, Ch 79
transaction	Ch 3, Ch 12, Ch 58
transaction (BEGIN, COMMIT, ROLLBACK)	Ch 46
transaction fact table	Ch 28
transaction log	Ch 49
transactional provider	Ch 20
transfer learning	Ch 43
transform	Ch 18
.transform() vs. .agg()	Ch 72
transformer	Ch 53
transformer (fit / transform)	Ch 36

Term	Taught in
transient failure	Ch 29, Ch 46, Ch 58
transitive dependency	Ch 28
transparency duties	Ch 64
transpose	Ch 35
treated and untreated groups	Ch 31
treatment coding	Ch 30
tree	Ch 33
tree height	Ch 72A
treemap	Ch 15
trend	Ch 40
triage (in preparation) ---	Ch 68
trigger	Ch 50
triggered	Ch 20
trigram	Ch 14
Trino	Ch 48
trivial rule	Ch 38
true negative	Ch 39
true positive	Ch 39
TRUNCATE	Ch 12
truncated axis	Ch 4, Ch 15
TruncatedSVD	Ch 42
truncation vs. rounding	Ch 71
trust policy	Ch 52
trusted location	Ch 19
trusted types	Ch 56
truthiness	Ch 17

Term	Taught in
try/except	Ch 17
TRY_CAST	Ch 45
Tukey HSD	Ch 30
tuning	Ch 39
tuple	Ch 17, Ch 33
two-by-two table	Ch 31
two-factor authentication	Ch 26
two-level index	Ch 18
two-pointer pattern	Ch 72A
two-proportion test	Ch 30
two-proportion z-test	Ch 22
two-sided test	Ch 22
two-stage least squares	Ch 31
two-way lookup	Ch 11
type (str, int, float, bool, None)	Ch 17
type 1	Ch 28
type 2	Ch 28
type 3	Ch 28
type checker	Ch 29
type conversion	Ch 17
type hint	Ch 29
type hints	Ch 72
Type I error	Ch 30, Ch 73
type I error	Ch 22
Type II error	Ch 30, Ch 73
type II error	Ch 22

Term	Taught in
type inference	Ch 77
TypeScript	Ch 19
tz-aware against naive	Ch 72

U

Term	Taught in
UAT	Ch 76B
UAT sign-off ---	Ch 76B
UDF	Ch 19
UMAP	Ch 38
UML class diagram	Ch 60
unbiased estimate	Ch 21
UNBOUNDED PRECEDING	Ch 13
unbounded table	Ch 50
underfitting	Ch 37, Ch 74
undersampling	Ch 39
undocumented constraints	Ch 59
Unicode	Ch 2
uniform distribution	Ch 21
uniform distribution of p-values	Ch 73
UNION / UNION ALL	Ch 12
UNION ALL in recursion	Ch 28
union, intersection, difference	Ch 17
uniq	Ch 34
UNIQUE	Ch 11, Ch 12
unique	Ch 32
unique constraint	Ch 58
uniqueness	Ch 1
unit conversion	Ch 14
unit economics	Ch 65
unit of analysis	Ch 30, Ch 36
unit test	Ch 29

Term	Taught in
unit-length vector	Ch 79
unknown reference	Ch 58
unmeasured ceiling	Ch 66
unpacking	Ch 17
unparseable rate	Ch 57
unpivot	Ch 11
unstack	Ch 18
unstructured data	Ch 1
unstructured text	Ch 41
unsupervised learning	Ch 38
untracked file	Ch 26
UPDATE	Ch 12
update anomaly	Ch 28
update step	Ch 38
upsert	Ch 12, Ch 45
upsert (by external id)	Ch 51
upsert (in an integration context)	Ch 78
upsert (ON CONFLICT, EXCLUDED)	Ch 77
UrlFetchApp	Ch 19
usage metrics	Ch 16
use case	Ch 25, Ch 76B
useful skill	Ch 8
user acceptance testing (UAT)	Ch 25
user story	Ch 25, Ch 76B
user-based collaborative filtering	Ch 42
user-defined function (UDF)	Ch 48
UserForm	Ch 19, Ch 70
USERPRINCIPALNAME	Ch 16

Term	Taught in
UTC	Ch 14
UTC-first scheduling	Ch 77
UTF-8	Ch 2
utilisation headroom	Ch 80
uv	Ch 29
uv.lock	Ch 29

Term	Taught in
vacuum	Ch 49
VADER	Ch 41
validate	Ch 18
validate= on a merge	Ch 72
validation (sanity check)	Ch 69
validation rule	Ch 14
validation set	Ch 36
validity	Ch 1
value	Ch 1, Ch 53
value filter	Ch 11
value/risk/effort prioritization	Ch 63
Value2	Ch 19
value_counts	Ch 18
Values area	Ch 11
vanishing gradient	Ch 43
vanity metric	Ch 23, Ch 75
VAR/RETURN	Ch 16
variable	Ch 17
variable cost	Ch 23
variance	Ch 21, Ch 35, Ch 37, Ch 73
variance inflation factor (VIF)	Ch 74
variance of a difference	Ch 73
variant	Ch 30
VBA	Ch 19
VBA object model	Ch 70

Term	Taught in
vector	Ch 35
vector database	Ch 79
vector search	Ch 55
vector storage arithmetic	Ch 79
vectorisation	Ch 69A
vectorization	Ch 72
vectorized operation	Ch 18
vendor lock-in	Ch 66
venv	Ch 29
version	Ch 49
version control	Ch 26
version history	Ch 2, Ch 10
version number	Ch 61
view	Ch 13, Ch 32
view against copy	Ch 72
violin plot	Ch 15
virtual environment	Ch 29
virtual environment (venv)	Ch 17
virtual machine	Ch 16
Visual Basic Editor	Ch 19
VLOOKUP	Ch 11
vocabulary	Ch 41
volume	Ch 52
volume check	Ch 47
volume-anomaly check (reused) -- -	Ch 82

Term	Taught in
VPC (virtual private cloud)	Ch 52
VS Code	Ch 17
VSTACK	Ch 11

W

Term	Taught in
walrus operator (:=)	Ch 72
WAPE	Ch 39, Ch 40
waterfall	Ch 25
waterfall (bridge) chart	Ch 15
watermark	Ch 45, Ch 50, Ch 77
wc	Ch 34
WCAG	Ch 15
web scraping	Ch 45
web-safe font	Ch 20
webhook	Ch 2, Ch 51
webhook idempotency	Ch 78
weekly active users	Ch 75
weekly budget	Ch 83
weekly check	Ch 9
weekly rhythm	Ch 6
weight	Ch 43, Ch 53
weight and bias (w and b)	Ch 35
weight decay	Ch 53
weighted average	Ch 4, Ch 11, Ch 21, Ch 41, Ch 73
weighted F1	Ch 39

Term	Taught in
Welch's t-test	Ch 22, Ch 30
what-if analysis	Ch 11
WhatsApp Business API	Ch 20
WHERE vs. HAVING	Ch 71
where-used query	Ch 28
while	Ch 17
whisker	Ch 15
wide and long	Ch 18
wide data	Ch 11
wide operation	Ch 48
wildcard	Ch 10, Ch 17
Wilson interval	Ch 22
win rate	Ch 23
window (tumbling, sliding, session)	Ch 50
WINDOW clause (recap)	Ch 28
window function	Ch 13
window ORDER BY	Ch 13
winner's curse	Ch 73
winsorizing	Ch 30
WITH	Ch 13
With	Ch 19

Term	Taught in
word embedding	Ch 41
word2vec	Ch 41
work or school account	Ch 16
work-in-progress limit	Ch 26
Workbook	Ch 19
workbook	Ch 10
worker (executor)	Ch 48
workflow	Ch 26
workflow trigger	Ch 52
working capital	Ch 23
working directory	Ch 20, Ch 26
working folder (current directory)	Ch 17
working memory	Ch 15
working table (previous pass)	Ch 28
Worksheet	Ch 19
worksheet (sheet)	Ch 10
workspace	Ch 16
wrap-around gap	Ch 78
write-ahead log (WAL)	Ch 45

Term	Taught in
write– audit– publish	Ch 77
write– audit– publish (WAP)	Ch 47
written against verbal offer	Ch 81
WSL	Ch 34

X

Term	Taught in
XGBoost	Ch 37
XLOOKUP	Ch 70
.xlsb	Ch 19
.xlsm	Ch 19
xlsxwriter	Ch 18
XML	Ch 2
XOR problem	Ch 43
$\bar{x}$ (sample mean)	Ch 21

Y

Term	Taught in
YAML	Ch 26
Yates' continuity correction	Ch 22, Ch 73
year-to- date (YTD)	Ch 13
your own summit	Ch 83

Z

Term	Taught in
z-score	Ch 21, Ch 35
Zapier	Ch 20
zip	Ch 17
zoneinfo	Ch 20
zoom level	Ch 60
zsh	Ch 34

M

Term	Taught in
μ (population mean)	Ch 21

Σ

Term	Taught in
Σ (sum)	Ch 21